

Homomorphic Secret Sharing with Verifiable Evaluation

Arka Rai Choudhuri¹, Aarushi Goel², Aditya Hegde², and Abhishek Jain^{1,2}

¹Nexus. arkarai.choudhuri@gmail.com

²Purdue University. aarushi@purdue.edu

³Johns Hopkins University. {ahegde,abhishek}@cs.jhu.edu

⁴NTT Research. abhishek.jain@ntt-research.com

Abstract

A homomorphic secret sharing (HSS) scheme allows a client to delegate a computation to a group of untrusted servers while achieving input privacy as long as at least one server is honest. In recent years, many HSS schemes have been constructed that have, in turn, found numerous applications to cryptography.

Prior work on HSS focuses on the setting where the servers are semi-honest. In this work we study HSS in the setting of malicious evaluators. We propose the notion of HSS *with verifiable evaluation* (ve-HSS) that guarantees correctness of output *even when all the servers are corrupted*. ve-HSS retains all the attractive features of HSS and adds the new feature of succinct public verification of output.

We present *black-box* constructions of ve-HSS by devising generic transformations for semi-honest HSS schemes (with negligible error). This provides a new non-interactive method for verifiable and private outsourcing of computation.

Contents

1	Introduction	1
1.1	Our Results	2
1.2	Applications	5
1.3	Related Work	6
2	Technical Overview	7
2.1	Single Client HSS with Verifiable Evaluation	7
2.2	Groth16 zkSNARKs [Gro16] are Splittable	12
2.3	Splittable Zero-Knowledge BARGs	14
2.4	Multi-Client HSS with Verifiable Evaluation	17
2.5	Applications	18
3	Preliminaries	20
3.1	Homomorphic Secret Sharing	20
3.1.1	HSS Schemes from Prior Works	23
3.2	Succinct Non-interactive Arguments	24
4	Splittable zkSNARG	26
4.1	Distributed Prover Setting	28
4.2	The Groth16 zkSNARK	30
4.3	The Waters-Wu zkBARG	34
5	HSS with Verifiable Evaluation	41
5.1	Instantiations	48
6	Multi-Client HSS with Verifiable Evaluation	50
6.1	Construction With Strong Privacy	54
6.2	Simpler Construction Without Strong Privacy	61
6.3	Instantiations	62
7	Applications	63
7.1	Private Delegation of Computation	63
7.2	Collaborative zkSNARKs	64

1 Introduction

In a t -private homomorphic secret sharing (HSS) scheme [BGI16a], a client uses a sharing algorithm to split a secret x into multiple shares such that any subset of at most t shares do not reveal anything about x . Such a scheme additionally supports homomorphic computation on the shares via the following algorithms:

- A local evaluation algorithm Eval that given a function f , maps a share x_i to an output share y_i .
- A reconstruction algorithm Rec that computes the output $y = f(x)$ from the output shares $\{y_i\}_i$.

Similar to additive secret sharing schemes, the reconstruction algorithm Rec is simply an additive function over the output shares. This in turn implies that the size of each share output by Eval is *succinct* and only depends on the output length of f , and is otherwise independent of the description of f .

A t -private HSS allows a client to delegate a (possibly resource-intensive) computation to a group of untrusted servers while achieving input privacy against any t number of servers.¹ This feature, in turn, has many applications in cryptography, including sublinear-communication secure multiparty computation (MPC) [BGI16a, Cou19, CM21, DIJL23], multi-server private information retrieval [GI14, BGI16a], correlated-randomness generators [BCGI18, BCG⁺19], reusable MPC [BGMM20], constrained pseudorandom functions [CMPR23], and more. By now, several constructions of HSS are known from standard group-based [BGI16a, BGI17, BCG⁺17, OSY21, RS21, ADOS22], lattice-based [DHRW16, BKS19] and code-based assumptions [BCG⁺19, CM21, DIJL23]. Some of these schemes differ in the number of servers they can support (two or more), the class of functions that can be evaluated, as well as whether or not they have (non-negligible) correctness error.

Prior work in this area focuses on semi-honest servers who perform the local evaluations honestly (see Section 1.3 for some exceptions). This means, in particular, that the output is only guaranteed to be correct when the servers are honest. In this work, we study HSS in the setting where the servers may be *malicious* and can deviate from the honest evaluation strategy. In particular, the new property we seek is public verifiability of the output by any external auditor, even when all the servers are malicious. Viewed from the lens of delegation of computation, we seek *verifiable* and private delegation of computation.

HSS with Verifiable Evaluation. We propose a new notion of HSS *with verifiable evaluation* (ve-HSS) that retains the attractive features of HSS and adds public verifiability to the output of the computation. Similar to (semi-honest) HSS, a ve-HSS scheme is equipped with sharing, Eval and Rec algorithms that provide the same functionality and efficiency properties. Since the evaluators may be malicious, the reconstruction algorithm Rec in ve-HSS produces a “candidate” output. We require an additional algorithm Verify that either accepts or rejects this output. Importantly, verification is *succinct*, namely, Verify runs in time independent of the complexity of the evaluated function and the number of servers.

We first augment the HSS privacy notion to require that the privacy of the client’s input should hold even if the malicious servers learn whether the candidate output produced was accepted or rejected by the client. This augmentation is necessary since malicious servers may produce maliciously computed shares, and attempt to glean information based on the client behavior. As in the case of HSS, we parameterize the privacy requirements by the number of corrupt servers t .

Motivated by the application to verifiable delegation of computation, we consider the following notions of soundness:

- *Local Soundness:* This notion guarantees that for any shared input $x = (x_{\text{pub}}, x_{\text{priv}})$ — where x_{pub} is the public input and x_{priv} is the private input known only to the client — the Verify algorithm, given x , accepts a candidate output y if and only if y is the result of evaluating the function f on x .

¹This is analogous to fully homomorphic encryption (FHE) which allows private delegation of computation to a single server.

- *Public Soundness*: This notion guarantees that for any shared input $x = (x_{\text{pub}}, x_{\text{priv}})$, the Verify algorithm, given x_{pub} , accepts a candidate output y if and only if $y = f(x_{\text{pub}}, \cdot)$ for “some” private input.

Crucially, we require these soundness properties to hold *even when all the servers are corrupted*. This means that an external auditor can publicly verify the correctness of the output without knowing anything about (or placing any trust assumptions in) the entities who computed the output. This property makes our notion *stronger* than standard secure multiparty computation (MPC), which can only guarantee output correctness when at least one party is honest.

It is easy to see that ve-HSS with succinct public verification implies succinct non-interactive arguments (SNARGs) [Mic94]. Indeed, the local soundness property implies SNARGs for deterministic computations while public soundness implies SNARGs for non-deterministic computations. The class of languages supported by the SNARG is determined by the family of functions supported by the ve-HSS scheme.

Multi-Client HSS with Verifiable Evaluation. We also study verifiability for *multi-client* HSS, where a set of mutually distrusting clients can share their inputs to a set of servers who can then perform joint evaluations on the inputs of the clients. The notions of privacy and soundness in this setting are strengthened to allow for collusions between the clients and servers. In particular, local soundness now allows each client to verify whether its private input x_i was used in the computation, i.e., $y = f(\cdot, x_i, \cdot)$ even if all the remaining clients and all the servers are corrupted. Public soundness, as before, guarantees that the output was obtained by evaluating f on the fixed public input and “some” set of private inputs, i.e., $y = f(x_{\text{pub}}, \cdot, \cdot)$, even if all parties are corrupt.

1.1 Our Results

In this work, we study HSS with verifiable evaluation with a focus on building solutions that make only *black-box* use of cryptography. We obtain new results both for the single-client and multi-client settings.

Single Client HSS with Verifiable Evaluation. We present two sets of results for single-client homomorphic secret sharing (HSS) with verifiable evaluation: one in the generic group model and the other in the standard model. Below, we present the results for each model separately, starting with our result in the generic group model.

Theorem 1 (Generic Group Model, Single Client, Informal). *In the generic group model, we construct a single client m -server HSS with verifiable evaluation that only makes black-box use of cryptography:*

- For NC¹: from (i) class groups assumptions; (ii) Decisional Composite Residuosity (DCR) assumption; or (iii) Learning with Errors (LWE) assumption.
- For P/poly: from (i) the security of indistinguishability obfuscation (iO) + one-way functions (OWF); or (ii) security of fully homomorphic encryption (FHE).

In the standard model, we consider boolean circuits that are SIMD (single instruction multiple data). We obtain analogous results as in the generic group model, but in the SIMD setting.

Theorem 2 (Standard Model, Single Client, Informal). *In the standard model, we construct a single client m -server HSS with verifiable evaluation that only makes black-box use of cryptography:*

- For SIMD NC¹: from the subgroup decision assumption in addition to either (i) class groups assumptions; (ii) Decisional Composite Residuosity (DCR) assumption; or (iii) Learning with Errors (LWE) assumption.
- For SIMD P/poly: from the subgroup decision assumption in addition to either (i) the security of indistinguishability obfuscation (iO) + one-way functions (OWF); or (ii) security of fully homomorphic encryption (FHE).

Main Compiler. The results stated above for the single-client setting are based on currently known results for semi-honest HSS. However, we derive these results using a general compiler that takes as its starting point any standard semi-honest HSS scheme with negligible correctness error. This means future constructions of such HSS schemes can be plugged into our compiler to obtain new instantiations.

At a high level, our compiler adds verifiability to a semi-honest HSS scheme by requiring the servers to additionally compute a zero-knowledge succinct non-interactive arguments (zkSNARGs) of correct computation *under the hood* using the semi-honest HSS, i.e. the zkSNARG is computed in a distributed manner by all the servers. In order to ensure that the compiler only makes black-box use of cryptography, we enforce certain structural (and security) constraints on the zkSNARGs. We formalize these properties via a new notion of zkSNARGs that we call *splittable* zkSNARGs.

We will describe the security properties shortly, but the primary ‘structural’ properties we require from such a splittable zkSNARG is that the computation of the zkSNARG proof can be divided into an initial *non-linear* computation phase over all of its input, followed by a computation phase that exclusively performs *linear* operations on the output of the non-linear phase. Importantly, we further require that the non-linear phase is non-cryptographic. We make this distinction since in our compiler the non-linear phase will be computed via the (semi-honest) HSS scheme, and the linear phase will be computed on the additive shares obtained from the aforementioned HSS evaluation.

With our new tool splittable zkSNARGs, we present below our main theorem statement: a compiler for transforming any semi-honest single-client HSS scheme with at most negligible correctness error into a single-client HSS with verifiable evaluation (ve-HSS).

Theorem 3 (Main Compiler, Informal). *Let HSS be a single-client m -server HSS scheme for a class of circuits $\mathbb{C}$ with negligible correctness error. Then given a splittable zkSNARG for $\mathbb{C}'$ where the non-linear computation phase can be implemented using a circuit in $\mathbb{C}$, there exists a single client m -server HSS with verifiable evaluation ve-HSS for circuit class $\min\{\mathbb{C}, \mathbb{C}'\}$, that only makes black-box use of cryptography.*

Here $\min\{\mathbb{C}, \mathbb{C}'\}$ simply refers to the smaller class among the two circuit classes. We present instantiations of our theorem in the table below using single-client HSS schemes that have negligible correctness error.

Splittable zkSNARG		HSS		ve-HSS
Assumption	Circuit Class $\mathbb{C}'$ (implementable in $\mathbb{C}$)	Assumption	Circuit Class $\mathbb{C}$	Circuit Class $\min\{\mathbb{C}, \mathbb{C}'\}$
Generic Group Model [Gro16]	NC^1 (in NC^1)	LWE [BKS19]	NC^1	NC^1
		DCR [OSY21, RS21]		
		Class Groups [ADOS22]		
	P/poly (in P/poly)	iO and OWF [BGI15] FHE [DHRW16, BGI ⁺ 18]	P/poly	P/poly
Subgroup Decision Assumption [WW22]*	SIMD NC^1 (in NC^1)	LWE [BKS19]	NC^1	SIMD NC^1
		DCR [OSY21, RS21]		
		Class Groups [ADOS22]		
	SIMD P/poly (in P/poly)	iO and OWF [BGI15] FHE [DHRW16, BGI ⁺ 18]	P/poly	SIMD P/poly

Figure 1: We present our results for the single client setting, where the phrase *implementable in $\mathbb{C}$* refers to the fact that the non-linear computation of the splittable zkSNARK is implementable by a circuit in $\mathbb{C}$. The assumption for the constructed ve-HSS scheme is the combination of the assumption for the splittable zkSNARG and the corresponding HSS scheme. [WW22]* indicates that we use our extended zkBARG to instantiate the compiler. LWE - learning with errors, DCR - decisional composite residuosity, OWF - one way functions, iO - indistinguishability obfuscation, FHE - fully homomorphic encryption.

We describe next our results for the aforementioned tool of splittable zkSNARGs, needed to instantiate our compiler.

Splittable zkSNARGs. We have already described the structural requirements for splittable SNARGs, where the computation of the proof can be split into non-linear and linear phases of computation. For security, in addition to the regular soundness properties of zkSNARGs, we require a splittable zkSNARG to have *robust verification*, i.e. the verifier will reject all proofs that contain additive errors. We defer a formal definition of this notion to later. We present two instantiations of splittable zkSNARGs that can be used to instantiate our compilers.

Splittable zkSNARGs for NP. Our first instantiation supports general NP languages and is based on the Groth16 zkSNARK² [Gro16] that achieve perfect zero-knowledge and computational soundness (proven in the generic group model). We show that the Groth16 zkSNARKs satisfy all the properties of a splittable zkSNARG.

Moreover, we show that the non-linear computation phase in the Groth16 zkSNARK requires evaluating a circuit of depth $O(D)$ (over a $\text{poly}(\lambda)$ -bit integer ring) if the relation circuit has depth D .

Splittable zkSNARGs for batch-NP. Our second instantiation supports batch-NP languages and can be based on *standard assumptions*. Recall that while SNARGs for all NP languages are not known based on standard assumption, SNARGs for batch-NP (also referred to as BARGs) [BHK17, KPY19, CJJ21] are known from standard assumptions [CJJ21, CJJ22, HJS22, WW22, CGJ⁺23]. BARGs allow a prover to convince a verifier of the validity of k NP statements using a proof that is significantly smaller than the combined witness of the k statements. Our splittable zkSNARG for batch-NP is based on the scheme of [WW22] that uses composite-order pairing groups and achieves computational soundness under the subgroup-decision assumption.

We first show how to lift the BARGs of [WW22] to achieve the *perfect zero-knowledge* property. Our construction only makes *black-box* use of cryptography.

Theorem 4 (Black-box Perfect Zero-knowledge BARGs, Informal). *Assuming the hardness of the subgroup decision problem in a composite-order pairing group, there exists a BARG for boolean circuit satisfiability that achieves perfect zero-knowledge and makes only black-box use of the group. The proof size is $\text{poly}(\lambda, |C|)$, the CRS size is $m^2 \cdot \text{poly}(\lambda)$ and the verification complexity is $\text{poly}(\lambda, |C|) + \text{poly}(\lambda, m, n)$, where λ is a security parameter, $C : \{0, 1\}^n \times \{0, 1\}^h \rightarrow \{0, 1\}$ is the Boolean circuit, n is the statement size, and m is the number of instances.*

We note that, in general, zkBARGs can be obtained by compiling BARGs with non-interactive zero knowledge proofs. Furthermore, recent works [CW23, BKP⁺23, BWW23] have shown how to construct NIZKs from BARGs, which in turn, yields zkBARGs from BARGs. While these transformations are generic, they make *non-black-box* use of cryptography in several steps. Theorem 4, in contrast, yields a new and direct construction of zkBARGs based on pairing groups that makes black-box use of the group.

We additionally demonstrate that our zkBARG construction obtained in the above theorem is indeed a splittable zkSNARG for batch-NP. We shall use this to construct ve-HSS schemes that can evaluate SIMD boolean circuits. In particular, we show that when computing the zkBARG for any SIMD computation, the non-linear computation phase can be computed by a circuit of depth $O(D)$ (over a $\text{poly}(\lambda)$ -bit integer ring), where D is the depth of the SIMD computation.

Instantiating our compiler in Theorem 3 using (i) semi-honest HSS schemes for NC¹ (assuming DCR [OSY21, RS21] or LWE [BKS19] or class groups [ADOS22]) and P/poly (assuming iO + OWF [BGI15] or FHE [DHRW16, BGI⁺18]); and (ii) splittable zkSNARGs for NC¹ and P/poly (in the generic group model)

²We use zkSNARK instead of zkSNARG when the underlying scheme is also an *argument of knowledge*.

and SIMD NC¹ and SIMD P/poly (assuming hardness of the decision subgroup problem) give us our main results in Theorems 1 and 2.

Multi-Client HSS with Verifiable Evaluation. Finally, we extend our results to the *multi-client* setting. Starting from semi-honest public-key HSS schemes³ with negligible error, we present a compiler for public-key multi-client ve-HSS using Splittable zkSNARGs and obtain the following results:

1. *Semi-Honest Clients:* When clients are semi-honest, we present a compiler for transforming any public-key HSS scheme with at most negligible correctness error into a multi-client ve-HSS using a suitable Splittable zkSNARG, assuming a secure “public-key setup”.

When instantiated with HSS schemes based on DCR [OSY21, RS21] and class group [ADOS22], our compiler yields a fully black-box solution for multi-client ve-HSS, where the public key setup can either be generated by a trusted third party or jointly by the servers using a black-box one-round maliciously secure distributed protocol from [ADOS22]. When using other HSS schemes to instantiate our compiler, the setup must be done by a trusted entity. This is because black-box protocols for public-key setup are not known for these other HSS schemes.

2. *Malicious Clients:* When the clients are malicious, we instantiate our compiler using HSS schemes based on DCR [OSY21, RS21] and class groups [ADOS22].

If some clients are malicious, it is crucial to ensure that they send well-formed HSS shares of their respective inputs. In addition to the distributed key setup protocol from [ADOS22], in this setting, we also use the black-box, one-round, maliciously secure, input sharing protocol for these HSS schemes from [ADOS22] to instantiate our compiler.

1.2 Applications

We now discuss some applications of our ve-HSS constructions.

Private Delegation of Computation. Our notion of ve-HSS provides a black-box solution to the problem of private and verifiable delegation of computation (by a single or multiple clients) to a group of untrusted servers. Our HSS-based approach offers a significant advantage over the FHE-based approaches explored in [GGW24, ACGS23]. In the FHE-based approach, to make the final proof publicly verifiable, the client must perform some post-processing on the output sent by the server. In contrast, our approach yields a truly non-interactive solution, where the output computed by the servers is already publicly verifiable and requires no additional intervention from the client. To the best of our knowledge, this yields the first such non-interactive and black-box solution for delegating the computation of *non-cryptographic functions*.

Next, we demonstrate that our approach can also be used to delegate the computation of some *cryptographic functions*, while also being black-box in the cryptographic operations involved in the computation of these functions. Specifically, we highlight the application of our constructions to the problem of privacy-preserving delegation of zkSNARG computations in a black-box manner — an area that has recently garnered significant interest [OB22, GGJ⁺23, CLMZ23, LZW⁺24, GGW24].

Private Delegation of zkSNARK Computation. Let $\mathcal{R} = \{x, w \mid C(x, w) = 1\}$ be an NP relation defined by a circuit C . Consider a client who wishes to compute a proof for the statement x using a particular zkSNARK proof system Π . We show how if Π is a splittable zkSNARG, then the client can use our single-client ve-HSS scheme — built using Π itself — to outsource this proof computation.

A few remarks are in order: first, we note that our group-based ve-HSS schemes can only support NC¹ circuits (a limitation of known semi-honest HSS schemes). Second, the above solution offers several

³There are secret-key and public-key variants of HSS schemes. In the public-key variant, anyone with access to the public-key can generate shares of their secret whereas in the secret-key variant one needs to know the key to generate secret shares.

advantages over recent works [GGJ⁺23, CLMZ23, LZW⁺24, GGW24] that employ general-purpose MPC techniques (or FHE) for private-outsourcing of zkSNARKs: (i) MPC-based approaches [GGJ⁺23, CLMZ23, LZW⁺24] several rounds of the interaction between the servers, with communication growing with the size of the circuit C . In contrast, our approach necessitates *zero* interaction amongst the servers. (ii) As discussed above, unlike the FHE-based approach in [GGW24], the proof computed by the servers in our solution is already publicly verifiable and requires no additional intervention from the client. Moreover, unlike [GGW24], the final zkSNARK generated using our delegation framework does not reveal to the verifier whether it was generated by the client himself or using a delegation scheme.

Private Delegation of Collaborative zkSNARK Computation. A recent work by Ozdemir and Boneh [OB22] introduced the notion of *collaborative zkSNARKs* as a general tool for publicly auditable privacy-preserving computations. Collaborative zkSNARKs is a distributed protocol enabling a set of parties, each holding a share of an NP witness, to collectively compute a zero-knowledge succinct non-interactive argument (zkSNARK) for a given statement. Using similar ideas as in the single-client setting, our construction of multi-client ve-HSS can be used by a group of clients to delegate the generation of a *splittable zkSNARK* to a group of untrusted servers.

Ozdemir and Boneh’s approach requires communication proportional to the size of the NP relation circuit. In contrast, a key advantage of our approach is that it requires *minimal interaction* between the servers, which are only needed for reconstructing the proof. The clients, meanwhile, only need to interact when computing and transmitting HSS shares of their combined witness to the servers. After this, the servers can independently compute the collaborative zkSNARK.

We refer the reader to Section 2.5, for a detailed overview of these applications.

1.3 Related Work

We provide a brief overview of related work.

Verifiable Function Secret Sharing. In a verifiable function secret sharing scheme [BGI16b, BBC⁺21, dCP22], the *client* may be malicious, and the servers want the guarantee that the *function shares that they received from the client are well-formed*, thus preventing any attempt by the client to learn unauthorized information about the servers’ database. This is quite different from our notion of verifiability that aims to establish the *correctness of evaluation* in the face of malicious servers.

Maliciously Secure Sublinear MPC Based on HSS. A recent work by Abram, Damgård, Orlandi and Scholl [ADOS22] presents a black-box construction of maliciously-secure sublinear MPC using HSS. Specifically, they construct two-round sublinear MPC with a public-key setup from DCR-based HSS schemes [OSY21, RS21] as well as new HSS schemes based on class groups. The public-key setup can be instantiated with a one round distributed protocol (using a trusted setup for RSA parameter generation in the DCR case).

By virtue of being a maliciously-secure MPC protocol, their protocol guarantees correctness of output as long as at least one of the servers is honest. However, it does *not* imply our notion of HSS with verifiable evaluation which requires local and public soundness properties even when all parties are corrupted.

Homomorphic Secret Sharing with Verifiability. Some prior works [TLM18, TM19, TBM20, YO19, CZ21, CZ20, ZW22, Che23, HZ20, MTG22] have previously considered different notions of verifiability in homomorphic secret sharing (under the umbrella term “verifiable HSS”) for limited class of functions. The constructions proposed in [TLM18, TM19, TBM20] are only for computing a sum or product of all the clients’ inputs and the clients are always assumed to be honest. While these results claim to achieve soundness (or verifiability) even when all servers are corrupt, they were later shown to be broken by [HZ20, MTG22]. None of the other results [YO19, CZ21, CZ20, ZW22, Che23] consider soundness when

all servers are corrupt. [YO19, CZ21] present HSS with verifiability of degree- d multiplications, where d depends on the number of servers. Verifiable HSS schemes in [CZ20, ZW22, Che23] are based on LWE and are designed for different degree polynomials (where the degree of the polynomial is dependent on the number of servers or the security parameter).

2 Technical Overview

As outlined in Section 1, our goal in this work is to design a ve-HSS scheme that uses cryptographic operations in a *black-box* manner. In this section, we discuss the main ideas underlying our result.

2.1 Single Client HSS with Verifiable Evaluation

We now describe the ideas underlying the construction of our *HSS with verifiable evaluation*. Throughout this discussion, we will operate in a simplified setting, assuming that there are only two servers⁴. These ideas can be easily generalized to settings with more than two servers.

Strawman Approach. A ve-HSS scheme proposes to add “verifiability” to existing HSS schemes. Within Cryptography, zero-knowledge proofs [GMR89] are the most widely used tool for adding “verifiability” to any computation. The celebrated result by Goldreich, Micali, Wigderson (GMW) [GMW87] showcased how any semi-honest multiparty computation protocol can be upgraded to withstand malicious corruptions, by requiring each participant to include a zero-knowledge proof (to attest to their honest behavior) with every message they send. Similarly, the folklore way to add verifiability to delegation of computation via FHE involves asking the server to additionally compute a zero-knowledge proof attesting that the FHE evaluation was done honestly.

This motivates the following natural (GMW-inspired) approach to adding verifiability to existing HSS constructions: the servers each compute a zero-knowledge proof alongside their outputs from the evaluation algorithm, certifying that they executed the evaluation algorithm honestly. Since the evaluation phase must be non-interactive and its output must be sublinear in the size of the function being evaluated, the servers can employ a zero-knowledge proof that is also a succinct non-interactive argument (SNARG) [Kil92, Mic94] that admits a succinct proof and sublinear verification time. This approach, however, inherently requires non-black-box use of cryptography. Specifically, since the evaluation algorithms of known HSS constructions rely heavily on cryptographic operations, it is unclear if the servers can compute a zero-knowledge SNARG (zkSNARG) to prove the honest execution of these operations without employing non-black-box techniques.

We remark that, other malicious MPC techniques [DPSZ12, GIP⁺14, CGH⁺18] involving information-theoretic MAC based checks⁵ or the use of distributed zero-knowledge proofs [BBC⁺19, BGIN19, BGIN20, BGIN21] to verify correctness of computation are also not helpful in our setting. The soundness argument in these approaches hold only if a subset of the servers are allowed to be corrupt. In contrast, as discussed in Section 1, we demand stronger notions of soundness, that hold even if *all* servers are corrupt.

Our Starting Idea. In order to avoid such non-black-box use of the cryptography, our initial insight is to have the *servers jointly compute a single SNARG proof to prove that the computation is correct*. In particular, instead of proving that each server performed the evaluation honestly, this is a proof attesting that the reconstructed output y is indeed a correct result of evaluating function f on input x .

⁴Existing constructions of HSS [BKS19, OSY21, RS21, ADOS22, CM21] that meet our criteria of correctness (and support more than constant-depth functions, without relying on heavy tools such as iO and FHE) only support two servers. Other known construction of Multiparty HSS [DIJL23] only satisfy weak correctness (which is not sufficient for us).

⁵An information-theoretic MAC based approach is also used in the design of maliciously secure sublinear MPC based on HSS [ADOS22].

Now, we need to determine how the servers can compute such a proof. Our second insight is to *utilize HSS itself to compute this proof*. In other words, rather than generating the proofs “externally” to validate the correctness of HSS Eval, we want to use HSS Eval itself to generate the joint proof. We would like to rely on the self-authenticating property of SNARGs to prevent incorrect outputs from being accepted. Implementing this idea, however, is not straightforward:

- A naïve implementation of this approach would entail non-black-box use of the cryptographic operations involved in computing the underlying SNARG proof, contrary to our requirements.
- To prove the security of the resulting ve-HSS scheme, the simulator/ideal adversary in the ideal world must be able to abort the output if a malicious server misbehaves. The standard method to simulate aborts in the GMW paradigm is to check individual proofs sent by the corrupt parties. However, in this approach, since we no longer require servers to attach individual proofs, it is not immediately clear how one would detect and simulate aborts in the ideal world.

Note that even if we are willing to forgo the black-box requirement, implementing the aforementioned idea using non-black-box cryptographic operations of the underlying SNARGs presents a significant obstacle. This is because soundness of all known constructions of SNARGs for NP [Mic94, Gro16, GWC19, Gro10, Lip12, DFH12, GGPR13, BCI⁺13, BCCT13, BCC⁺17] are proven in idealized models such as the random oracle model (ROM), generic group model (GGM), or the algebraic group model (AGM). Employing the above non-black-box approach for SNARGs based on ROM or GGM would inevitably require non-black-box descriptions of these oracles. For SNARGs based on AGM, it remains unclear if the soundness of the SNARG computed using HSS (which also relies on cryptographic assumptions) can still be established in the AGM model, or if it requires making additional hardness assumptions.

Consequently, a black-box approach appears to be the only viable option for instantiating the above idea. However, it is not immediately clear whether existing HSS schemes can support computation of a SNARG inside them, while still being black-box in the underlying cryptographic operations. Nevertheless, this idea of computing SNARGs using HSS Eval forms the starting point of our approach, and we develop new ideas to overcome these challenges.

Splittable SNARGs. Since all SNARGs rely on cryptographic hardness assumptions, generating a SNARG proof for arithmetic relations involves two types of operations: cryptographic and non-cryptographic (i.e., simple field arithmetic). To compute the SNARG proof within HSS (while being black-box), our next insight is to make use of HSS Eval for the field arithmetic, and devise a separate method for the servers to compute the necessary cryptographic operations *outside* of HSS. However, since these two types of operations within SNARG generation may be highly intertwined, it may not be always be feasible to compute the cryptographic operations in a distributed and non-interactive manner. As such, it is unclear if it is even possible to implement this idea.

Our main contribution lies in showing that this idea is indeed feasible to implement. All we require is for the SNARG at hand to satisfy the following “split-prover” property. Namely, it must be possible to clearly divide the generation of splittable SNARG proofs into two steps:

1. *Low Depth Non-Cryptographic Operations:* The first step must only entail non-cryptographic operations over a ring. The computational depth of this step should be such that this overall computation is within the class of functions that are currently supported by existing HSS constructions.⁶
2. *Linearly Distributable Cryptographic Operations:* The second step of the computation may involve cryptographic operations. However, these operations must be feasible to execute in a distributed manner

⁶Most existing HSS schemes support evaluating a limited class of circuits e.g., branching programs [BG16a, BG17, BCG⁺17, DIJL23, FGJS17, BKS19, OSY21, RS21, ADOS22] and low-degree multi-variate polynomials [BCG⁺19, COS⁺22, CM21, LMS18, ILM21, FKW22, DIJL23].

without requiring interaction. Specifically, considering that the first step is computed using HSS Eval, the servers will possess additive shares of the output from this phase. Using these additive shares, each server should be capable of independently performing certain operations and generating a partial SNARG proof. We will sometimes refer to these partial proofs as shares of the proof. Importantly, there must exist an efficient and simple algorithm to reconstruct the final proof using the partial proofs computed by all servers.

We refer to such SNARGs that satisfy these two properties as *splittable SNARGs*. Looking ahead, we will also require splittable SNARGs to satisfy a third property - a weaker form of the well-studied notion called *unambiguity* [RRR21]. We discuss this in more detail towards the end of this subsection. In subsequent subsections, we demonstrate how some existing SNARGs do satisfy these properties.

Candidate Approach. Given a splittable SNARG, as described above, and any HSS scheme with negligible correctness error, the servers run the evaluation algorithm in our ve-HSS scheme as follows:

- Given HSS shares of the input (say $x = (x_{\text{priv}}, x_{\text{pub}})$), the servers execute the evaluation algorithm of the underlying HSS scheme (say HSS.Eval) to compute additive shares of $y = f(x)$.
- Let $\mathcal{R}_{\text{local}} : \{(f, x, y) \mid \text{st}, f(x) = y\}$ be a deterministic relation, i.e., $\mathcal{R}_{\text{local}} \in \mathcal{P}$. Using HSS shares of x , the servers use HSS Eval to compute additive shares of the output of the first part of the computation (i.e., involving low depth non-cryptographic operations) required for generating a SNARG for the relation $\mathcal{R}_{\text{local}}$ ⁷.
- Using additive shares of x, y and additive shares of the outcome of the previous computation, the servers then perform the second step of the computation (i.e., linearly distributable cryptographic operations) in SNARG generation to obtain shares of the final SNARG proof.
- The servers send additive shares of y as well as shares of the SNARG proof to the client.

Finally, the client can reconstruct these shares, verify the proof (say π_{local}) w.r.t., x, y and accept y as the correct output only if this proof is successfully verified. It is easy to see that if the servers do not have access to the shares of the output or the proof and do not learn whether or not this proof verified, this approach trivially achieves privacy against malicious servers. This is because, in this non-interactive protocol, the view of the malicious server only consists of the HSS share of x that was provided to it in the input sharing phase. From the privacy guarantee of the underlying HSS scheme, it follows that this share computationally hides the input x . Moreover, local soundness of this construction follows from the soundness of the underlying SNARG.

Public Soundness. While the above approach suffices for local soundness, as previously discussed, we require the vHSS scheme to also satisfy a notion of public soundness. Moreover, we require a stronger notion of privacy, where the servers can learn the output as well as the outcome of the honest client's verification. Recall that public soundness necessitates that upon observing the output of the evaluation phase, anyone (not solely the client) can be convinced that there exists some x_{local} such that y is a valid output of f given public input x_{pub} . To accomplish this, we can modify the aforementioned protocol as follows: in addition to providing a "local proof" Π_{priv} for the client, as described previously, the servers will also compute a "publicly verifiable SNARG" (say Π_{pub}) for the relation $\mathcal{R}_{\text{pub}} = \{(f, x_{\text{pub}}, y) \mid \exists x_{\text{priv}}, \text{ s.t., } f(x_{\text{pub}}, x_{\text{priv}}) = y\}$ in a similar manner. Shares of Π_{local} will only be sent to the clients, while the shares of the output y and this publicly verifiable proof Π_{pub} will be published publicly. The client will accept the output if and only

⁷Note that given x , since the computation of $f(x)$ is deterministic, we do not need to start with HSS shares of y for this computation. It can be assumed to be implicitly computed as part of this computation.

if both Π_{local} and Π_{pub} verify. Note that unlike $\mathcal{R}_{\text{local}}$, $\mathcal{R}_{\text{pub}}$ is not in $\mathcal{P}$. Therefore, while it suffices to use a SNARG for $\mathcal{P}$ for the computation of Π_{local} , we crucially require a SNARG for $\mathcal{NP}$ for the computation of Π_{pub} .

Privacy. Recall that our privacy notion requires that the view of the malicious server, consisting of its input share and the decision of the Verify algorithm, can be simulated solely based on the function output without the client’s input. This mirrors the requirement in any MPC protocol achieving security with abort. This strong privacy guarantee is crucial to prevent an adversary from causing selective aborts based on x_{priv} (see [LP11], for a detailed discussion on this issue in MPC protocols). An observant reader might notice that achieving this notion of privacy is not as straightforward as before. This presents the following challenges:

Challenge 1: Since we allow for malicious corruptions of one of the servers, the shares of the candidate y and Π_{pub} computed in the evaluation phase are not guaranteed to have been honestly computed.⁸ This *incorrect* output and proof could potentially leak information about the private input.

Addressing this Challenge: We observe that unlike in an interactive protocol, where the computation performed by the honest parties can be influenced by the strategy adopted by the adversary, this is not feasible in a non-interactive protocol. Since our evaluation phase is non-interactive, the shares of y and Π_{pub} output by the honest party, cannot be influenced by the adversary. Therefore, an incorrect output y —which can only result from the adversary sending an incorrect share of y —does not compromise privacy.

Furthermore, since Π_{pub} is publicly revealed, ensuring privacy necessitates the use of a zero-knowledge SNARG (zkSNARG) to generate Π_{pub} . We assume that the randomness used to generate the zkSNARG Π_{pub} is sampled and HSS-shared by the client with the servers in the input sharing phase. Given HSS shares of this randomness, the computation carried out by the servers in the evaluation phase is deterministic.

Challenge 2: A rushing adversary might opt to wait for the honest server to publish their additive share of y and Π_{pub} , and then adjust their output shares of y , Π_{local} and Π_{pub} , accordingly. To prove security, we require the simulator/ideal adversary in the ideal world to be able to efficiently simulate the joint view of the adversarial server and the output of the honest client. However, if the adversary chooses to transmit malformed shares of the proofs Π_{local} , Π_{pub} (while still providing an honestly computed share of y), it becomes unclear how the simulator in the ideal world can accurately predict whether the resulting reconstructed proofs will verify with respect to x_{priv} , x_{pub} and an honestly computed y (i.e., it cannot simulate the output of the honest client).

Note that since Π_{pub} is publicly reconstructed, the simulator in the ideal world can reconstruct this proof on its own and check whether it verifies w.r.t. the reconstructed y . However, since it does not have x_{local} (i.e., the input of the honest client), the same strategy cannot be used for deciding the outcome of Π_{local} , which is verified w.r.t. x_{local} .

Addressing this Challenge: As discussed earlier, this issue does not arise in a GMW-style approach [GMW87], where each server provides an individual proof confirming honest behavior. This is because each of these proofs can be independently verified with respect to a *public* state. In our setting however, we have a jointly computed proof that needs to be verified w.r.t. a *private* state. Nevertheless, in order to simulate the outcome of Verify, we would like our simulator to perform a similar local verification based on the proof share of the corrupt server.

To enable this, we require the splittable zkSNARG used for generating Π_{local} to satisfy an additional property, which we term *distributed-prover robust verification (DP-robust verification)*. Informally speaking, this property dictates that given a fixed (and honestly generated) share of Π_{local} computed by the honest

⁸Since our evaluation phase is non-interactive, the servers have no way to verify that this output and proof were honestly generated before reconstructing them.

server (using HSS shares of x_{priv}), a computationally bounded malicious server can (w.h.p.) only find a unique share of Π_{local} , such that the reconstructed Π_{local} verifies with respect to x_{priv} , y and x_{pub} . Using this property, the simulator in the security argument can determine the output of the honest client based on whether (1) Π_{pub} verifies; and (2) the share of Π_{local} provided by the adversarial server matches the one it should have computed given its share of x_{priv} .

Summary. We now summarize how we prove security of our proposed construction. As discussed earlier, local and public soundness follow from the soundness of the underlying zkSNARGs. To prove privacy, our simulator in the ideal world will proceed as follows:

1. The simulator queries the ideal functionality to learn y . In this non-interactive protocol, the adversary can always learn the correct output.
2. It computes HSS shares of $x_{\text{priv}} = 0$ and some randomness to be used for generating π_{pub} . Indistinguishability of these shares and the honestly generated shares sent by an honest client in the real world follows from privacy of the underlying HSS scheme.
3. It uses the simulator of the underlying zkSNARG to simulate an accepting proof Π_{pub} .
4. The simulator computes the “expected” shares of y (say $y^{\mathcal{A}}$), Π_{local} (say $\Pi_{\text{local}}^{\mathcal{A}}$) and Π_{pub} (say $\Pi_{\text{pub}}^{\mathcal{A}}$) using HSS Eval and the HSS shares sent to the adversary.
5. It simulates the shares of y, Π_{pub} sent by the honest server as follows: $y^{\mathcal{H}} = y - y^{\mathcal{A}}$ and $\Pi_{\text{pub}}^{\mathcal{H}} = \Pi_{\text{pub}} - \Pi_{\text{pub}}^{\mathcal{A}}$. It is easy to see that $y^{\mathcal{H}}$ is identical to the share sent by an honest server in the real world. Indistinguishability of $\Pi_{\text{pub}}^{\mathcal{H}}$ from that sent by the honest server in the real world follows from zero-knowledge of the underlying zkSNARG.
6. Finally, upon receiving the shares $\tilde{y}^{\mathcal{A}}, \tilde{\Pi}_{\text{local}}^{\mathcal{A}}, \tilde{\Pi}_{\text{pub}}^{\mathcal{A}}$ sent by the adversarial server, the simulator checks:
 - If $y^{\mathcal{A}} \neq \tilde{y}^{\mathcal{A}}$, then from soundness of the underlying zkSNARGs, it follows that the adversary will not be able to produce shares such that the local proof verifies. And therefore the output of the honest client in this case must be $\perp$.
 - If $y^{\mathcal{A}} = \tilde{y}^{\mathcal{A}}$, but $\tilde{\Pi}_{\text{local}}^{\mathcal{A}} \neq \Pi_{\text{local}}^{\mathcal{A}}$, then from DP-robust verification of the underlying zkSNARG, it follows that the output of the honest client in this case must be $\perp$.
 - If $y^{\mathcal{A}} = \tilde{y}^{\mathcal{A}}$ and $\tilde{\Pi}_{\text{local}}^{\mathcal{A}} = \Pi_{\text{local}}^{\mathcal{A}}$, then the simulator checks if $\tilde{\Pi}_{\text{pub}}^{\mathcal{A}} + \Pi_{\text{pub}}^{\mathcal{H}}$ is an accepting proof. If so, the output of the honest clients must be y , else it must be $\perp$.

This concludes a proof sketch for our proposed construction.

DP-Robust Verification. A few remarks on DP-robust verification are in order:

- *Robust verification implies DP-robust verification:* Looking ahead in [Section 4](#), we first introduce a notion called *robust verification* (a single-prover analogue of DP-robust verification) and then demonstrate that this notion implies DP-robust verification. At a high level, we say that a zkSNARG satisfies robust verification if, given a valid statement x and witness w of choice, an adversary cannot, with high probability, find a non-zero additive error ϵ such that, if Π is an honestly generated accepting proof for x , then $\Pi + \epsilon$ is also an accepting proof for x .
- *Robust Verification vs Unambiguity:* It is worth noting that robust verification represents a weaker variant of a well-studied notion [\[RRR21\]](#) called unambiguity of a proof system, which demands that a computationally bounded adversary cannot produce an accepting proof different from the “true/prescribed

proof” even for true statements. This notion has recently found application in connection to PPAD hardness [CHK⁺19a, CHK⁺19b, EFKP20, KPY20, JKKZ21, LV20, BCH⁺22]. However, achieving this property is challenging and most proof systems (including the ones we will instantiate our approach with) do not satisfy this property. Fortunately, we only require a weaker variant (as described above) of this property. It is easy to see that a proof system that satisfies the standard notion of unambiguity will also satisfy robust verification. However, the inverse implication does not hold true.

- *Native Robust verification in randomized zkSNARKs:* We are only able to prove robust verification natively for randomized variants of the splittable zkSNARKs that we use to instantiate our approach. Therefore, even though Π_{local} could potentially have been generated using a deterministic-prover zkSNARK, we will compute it using a randomized-prover zkSNARK.
- *Instantiations of Splittable zkSNARKs:* In Section 4, we show that all linear PCP-based zkSNARKs [BCI⁺13] satisfy the structural properties of splittable zkSNARKs. Specifically, the prover algorithm in these zkSNARKs can be divided into two steps: low-depth non-cryptographic operations and linearly distributable cryptographic operations. Moreover, it is always possible to transform a linear PCP-based zkSNARK to one that has robust verification,⁹ at the cost of requiring the verifier to store private state generated during setup i.e., the resulting zkSNARK is designated-verifier. While a designated-verifier zkSNARK is compatible with our ve-HSS compiler, we show that the linear-PCP based zkSNARK proposed by Groth [Gro16] satisfies robust verification without any modifications. Furthermore, we present a zero-knowledge variant of the Waters-Wu [WW22] BARG scheme, and demonstrate that it satisfies all the structural properties of splittable zkSNARKs and has robust verification. Unlike the generic transformation, these zkSNARKs satisfy robust verification while being publicly verifiable, which may be a property of independent interest.

2.2 Groth16 zkSNARKs [Gro16] are Splittable

[Gro16] (henceforth referred to as the Groth16 zkSNARK) is a constant-sized zkSNARK, wherein the final proof consists of only three group elements. This scheme has perfect zero-knowledge, while its soundness holds in the generic group model (GGM). We observe that this scheme satisfies robust verification and all the other properties that we require from a splittable zkSNARK. In this section, we briefly recall this scheme and then discuss why it can be used to instantiate our approach.

Overview of Groth16 zkSNARKs. Let $(p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_T)$ be bilinear groups such that $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are groups of prime order p with generators g_1, g_2, g_T respectively, that satisfy the relation $e(g_1, g_2) = g_T$.

- *Correlated Random Setup (CRS):* The CRS is highly structured and is relation-specific. It comprises of correlated group elements in $\mathbb{G}_1$ and $\mathbb{G}_2$.
- *Proof Generation:* The relation being proven is encoded as an instance of Quadratic Arithmetic Program (QAP) [GGPR13]. The proof generation algorithm comprises of the following steps:
 1. *Extended Witness Generation:* The initial step involves deriving an “extended witness” by using the statement and witness for the given relation. This extended witness can essentially be perceived as the complete computation trace or the list of all intermediate wire values computed during the evaluation of the relation circuit using the statement and witness as input.
 2. *Additional Constant-Depth Field Operations:* The proof generation algorithm requires performing a constant-depth computation over this extended witness (and some additional randomness sampled by the prover to ensure zero-knowledge).

⁹We thank the anonymous TCC 2024 reviewer for pointing this out.

3. *Combining with CRS Terms:* The output of the previous step is then used to linearly combine (using group exponentiations) the group elements from the CRS to compute the final proof, which comprises of three group elements: $A, C \in \mathbb{G}_1$ and $B \in \mathbb{G}_2$.
- *Verification:* Given the CRS, statement and the proof elements A, B, C , the verifier performs a constant number of pairing operations to verify if the proof is accepting or not.

Splittable zkSNARG Properties. Recall that, in our application, the relation that will be proved using this zkSNARK is of the form: $\{f, x_{\text{pub}}, x_{\text{priv}}, y \mid \text{st.}, f(x_{\text{pub}}, x_{\text{priv}}) = y\}$, where f, y are always part of the statement and depending on the type of proof (i.e., local/public), x_{priv} is either part of the statement or the witness.

The depth of the computation necessary for generating each element of the extended witness for such relations is less than or equal to the depth of the circuit representation of f . Since the additional computation over the extended witness requires constant depth field operations, overall, the depth of the non-cryptographic operations involved in the generation of Groth16 zkSNARKs is asymptotically equivalent to that of f . However, most existing HSS schemes only support evaluating a restricted class of circuits, called Restricted Multiplication Straight-line (RMS) programs. Informally, an RMS program is an arithmetic circuit over bounded integers, where one of the two inputs of a multiplication gate must always be a circuit input wire. We show in [Theorem 9](#) that for any function f that can be computed by an NC^1 circuit, an HSS scheme for RMS programs¹⁰ can compute the non-cryptographic operations involved in Groth16. We refer the reader to [Section 5](#) for more details.

The only cryptographic operations involved in the generation of this SNARK are the group exponentiations required when combining the CRS terms with the output of the non-cryptographic operations. We now explain (using a simplified example) how the servers can generate shares of A, B, C : Let $a_1, \dots, a_\ell \in \mathbb{Z}_p$ be some terms output by the non-cryptographic operations in the previous step, and $Q_1, \dots, Q_\ell \in \mathbb{G}_1$ be some terms in the CRS. In Groth16 zkSNARKs, the proof term A is computed as $A = \prod_1^\ell Q_i^{a_i}$. Given additive shares of $a_1, \dots, a_\ell$ over $\mathbb{Z}_p$, the j^{th} server can compute $A^j = \prod_1^\ell Q_i^{a_i^j}$, where a_i^j is the share of a_i held by the j^{th} server (for each $j \in \{1, 2\}$). The final A can be reconstructed as $A = A^1 \cdot A^2$ using the shares computed by each of the servers¹¹. Shares of the other two terms B and C can also be computed in a similar manner. This idea of computing group exponentiations in a distributed manner using additive shares of the exponent has been explored in several prior works (see [\[OB22\]](#) and references contained therein for more details).

Robust Verification. Given the above discussion, it is clear that Groth16 zkSNARK meets all the structural requirements we demand from splittable zkSNARG. To establish that this proof system is applicable within our framework, all that remains is to demonstrate that it also achieves robust verification.

Given the proof terms A, B, C , and the statement $\text{st} := (x = (x_{\text{priv}}, x_{\text{pub}}), y)$, the verifier in Groth16 checks the following equation:

$$e(A, B) \stackrel{?}{=} e(Q_1, Q_2) \cdot e(Q_3^{x_{\text{priv}}}, Q_4^{x_{\text{pub}}}) \cdot Q_5^y \cdot e(C, Q_7)$$

where, $Q_1, \dots, Q_7$ correspond to some group elements in the CRS that are used for this verification check.

To establish robust verification, we want to show that given some errors $g_1^{\epsilon_A}, g_2^{\epsilon_B}, g_1^{\epsilon_C}$ by the adversary, if $\epsilon_A \neq 0$ or $\epsilon_B \neq 0$ or $\epsilon_C \neq 0$, and (A, B, C) is an accepting proof, then the following is not an accepting proof (w.h.p):

$$\bar{A} = A \cdot g_1^{\epsilon_A} \quad \bar{B} = B \cdot g_2^{\epsilon_B} \quad \bar{C} = C \cdot g_1^{\epsilon_C}$$

¹⁰We require the HSS scheme to support evaluating RMS programs over exponentially bounded integers. This is true for all HSS schemes with negligible correctness error.

¹¹Note that this multiplicative reconstruction can also be viewed as additive reconstruction over $\mathbb{Z}_p$ in the exponent of g_1 .

The verification check given these malformed proof terms can now be written as

$$\begin{aligned}
& e(A \cdot g_1^{\epsilon_A}, B \cdot g_2^{\epsilon_B}) \stackrel{?}{=} e(Q_1, Q_2) \cdot e(Q_3^{x_{\text{priv}}} \cdot Q_4^{x_{\text{pub}}} \cdot Q_5^y, Q_6) \cdot e(C \cdot g_1^{\epsilon_C}, Q_7) \\
& \Rightarrow e(A, B) \cdot e(g_1^{\epsilon_A}, B) \cdot e(A, g_2^{\epsilon_B}) \cdot e(g_1^{\epsilon_A}, g_2^{\epsilon_B}) \\
& \stackrel{?}{=} e(Q_1, Q_2) \cdot e(Q_3^{x_{\text{priv}}} \cdot Q_4^{x_{\text{pub}}} \cdot Q_5^y, Q_6) \cdot e(C, Q_7) \cdot e(g_1^{\epsilon_C}, Q_7) \\
& \Rightarrow e(g_1^{\epsilon_A}, B) \cdot e(A, g_2^{\epsilon_B}) \cdot e(g_1^{\epsilon_A}, g_2^{\epsilon_B}) \stackrel{?}{=} e(g_1^{\epsilon_C}, Q_7) \\
& \Rightarrow e(g_1^{\epsilon_A}, B) \cdot e(A, g_2^{\epsilon_B}) \stackrel{?}{=} e(g_1^{\epsilon_C}, Q_7) \cdot e(g_1^{\epsilon_A}, g_2^{\epsilon_B})^{-1}
\end{aligned}$$

We observe that because of the randomness used in the generation of A, B is uniformly sampled from $\mathbb{Z}_p$, these proof terms A and B are uniformly distributed over $\mathbb{G}_1$ and $\mathbb{G}_2$. Moreover, we know that Q_7 is guaranteed to not be g_2^0 . Therefore, the probability that an adversarial server who does not have the randomness used in the generation of A and B can find non-zero field elements $\epsilon_A, \epsilon_B, \epsilon_C$ such that the above check verifies, is exponentially small (in the size of the field). As a result, this randomized variant of Groth16 satisfies robust verification. We refer the reader to [Theorem 9](#) for a formal proof for why Groth16 zkSNARKs satisfy robust verification.

Recall that in the single-client scenario, for local proof we only need to prove a relation in $\mathcal{P}$. Moreover, since this proof is only sent to the client, one might intuitively consider using a de-randomized variant of Groth16 zkSNARKs. However, given that our proof of robust verification only holds for the randomized version of Groth16, we must use this variant for local proofs as well. Overall, instantiating our approach from [Section 2.1](#) using the Groth16 zkSNARKs gives us a single-client ve-HSS scheme. Since soundness of this proof system holds in the generic group model, our resulting ve-HSS scheme is also secure in the generic group model.

2.3 Splittable Zero-Knowledge BARGs

BARGs (Succinct Non-Interactive Batch Arguments) are SNARGs designed for a specific set of relations where a batch of statements, all requiring verification against the same NP relation, are involved. Thanks to recent advancements [[CJJ21](#), [CJJ22](#), [HJKS22](#), [WW22](#), [CGJ⁺23](#)], BARGs are now known from various standard assumptions, (i.e., without relying on any idealized models). As discussed in [Section 1.1](#), [[WW22](#)] (referred to hereafter as Waters-Wu BARGs) are the only known BARGs that avoid non-black-box usage of cryptography, but they do not achieve zero-knowledge. In this section, we discuss how we can adapt the Waters-Wu BARGs to obtain a black-box construction of zero-knowledge BARG (zkBARGs). We then proceed to show that this zkBARG is a splittable zkSNARG and can be used to instantiate our approach.

Recap of Waters-Wu BARGs. We start by recalling the design of Waters-Wu BARGs. At a high level, this BARG scheme is a *commit-and-prove* SNARG that achieves non-adaptive soundness. Let $\mathbb{G}$ and $\mathbb{G}_T$ be symmetric composite order pairing groups of order $N = pq$ for primes p and q . Thus $\mathbb{G} \cong \mathbb{G}_p \times \mathbb{G}_q$ where $\mathbb{G}_p$ is a subgroup of order p generated by $g_p = g^q$ where g is the generator for $\mathbb{G}$, and $\mathbb{G}_q$ is a subgroup of order q generated by $g_q = g^p$. Further, there exists an efficiently computable non-degenerate bilinear map, $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$, such that for all $a, b \in \mathbb{Z}_N$ $e(g^a, g^b) = e(g, g)^{ab}$. The subgroups $\mathbb{G}_p$ and $\mathbb{G}_q$ themselves are orthogonal with respect to the pairing operation, i.e. $e(g_p, g_q) = g_T^0$ where g_T is the generator for $\mathbb{G}_T$. The security of the construction relies on the *decision subgroup assumption* [[BGN05](#)] which states that a random element from the subgroup $\mathbb{G}_p$ is indistinguishable from a random element in the full group $\mathbb{G}$.

Let $C : \{0, 1\}^n \times \{0, 1\}^h \rightarrow \{0, 1\}$ be the Boolean circuit (composed of only NAND gates) used in the BARG, where the BARG proof is constructed over a batch of T instances $(x_1, \dots, x_T)$ such that for each

$i \in [T]$, there exists witness w_i such that $C(x_i, w_i) = 1$. The CRS is defined to be

$$\text{crs} = (\{A_i = g_p^{\alpha_i}\}_{i \in [T]}, A = \prod_{i \in [T]} A_i = g_p^{\sum_{i \in [T]} \alpha_i}, \{B_{i,j} = g_p^{\alpha_i \alpha_j}\}_{i,j \neq i})$$

where the α_i s are independently and randomly sampled from $\mathbb{Z}_N$.

- *Algebraic Vector Commitment*: For each wire k in the circuit C , a commitment U_k is computed as $U_k = \prod_{i \in [T]} A_i^{w_{i,k}} \in \mathbb{G}_p$ where $w_{i,k}$ is the k -th wire value in the i -th instance. The number of such commitments produced is equal to the number of wires in the circuit.
- *Wire Validity Check*: To prove that each commitment commits to a binary vector, the prover computes for each wire k , a proof term $V_k = \prod_{i \neq j} B_{i,j}^{(1-w_{i,k})w_{j,k}}$.
- *Gate Validity Check*: To prove that the (NAND) gate computation was done correctly, for every gate ℓ indexed by the two incoming wires k_1 and k_2 and outgoing wire k_3 , the prover computes $W_\ell = \prod_{i \neq j} B_{i,j}^{w_{j,k_3} - w_{i,k_1} w_{j,k_2} - 1}$.
- *Verification*: Given crs, the commitments $\{U_k\}_k$, the wire validity proofs $\{V_k\}_k$ and gate validity proofs $\{W_\ell\}_\ell$. The verifier checks for each k , if $e(A, U_k) = e(U_k, U_k) \cdot e(g_p, V_k)$; and for each gate tuple $\ell = (k_1, k_2, k_3)$, if $e(A, U_{k_3}) \cdot e(U_{k_1}, U_{k_2}) = e(A, A) \cdot e(g_p, W)$. Additionally, the verifier re-computes the commitments for the input wires corresponding to the statements to check if they were honestly generated.

Adding Zero-Knowledge. The algebraic vector commitments to internal wire values in this scheme are deterministically computed and hence are not *hiding*, it is easy to see that as such these BARGs do not achieve zero-knowledge. Towards adding zero-knowledge to this construction, our first idea is to use a hiding commitment. Such commitments can be obtained by simply adding randomness to the above algebraic vector commitments. In particular, for each wire k in the circuit C , the prover first samples a random $r_k \in \mathbb{Z}_p$ and then computes $U_k = g_p^{r_k} \cdot \prod_{i \in [T]} A_i^{w_{i,k}} \in \mathbb{G}_p$.¹² This randomized U_k now information-theoretically hides the $w_{i,k}$ values.

For *completeness*, we now also need to modify V_k and W_ℓ , in such a way that they can somehow be verified w.r.t. the randomized commitments U_k . Importantly, these modification should not break soundness of this proof system. We propose to modify V_k as follows:

$$V_k = \frac{\prod_{i \neq j} B_{i,j}^{(1-w_{i,k})w_{j,k}} \cdot \prod_{i \in [1,m]} A_i^{r_k(1-2w_{i,k})}}{g_p^{r_k^2}}.$$

Interestingly, in [Theorem 10](#), we show that given the randomized commitments U_k and these V_k terms, we can continue to use the same verification check as in Waters-Wu BARGs for wire validity checks. In [Section 4.3](#), we show that W_ℓ can also be modified in a similar manner, such that, together with the randomized U_k , gate validity checks can be performed in a similar manner as in Waters-Wu BARGs. While these modifications suffice for completeness, we still need to show the resulting BARG achieves soundness and zero-knowledge.

Zero-Knowledge: Next, we show that our modified BARG achieves perfect zero-knowledge. Our first observation here is that, each U_k is identically distributed to an element that is obtained by first uniformly

¹²Since the statement in public and need not be hidden, the prover can simply set $r_k = 0$ when computing commitments for the input wires corresponding to the statements.

sampling a random value (say z_k) from $\mathbb{Z}_N$ and then computing $g_p^{z_k}$. Moreover, given these values of U_k and the crs, the remaining proof terms V_k and W_ℓ are uniquely determined by the verification equations for wire-validity and gate-validity checks. Our proof for perfect zero-knowledge, now follows from these observations in a straightforward manner. Formally speaking, a simulator with knowledge of all the α_i terms used for generation of the crs can simulate an accepting proof for any batch of statements as follows: (1) for each wire k , it first samples a random $z_k \in \mathbb{Z}_N$ and then computes $U_k = g_p^{z_k}$. (2) Using knowledge of α_i s and the z_k values, it can now compute all the V_k and W_ℓ terms such that all the verification checks go through. It is easy to see that this proof is perfectly indistinguishable from an honestly generation proof for a true batch of statements.

Soundness: Observe that, our idea for adding zero-knowledge to the Waters-Wu BARGs only entails making modifications to the proof generation algorithm. Importantly, we do not require modifying the crs and the verification algorithm. In other words, both the crs and the verification algorithm in this modified construction are identical to that in Waters-Wu BARGs. As a result, soundness of our scheme simply follows from the soundness of Waters-Wu BARGs.

Splittable zkSNARG Properties. Similar to the Groth16 zkSNARKs, the first step in generating our zkBARG, involves computing all wire values corresponding to the statements, followed by the computation of all the exponent terms needed to compute V_k and W_ℓ . For instance, for each V_k , these exponent terms are $(1 - w_{i,k})w_{j,k}$, $r_k(1 - 2w_{i,k})$ and r_k^2 . Since these only involve operations over $\mathbb{Z}_N$, they can be computed in a distributed manner using HSS Eval.

Given the crs terms, all wire values, and these exponent terms, the subsequent step is to compute U_k, V_k, W_ℓ using group exponentiations. As discussed in [Section 2.2](#), these operations can be performed in a non-interactive distributed manner, provided additive shares of the wire values and the exponent terms are available (which we obtain from the previous step). This establishes that our zkBARG satisfies all the structural properties of a splittable zkSNARG.

Robust Verification. Finally, we show via a sequence of the following claims that this scheme also has robust verification:

- Firstly, we observe that an adversary cannot add a non-zero error to the *vector commitments corresponding to the statement*. Recall that, since the statements are public, we do not randomize these commitments in our zkBARG. Similar to Water-Wu BARGs, the verifier verifies the correctness of these commitments by recomputing them. Therefore, any deviation from an honestly computed commitment to the statements will result in the proof being rejected.
- Next, we establish that the probability of an adversary successfully choosing a non-zero error on any of the remaining commitments U_k or the V_k terms, while still surpassing the wire-validity checks, is exponentially small. The proof of this claim employs a similar argument to our proof of robust verification for Groth16 zkSNARKs and crucially relies on the fact that the randomness r_k used in the computation of U_k remains unknown to the adversary.
- Lastly, if no errors are introduced on the U_k terms, then there exist unique values of W_ℓ that will allow the gate-validity checks to pass. This ensures that if the adversary sends errors for any of the W_ℓ terms, the verification will fail.

We defer a formal proof to [Theorem 11](#). Instantiating our approach from [Section 2.1](#) using this new construction of zkBARGs gives us a single-client ve-HSS scheme for SIMD computations. The security of this scheme holds in the standard model.

2.4 Multi-Client HSS with Verifiable Evaluation

So far, our discussion has been limited to the single-client setting. Now, we shift our focus to the scenario where multiple (say n) mutually distrustful clients seek to share their inputs via HSS with the servers to compute a joint function on these inputs. In this section, we now consider the setting where multiple mutually distrusting clients (we consider both semi-honest and malicious client settings) have inputs that they wish to HSS share with the servers to have them compute a joint function on these inputs.

Candidate Approach. In the single-client setting, the local proof assures the client that the output was honestly computed using their input. In the multi-client setting, we require the same assurance for every client who shares their inputs via HSS with the servers. A straightforward way to achieve this would be to have the servers compute multiple local proofs, one for each server. Specifically, the local proof sent to the i^{th} server will pertain to the relation $\mathcal{R}_i = \{x_i, y, f, x_{\text{pub}} : \exists \{x_j\}_{j=1, i \neq j}^n, \text{st.}, f(x_{\text{pub}}, x_1, \dots, x_n) = y\}$. Observe that, unlike in the single-client setting, this is no longer a deterministic relation. To ensure privacy (of the other clients' inputs), we must use a zero-knowledge SNARG to generate this proof. The publicly verifiable proofs can be computed using any splittable zkSNARG as in the single-client setting, albeit for the relation $\mathcal{R}_{\text{pub}} = \{y, f, x_{\text{pub}} : \exists \{x_i\}_{i=1}^n, \text{st.}, f(x_{\text{pub}}, x_1, \dots, x_n) = y\}$. Together these two types of proofs ensure local and public soundness in the multi-client setting. However, privacy of this candidate approach does not follow as easily as before.

Challenges in the Multi-Client Setting. Unlike in the single-client setting, corrupt clients can also have inputs in the multi-client setting. This presents some unique challenges in this setting.

- *Privacy:* In the single-client setting, the output y is uniquely determined by the input of the honest client. As a result, if the local proof verifies, then the correctness of the output y follows from the soundness of the underlying splittable zkSNARG. However, in the multi-client setting, the output y might not be determined by the input of any single client. Since the statements proved using the local and public proofs consist of only a subset of the inputs, we can no longer rely solely on these proofs to verify the correctness of the reconstructed output.

We explain this using a simple example. Let us assume that we have two clients C_0, C_1 with one bit input each (let C_i have input bit b_i), who wish to delegate the computation of the product of their inputs using ve-HSS. Let us further assume that the malicious server employs the following strategy: it first waits for the honest server to send its share of the output and then chooses its share of the output adaptively, such that the reconstructed output is always 0. Now consider the scenario, where both b_0 and b_1 are 1 and therefore the correct output is $y = 1$. However, using the aforementioned attack strategy, if the adversarial server succeeds in forcing the reconstructed output to be $y = 0$, it can still potentially generate convincing local proofs. This is because the local proof for C_i only guarantees that there exists b_{1-i} such that the AND of b_i and b_{1-i} is 0.

The issue with our previous approach based on DP-robust verification is that our simulator can only check validity of the proof share sent by an adversarial server for the “correct” y . Here, since the resulting proof maybe w.r.t. a different $\bar{y}$, DP-robust verification does not suffice. We need a stronger version of DP-robust verification in this setting, that also allows the adversary to maul the statement w.r.t. which the proof is generated.

Unfortunately, we do not know how to prove this *strong* DP-robust verification for the splittable zk-SNARGs considered in [Section 2.2](#) and [Section 2.3](#). To circumvent this issue, in [Section 6](#), we show how to combine our ideas from the single-client setting with information-theoretic MAC-based ideas from [\[ADOS22\]](#) (see [Section 1.3](#)) to obtain a secure multi-client vHSS.

- *Input Sharing*: Unlike in the single-client setting, where the client can use a *secret-key* HSS scheme to compute shares of its input, in the multi-client setting, it is easier for the clients to compute HSS shares of their respective inputs if they start with a public-key HSS scheme. However, this requires the public key and the requisite evaluation keys to be generated in a secure manner. Moreover, given the outcome of this setup procedure, in case some of the clients may also be malicious, we also need to ensure that they send well-formed HSS shares of their inputs. We address this as follows:

1. *Semi-Honest Clients*: When clients are semi-honest, our compiler can be used to transform any public-key HSS scheme with at most negligible correctness error into a multi-client ve-HSS using a suitable Splittable zkSNARG, assuming a secure “public-key setup”.

Abram et al. [ADOS22] demonstrate that for DCR-based [OSY21, RS21] and class group-based [ADOS22] HSS schemes, there exists a maliciously secure one-round distributed protocol for public-key setup that only requires making a black-box use of cryptography. When instantiated with these HSS schemes, our compiler yields a fully black-box solution for multi-client ve-HSS, where the public key setup can be done jointly by the servers using the distributed protocol from [ADOS22].

Alternatively, we can instantiate our compiler using other HSS schemes. However, since black-box protocols for public-key setup are not known for these other HSS schemes, the setup in these cases must be done by a trusted entity.¹³

2. *Malicious Clients*: Abram et al. [ADOS22] present a black-box, one-round, maliciously secure input sharing protocol (in the random oracle model) for HSS schemes based on DCR [OSY21, RS21] and class groups [ADOS22]. We leverage this (and their distributed protocol for key setup) to demonstrate that our compiler can be instantiated with these HSS schemes to obtain a black-box solution for multi-client ve-HSS.¹⁴

2.5 Applications

In this section, we discuss applications of our ve-HSS schemes to private delegation of different types of functions.

Delegation of Non-Cryptographic Functions. Our notion of ve-HSS naturally provides a framework for private outsourcing of computation, whether by a single client or a group of clients, to two or more servers. It is easy to see that for non-cryptographic functions, our constructions offer a black-box solution to this problem. As discussed in Section 1.2, this offers several advantages over an FHE based delegation.

Delegation of zkSNARK Computation. We now address the problem of private outsourcing of zkSNARK computation to a group of untrusted servers by a single client. Specifically, given a public statement x , consider a client who wishes to obtain a proof (using a specific zkSNARK scheme Π) attesting that it knows some witness w such that $C(x, w) = 1$. Instead of computing the proof itself, the client wants to outsource this computation to a group of untrusted servers. This problem has been explored in several recent works [GGJ⁺23, CLMZ23, LZW⁺24, GGW24].

Since the computation of a zkSNARK is inherently cryptographic, naïvely using any delegation framework for outsourcing this task may lead to non-black-box use of the cryptographic operations involved in generating the zkSNARK, even if the original delegation scheme is black-box in the underlying cryptographic primitives used in its construction. However, we demonstrate that our specific construction of

¹³This setup could also be implemented jointly by the servers using a generic interactive maliciously secure MPC protocol. However, this would result in a non-black-box use of cryptography during the key setup phase.

¹⁴We note that initializing our compiler with other HSS schemes, for which such black-box protocols for public-key setup and input sharing are not currently known, does not result in a fully black-box solution.

single-client ve-HSS yields a solution for delegating zkSNARK computation that remains black-box in the cryptographic operations used in its generation.

In particular, we observe that if the zkSNARK Π the client wishes to employ is a splittable zkSNARK, then our construction of a single-client ve-HSS provides a fully black-box solution as follows:

1. The client first uses HSS Share to compute shares of $(x_{\text{priv}} = w)$.
2. The client then sends $(x_{\text{pub}} = x)$ along with these HSS shares of x_{priv} to the servers and instructs them to compute C using our single-client ve-HSS (instantiated with Π).
3. Recall that, in our ve-HSS construction, the output of Rec algorithm contains the output $C(x, w) = 1$ a local proof Π_{local} and a public proof Π_{pub} . Here Π_{pub} is a zkSNARK proof for the relation $\mathcal{R}_{\text{pub}} = \{(C, x = x_{\text{pub}}, 1) \mid \exists w = x_{\text{priv}}, \text{ s.t., } C(x, w) = 1\}$. This corresponds exactly to the zkSNARK that the client wanted to delegate in the first place, so the servers in this construction simply output their respective shares of Π_{pub} and ignore the shares of the local proofs.

This yields a fully non-interactive solution for the problem of private delegation of zkSNARK computation. As discussed in [Section 1.2](#), in contrast, prior MPC-based approaches [GGJ⁺23, CLMZ23, LZW⁺24] require a large amount of communication amongst the servers. An advantage of our solution over the FHE based approach proposed in [GGW24] is that the servers can independently compute shares of the delegated zkSNARK proof, without requiring additional intervention from the client.

Delegation of Collaborative zkSNARK Computation. Next, we consider the setting where a group of mutually distrustful clients wish to *jointly* compute a zkSNARK for some statement x using their combined witnesses $(w_i)_{i \in [n]}$ for the relation: $\mathcal{R} : \{(C, x) \mid \exists (w_i)_{i \in [n]}, \text{ s.t., } C(x, (w_i)_{i \in [n]}) = 1\}$. Such zkSNARKs were recently introduced in [OB22], and are referred to as Collaborative zkSNARKs.

We demonstrate that our construction for multi-client ve-HSS, yields a black-box approach for private delegation of splittable collaborative zkSNARKs. This can essentially be viewed as a multi-client analogue of the previous application. While the main ideas used for enabling this application are similar to those used in the previous application, there are some distinctions:

1. Unlike in the single-prover setting, where the prover knows whether it has a valid witness corresponding to the given statement and relation, in collaborative zkSNARKs, no individual party owns the entire witness. Consequently, the clients do not know if the statement is true or not. As a result, the definition of zero-knowledge is slightly different in collaborative zkSNARKs. Specifically, in this setting, privacy of the honest clients' witnesses must be guaranteed irrespective of whether statement is true or false.

When using our multi-client ve-HSS scheme for delegating the computation of collaborative zkSNARKs, we propose the following approach to always ensure privacy of the honest clients' witnesses: similar to our previous application, upon receiving x and HSS shares of the witnesses, the servers compute (using the a simplified version of our multi-client ve-HSS) shares of $C(x, (w_i)_{i \in [n]})$ and the public proof (while ignoring the local proofs). In this application, before revealing their shares of the public proof, we instruct the servers to use HSS Eval to multiply the output of $C(x, (w_i)_{i \in [n]})$ with the public proof.

In case $C(x, (w_i)_{i \in [n]}) = 1$, the resulting value corresponds the the desired collaborative zkSNARK. Privacy of the honest clients' witnesses follows from the zero-knowledge property of the underlying zkSNARK. And in case $C(x, (w_i)_{i \in [n]}) = 0$, above modification ensures that no information (apart from the outcome of $C(x, (w_i)_{i \in [n]})$) about the honest clients' witnesses is revealed.

2. As discussed in [Section 2.4](#), we require clients to perform information-theoretic MAC checks in the ve-HSS construction, based on ideas from [ADOS22], to ensure that adversarial servers cannot tamper with the inputs of clients and deceive them into accepting an incorrect output. Thus, our collaborative

zkSNARK construction requires the servers to also participate in the input-sharing phase to send HSS input shares of their individual MAC keys. The servers then locally perform the MAC check to verify that the relation circuit was evaluated correctly. Since at least one server is assumed to be honest in the collaborative zkSNARK model, the soundness of the MAC check ensures that the statement is indeed verified with the witness shared by clients. Moreover, since the MAC check is run by the servers, the clients are no longer required after the input-sharing phase.

3 Preliminaries

Basic Notation. We use $[x, y]$ to denote the set $\{x, x+1, \dots, y\}$ where $x, y \in \mathbb{Z}$ and $x \leq y$. We use $\mathbf{x} \in X^\ell$ to denote the tuple of ℓ elements in X and $|\mathbf{x}| = \ell$ to denote the length of $\mathbf{x}$. For a function $f : X \rightarrow Y$ and tuple $\mathbf{x} = (x_1, \dots, x_\ell) \in X^\ell$ we use $\mathbf{y} = f(\mathbf{x})$ to denote $\mathbf{y} = (f(x_1), \dots, f(x_\ell)) \in Y^\ell$. We use λ as the security parameter, $\text{negl}(\lambda)$ to denote negligible functions, and $\text{poly}(\lambda)$ to denote polynomials in λ . For two probability ensembles $\{A_i\}_i$ and $\{B_i\}_i$, we use $\{A_i\}_i \stackrel{\Delta}{=} \{B_i\}_i$ to denote that the ensembles are identical, $\{A_i\}_i \stackrel{\approx}{\sim} \{B_i\}_i$ to denote that the ensembles are statistically close and $\{A_i\}_i \stackrel{\epsilon}{\sim} \{B_i\}_i$ to denote that the ensembles are computationally indistinguishable. We use $\Pr[E \mid A]$ to denote the probability of an event E in an experiment defined by executing A .

Given a ring R , we will slightly abuse notation by treating a bit $x \in \{0, 1\}$ as an element of R via the natural mapping: 0 maps to the additive identity 0_R and 1 maps to the multiplicative identity 1_R . For a circuit C over a ring R , the depth of C is the maximum number of gates on any path from an input to an output, while the multiplicative depth of C is the maximum number of multiplication gates on any such path. In this work, we only consider circuits with 2-input gates.

Additive Secret Sharing. Let $\mathbb{G}$ be a group and let $m \in \mathbb{N}$. We use $[x] = ([x]_1, \dots, [x]_m) \in \mathbb{G}^m$ to denote an m -party additive secret sharing of a secret $x \in \mathbb{G}$, where the i -th party holds the share $[x]_i$ and $\sum_{i=1}^m [x]_i = x$. Observe that the scheme is *additively homomorphic* i.e., it allows parties to compute on the secrets by carrying out the same operations on corresponding shares. For a subset $I \subset [1, m]$, we use $[x]_I$ to denote $\{[x]_i\}_{i \in I}$. For a tuple $\mathbf{x} = (x_1, \dots, x_\ell) \in \mathbb{G}^\ell$, we use $[\mathbf{x}]$ to denote the sharings $([x_1], \dots, [x_\ell])$ and use $[\mathbf{x}]_i$ to denote the shares $([x_1]_i, \dots, [x_\ell]_i)$ of the i -th party. We say $[x] \in R^m$ is an additive sharing of $x \in R$, for a ring R , to denote that it is an additive sharing over the group $(R, +)$. In this case, we say $[b]$ is an additive sharing of a bit $b \in \{0, 1\}$ over R if $[b]$ reconstructs to the additive identity 0_R when $b = 0$ and it reconstructs to the multiplicative identity 1_R when $b = 1$.

3.1 Homomorphic Secret Sharing

An HSS scheme allows a client to secret share its inputs among a set of servers, which can then *non-interactively* evaluate a function on the shared inputs to obtain an additive sharing of the output. Informally, an HSS scheme can be viewed as a distributed variant of homomorphic encryption where inputs are secret shared instead of being encrypted and parties obtain additive shares of the output upon evaluation instead of an encrypted output. The inputs need not be secret shared using an additive secret sharing scheme and we use $\langle \mathbf{x} \rangle = (\langle \mathbf{x} \rangle_1, \dots, \langle \mathbf{x} \rangle_m)$ to denote the HSS input shares for a tuple $\mathbf{x}$ where the i -th server's share is denoted by $\langle \mathbf{x} \rangle_i$. For any $I \subseteq [1, m]$, we use $\langle \mathbf{x} \rangle_I$ as a shorthand to denote $\{\langle \mathbf{x} \rangle_i\}_{i \in I}$.

We first recall the definition of HSS in the single client and multiple server setting and then discuss public-key HSS which extends the definition to multiple clients.

Definition 1 (Homomorphic Secret Sharing [BGI16a]). *Let $m := m(\lambda)$ be an integer-valued polynomial, R be a ring and let $\mathbb{C}_{\text{hss}}$ be a class of circuits over R . An m -server Homomorphic Secret Sharing (HSS) scheme for $\mathbb{C}_{\text{hss}}$ is a pair of algorithms $\text{HSS} = (\text{Share}, \text{Eval})$ with the following syntax.*

- $\text{Share}(1^\lambda, \mathbf{x}) \rightarrow (\langle \mathbf{x} \rangle_1, \dots, \langle \mathbf{x} \rangle_m)$ is a PPT algorithm that takes as input $\mathbf{x} \in R^*$ and for each $i \in [1, m]$, outputs a share $\langle \mathbf{x} \rangle_i$ for the i -th server.
- $\text{Eval}(i, C, \mathbf{x}_{\text{pub}}, \langle \mathbf{x} \rangle_i) =: [\mathbf{y}]_i$ is a polynomial time algorithm that takes as input a server index i , a circuit $C \in \mathbb{C}_{\text{hss}}$, common server input $\mathbf{x}_{\text{pub}} \in R^*$, and the share of the i -th server $\langle \mathbf{x} \rangle_i$ and computes an additive share of the circuit output $[\mathbf{y}]_i$ over R .

An HSS scheme should satisfy the following properties.

- **Correctness:** For all polynomials $\text{poly}(\cdot)$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, circuits $C \in \mathbb{C}_{\text{hss}}$, and inputs $\mathbf{x}_{\text{pub}} \in R^*$ and $\mathbf{x} \in R^*$ we have

$$\Pr \left[[\mathbf{y}]_1 + \dots + [\mathbf{y}]_m \neq C(\mathbf{x}_{\text{pub}}, \mathbf{x}) \mid \begin{array}{l} (\langle \mathbf{x} \rangle_1, \dots, \langle \mathbf{x} \rangle_m) \leftarrow \text{Share}(1^\lambda, \mathbf{x}) \\ [\mathbf{y}]_i := \text{Eval}(i, C, \mathbf{x}_{\text{pub}}, \langle \mathbf{x} \rangle_i), \forall i \in [1, m] \end{array} \right] \leq \text{negl}(\lambda)$$

where $|P|, |\mathbf{x}_{\text{pub}}|, |\mathbf{x}| \leq \text{poly}(\lambda)$. Alternatively, in a δ -correct scheme, the above probability is at most $\delta + \text{negl}(\lambda)$ and the Eval algorithm is allowed to run in time polynomial in $1/\delta$.

- **Security:** For every polynomial size adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ we have

$$\Pr \left[b' = b \mid \begin{array}{l} (\text{st}, \mathcal{I}, \mathbf{x}_0, \mathbf{x}_1) \leftarrow \mathcal{A}(1^\lambda) \\ b \leftarrow \{0, 1\} \\ (\langle \mathbf{x}_b \rangle_1, \dots, \langle \mathbf{x}_b \rangle_m) \leftarrow \text{Share}(1^\lambda, \mathbf{x}_b) \\ b' \leftarrow \mathcal{A}(\text{st}, \langle \mathbf{x}_b \rangle_{\mathcal{I}}) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where $\mathcal{I} \subset [1, m]$, $\mathbf{x}_0, \mathbf{x}_1 \in R^*$, and $|\mathbf{x}_0| = |\mathbf{x}_1|$.

Observe that the output of Eval is *compact* since it computes an additive share i.e., the size of the output share depends only on the size of the circuit output and is independent of the size of the inputs and the circuit. Moreover, since Share is PPT, the size of each input share is polynomial in the size of the circuit input $\mathbf{x}$ and the security parameter but independent of the circuit size.

Public-key HSS. A public-key HSS scheme [BGI16a] allows computation on the combined inputs of multiple clients. It includes an additional key generation algorithm that outputs a public key pk and an evaluation key ek_i for each server. Any client can encrypt its input x using the public key pk to compute the ciphertext $\tilde{x}$. Each server can then use its evaluation key and the ciphertexts provided by the clients to locally evaluate a circuit on the joint inputs of all clients, and obtain an additive share of the circuit output.

Definition 2 (Public-key Homomorphic Secret Sharing [BGI16a]). Let $m := m(\lambda)$ be an integer-valued polynomial, R be a ring and let $\mathbb{C}_{\text{hss}}$ be a class of circuit over R . An m -server public-key Homomorphic Secret Sharing (HSS) scheme for $\mathbb{C}_{\text{hss}}$ is a tuple of algorithms $\text{HSS} = (\text{KeyGen}, \text{Enc}, \text{Eval})$ with the following syntax.

- $\text{KeyGen}(1^\lambda) \rightarrow (\text{pk}, \text{ek}_1, \dots, \text{ek}_m)$ is a PPT algorithm that outputs a public key pk and m evaluation keys, where ek_i is the evaluation key of the i -th server.
- $\text{Enc}(\text{pk}, x) \rightarrow \tilde{x}$ is a PPT algorithm that takes a value $x \in R$ and outputs a ciphertext $\tilde{x}$.
- $\text{Eval}(\text{ek}_i, P, \mathbf{x}_{\text{pub}}, \tilde{x}_1, \dots, \tilde{x}_n) =: [\mathbf{y}]_i$ is a polynomial time algorithm that takes as input the evaluation key ek_i of the i -th server, a circuit $C \in \mathbb{C}_{\text{hss}}$, common server input $\mathbf{x}_{\text{pub}} \in R^*$, and n ciphertexts corresponding to the client inputs and computes an additive share of the circuit output $[\mathbf{y}]_i$ over R .

A public-key HSS scheme should satisfy the following properties.

- **Correctness:** For all polynomials $\text{poly}(\cdot)$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, n \in \mathbb{N}$, circuits $C \in \mathbb{C}_{\text{hss}}$, and inputs $\mathbf{x}_{\text{pub}} \in R^*$ and $x_1, \dots, x_n \in R$ we have

$$\Pr \left[\begin{array}{c} [y]_1 + \dots + [y]_m \\ \neq \\ C(\mathbf{x}_{\text{pub}}, x_1, \dots, x_n) \end{array} \middle| \begin{array}{l} (\text{pk}, \text{ek}_1, \dots, \text{ek}_m) \leftarrow \text{KeyGen}(1^\lambda) \\ \tilde{x}_i \leftarrow \text{Enc}(\text{pk}, x_i), \forall i \in [1, n] \\ [y]_i := \text{Eval}(\text{ek}_i, P, \mathbf{x}_{\text{pub}}, \tilde{x}_1, \dots, \tilde{x}_n), \forall i \in [1, m] \end{array} \right] \leq \text{negl}(\lambda)$$

where $|P|, n, |\mathbf{x}_{\text{pub}}| \leq \text{poly}(\lambda)$.

- **Security:** For every polynomial size adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\lambda)$ such that for all $\lambda \in \mathbb{N}$ and $\mathcal{I} \subset [1, m]$ we have

$$\Pr \left[b' = b \middle| \begin{array}{l} (\text{pk}, \text{ek}_1, \dots, \text{ek}_m) \leftarrow \text{KeyGen}(1^\lambda) \\ (x_0, x_1, \text{st}) \leftarrow \mathcal{A}(1^\lambda, \text{pk}, \{\text{ek}_i\}_{i \in \mathcal{I}}) \\ b \leftarrow \{0, 1\} \\ \tilde{x}_b \leftarrow \text{Enc}(\text{pk}, x_b) \\ b' \leftarrow \mathcal{A}(\text{st}, \tilde{x}_b) \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda)$$

where $x_0, x_1 \in R$.

As in the case of single client HSS, the fact that Eval computes additive shares of the circuit output implies that its output is *compact* — the size of the output share depends only on the size of the circuit output and is independent of the size of the input and circuit.

Remark 1 (Simulating Output Shares in HSS). Observe that the security of HSS schemes only ensure that the input shares (or ciphertexts in case of public-key HSS) sent to corrupt servers can be simulated. On the other hand, applications of HSS will often involve reconstructing the output, which will in turn require simulating the output shares of the honest server in the security proof. However, the security and correctness properties of HSS allow simulating the honest server's output share too. Specifically, we first simulate input shares or ciphertexts sent by clients to corrupt servers and then compute the shares of the output $[y]_{\mathcal{I}}$ obtained by corrupt servers upon running Eval. The output share of the honest server can then be computed using the circuit output y and the additive shares $[y]_{\mathcal{I}}$ obtained by corrupt servers. Crucially, since evaluation is deterministic, the corrupt servers indeed obtain $[y]_{\mathcal{I}}$ upon HSS evaluation. The correctness property of HSS then ensures that the simulated output share is indistinguishable from the output share of the honest server.

Restricted Multiplication Straight-line Programs. A large class of existing HSS schemes [BGI16a, BKS19, OSY21, RS21, ADOS22] only support evaluating Restricted Multiplication Straight-line (RMS) programs [Cle90, BGI16a]. An RMS program is a circuit over integers with the restriction that every multiplication is between an input value and an intermediate value of the computation, called a memory value. Boyle et. al [BGI16a] showed that the class of polynomial size RMS programs includes the class of polynomial size branching programs, which is in turn known to contain the class of NC^1 circuits.

Definition 3 (Restricted Multiplication Straight-line Program [Cle90, BGI16a]). A Restricted Multiplication Straight-line (RMS) Program P consists of a magnitude bound $B_{\text{rms}} \in \mathbb{N}$ and an arbitrary sequence of the following four instructions.

- $\text{LoadInput}(l_x) \rightarrow M_x$: Load the value of the input l_x to the memory M_x .
- $\text{Add}(M_x, M_y) \rightarrow M_z$: Add the values of the memory M_x and M_y and assign the result to the memory M_z .
- $\text{Mult}(l_x, M_y) \rightarrow M_z$: Multiply the value of the input l_x by the value of the memory M_y and assign the result to the memory M_z .

– $\text{Output}(M_z, p_{\text{rms}}) \rightarrow z \bmod p_{\text{rms}}$: Output the value of the memory M_z reducing it modulo p_{rms} .

If at any step of the execution, the magnitude of a memory value exceeds the bound B_{rms} , the output of the program on the corresponding input is defined to be $\perp$. The size of an RMS program is defined as the number of instructions.

In an HSS scheme for RMS programs with bound B_{rms} , servers obtain an additive sharing of the program output upon HSS evaluation when the RMS program does not evaluate to $\perp$; otherwise, they obtain $\perp$. In particular, computation takes place over the ring $\mathbb{Z}_{p_{\text{rms}}}$.

The following lemma states that any polynomial-size log-depth circuit over an integer ring can be computed by a polynomial-size RMS program.

Lemma 5. *For any n -input polynomial size circuit C over $\mathbb{Z}_p$ with $O(\log n)$ -depth, there exists a polynomial size RMS program P with bound $B_{\text{rms}} \in p^{O(n)}$ such that for all $\mathbf{x} \in \mathbb{Z}_p^n$, $P(\mathbf{x}) = C(\mathbf{x})$.*

Proof. P takes the same inputs as C and computes a memory value M_y for each intermediate value y of the circuit, in the order determined by a breadth-first traversal of C . Specifically, for each input x , P computes a $\text{LoadInput}(l_x)$ instruction and obtains a memory value M_x . For every gate g of C at depth i , if g is an addition gate with inputs y and z , P computes the $\text{Add}(M_y, M_z)$ instruction. On the other hand, if g is a multiplication gate with inputs y and z , P first computes $\text{Mult}(l_x, M_y)$ for each program input x and then recomputes the RMS program used to compute M_x , except that it replaces the memory value $M_{z'}$ with $M_{z' \cdot y}$ for each instruction in this sub-program. Finally, it computes $\text{Output}(M_z, p)$ for the output wire z of the circuit.

Observe that $C(\mathbf{x})$ when computed over the integers is less than p^n for all $\mathbf{x} \in \mathbb{Z}_p^n$ since C has depth $O(\log n)$. Since P has bound $p^{O(n)}$, it follows from our description of P that for every input $\mathbf{x} \in \mathbb{Z}_p^n$, $P(\mathbf{x}) = C(\mathbf{x})$. Next, note that computing the output of a gate at depth i of C from the inputs $\mathbf{x}$ requires at most twice the number of instructions than computing the output of a gate at depth- $(i - 1)$. Since C has polynomial size and $O(\log n)$ -depth, it then follows from a straightforward counting argument that the size of P is polynomial in n . ■

3.1.1 HSS Schemes from Prior Works

We briefly summarize the HSS schemes from prior works. The first HSS scheme was proposed by Boyle et al. [BGI16a] where they constructed a 2-server HSS scheme and a 2-server public-key HSS scheme for evaluating polynomial size RMS programs, assuming the hardness of the Decisional Diffie-Hellman (DDH) assumption. However, their HSS schemes suffer from inverse polynomial correctness error and only supports evaluating RMS programs with a magnitude bound that is polynomial in the security parameter. Subsequently, a number of works have proposed 2-server public-key HSS schemes with negligible correctness error [BKS19, OSY21, RS21, ADOS22]. These schemes support evaluating polynomial size RMS programs, where the magnitude bound is exponential in the security parameter.

In more detail, [BKS19] construct HSS schemes from the Learning With Errors (LWE) assumption while [OSY21, RS21] construct HSS schemes based on the Decisional Composite Residuosity (DCR) assumption. The primary difference between [OSY21] and [RS21] is that the HSS schemes of [OSY21] are based on the Paillier and Brakerski-Goldwasser [BG10] encryption schemes while the HSS scheme of [RS21] is based on the Damgård-Jurik encryption scheme [DJ01]. Abram et al. [ADOS22] generalize and extend the constructions of [OSY21, RS21] by presenting a general group-theoretic framework which they call as the Non-Interactive Discrete Log Sharing (NIDLS) framework. They then show that their framework can be instantiated with class groups [CL15]. Since any public-key HSS scheme is also a single client HSS scheme, these works also imply 2-server HSS schemes from the same assumptions.

The HSS scheme of [DIJL23], based on sparse LPN, supports more than 2 servers but has inverse polynomial correctness error and only allows evaluating multivariate polynomials with polynomially many monomials and of total degree at most $O(\log \lambda / \log \log \lambda)$. HSS schemes for evaluating larger classes of circuits rely on strong assumptions [BGI15, DHRW16, BGI⁺18, COS⁺22]. These schemes support evaluating any polynomial size circuit as well as support having more than 2 servers. Specifically, for all positive integers m , m -server HSS schemes for P/poly are known either assuming the existence of sub-exponentially secure indistinguishability obfuscation and one-way functions [BGI15] or from circular secure Additive-Function Sharing (AFS) spooky encryption [DHRW16, BGI⁺18, COS⁺22]. AFS spooky encryption is known from LWE [DHRW16, COS⁺22]. These HSS schemes [BGI15, COS⁺22] also support evaluating polynomial size circuits over integer rings that are exponentially large in the security parameter. The m -server HSS schemes [DHRW16, COS⁺22] from AFS-spooky encryption are also public-key HSS schemes.

3.2 Succinct Non-interactive Arguments

A Succinct Non-interactive Argument (SNARG) allows a prover holding a witness w to compute a proof π that convinces a verifier about the validity of a given statement x , such that the size of π and the verification time are sublinear in the size of w . A zero-knowledge SNARG (zkSNARG) additionally ensures that the proof does not leak any additional information about the witness w . In this work, we consider zkSNARGs for the circuit satisfiability language.

Circuit Satisfiability Language. The circuit satisfiability language SAT is defined as

$$\text{SAT} = \{(C, x, y) \mid \exists w, C(x, w) = y\}$$

where $C : R^{\ell_{\text{inp}}} \times R^{\ell_{\text{wit}}} \rightarrow R^{\ell_{\text{out}}}$ is a circuit, R is a ring, ℓ_{inp} , ℓ_{wit} and ℓ_{out} are natural numbers, $x \parallel y \in R^{\ell_{\text{stmt}}}$ is the statement and $\ell_{\text{stmt}} = \ell_{\text{inp}} + \ell_{\text{out}}$. The description of the circuit also defines ℓ_{inp} , ℓ_{out} , ℓ_{wit} , and R . We will overload notation and use SAT to also refer to the circuit satisfiability relation so that $(C, x, y; w) \in \text{SAT}$ when $C(x, w) = y$.

Definition 4 (Zero-knowledge Succinct Non-interactive Argument). *Let $\mathbb{C}_{\text{ZK}}$ be a class of circuits. A zero-knowledge Succinct Non-interactive Argument (zkSNARG) for $\mathbb{C}_{\text{ZK}}$ -circuit satisfiability is a triple of algorithms $\text{SNARG} = (\text{Setup}, \text{Prove}, \text{Verify})$ with the following syntax.*

- $\text{Setup}(1^\lambda, C) \rightarrow \text{crs}$ is a PPT algorithm that takes the description of a circuit $C \in \mathbb{C}_{\text{ZK}}$ as input and outputs a common reference string crs .
- $\text{Prove}(\text{crs}, x, y, w) \rightarrow \pi$ is a PPT algorithm that takes the common reference string crs , a statement (x, y) and a witness w as input and outputs a proof π .
- $\text{Verify}(\text{crs}, x, y, \pi) \rightarrow b$ is a PPT algorithm that takes the common reference string crs , a statement (x, y) and a proof π as input and outputs a bit b indicating if the proof is accepted or rejected.

We require a zkSNARG to satisfy the following properties.

- **Succinctness:** *There exists polynomial $p(\cdot)$ and a sublinear function $g(x) \in o(x)$ such that for all polynomials $\text{poly}(\cdot)$ and $\lambda, \ell_{\text{stmt}}, \ell_{\text{wit}} \in \mathbb{N}$ where $\ell_{\text{stmt}}, \ell_{\text{wit}} \leq \text{poly}(\lambda)$, the size of the proof π output by Prove for a statement of length ℓ_{stmt} and witness of length ℓ_{wit} is at most $p(\lambda)g(\ell_{\text{wit}})$ and the runtime of Verify is at most $p(\lambda\ell_{\text{stmt}})g(\ell_{\text{wit}})$.*
- **Completeness:** *For all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}_{\text{ZK}}$ we have*

$$\Pr \left[\begin{array}{c} C(x, w) = y \\ \wedge \\ \text{Verify}(\text{crs}, x, y, \pi) = 0 \end{array} \mid \begin{array}{c} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (x, y, w) \leftarrow \mathcal{A}(1^\lambda, \text{crs}) \\ \pi \leftarrow \text{Prove}(\text{crs}, x, y, w) \end{array} \right] \leq \text{negl}(\lambda)$$

- **Soundness:** For all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}_{\text{ZK}}$ we have

$$\Pr \left[\begin{array}{c} (C, x, y) \notin \text{SAT} \\ \wedge \\ \text{Verify}(\text{crs}, x, y, \pi) = 1 \end{array} \middle| \begin{array}{c} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (x, y, \pi) \leftarrow \mathcal{A}(1^\lambda, \text{crs}) \end{array} \right] \leq \text{negl}(\lambda).$$

A zkSNARG is said to have non-adaptive soundness if for all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}_{\text{ZK}}$ we have

$$\Pr \left[\begin{array}{c} (C, x, y) \notin \text{SAT} \\ \wedge \\ \text{Verify}(\text{crs}, x, y, \pi) = 1 \end{array} \middle| \begin{array}{c} (x, y, \text{st}) \leftarrow \mathcal{A}(1^\lambda) \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ \pi \leftarrow \mathcal{A}(\text{st}, \text{crs}) \end{array} \right] \leq \text{negl}(\lambda).$$

- **Zero-Knowledge:** There exists a PPT simulator Sim such that for all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ so that for all $\lambda \in \mathbb{N}$ and circuit $C \in \mathbb{C}_{\text{ZK}}$ we have

$$\left| \Pr \left[\begin{array}{c} \mathcal{A}(\text{st}, \pi) = 1 \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (x, w, \text{st}) \leftarrow \mathcal{A}(1^\lambda, \text{crs}) \\ y := C(x, w) \\ \pi \leftarrow \text{Prove}(\text{crs}, x, y, w) \end{array} \right] - \Pr \left[\begin{array}{c} \mathcal{A}(\text{st}, \pi) = 1 \\ (x, w, \text{st}) \leftarrow \mathcal{A}(1^\lambda, \text{crs}) \\ y := C(x, w) \\ \pi \leftarrow \text{Sim}(\text{st}_s, x, y) \end{array} \right] \right| \leq \text{negl}(\lambda).$$

Succinct Non-interactive Arguments of Knowledge. A zero-knowledge Succinct Non-interactive Argument of Knowledge (zkSNARK) is a zkSNARG that additionally has the knowledge soundness property. Knowledge soundness requires that for all polynomial size adversaries $\mathcal{A}$ there exists a polynomial size extractor $\text{Ext}_{\mathcal{A}}$ and a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}_{\text{ZK}}$ we have

$$\Pr \left[\begin{array}{c} C(x, w) \neq y \\ \wedge \\ \text{Verify}(\text{crs}, x, y, \pi) = 1 \end{array} \middle| \begin{array}{c} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (x, y, \pi) \leftarrow \mathcal{A}(1^\lambda, \text{crs}) \\ w \leftarrow \text{Ext}_{\mathcal{A}}(\text{crs}) \end{array} \right] \leq \text{negl}(\lambda).$$

Batch Arguments. Non-interactive batch arguments (BARGs) allow constructing and verifying proofs for multiple instances of an NP language. In this work, we consider BARGs for boolean circuit satisfiability. Let $C : \{0, 1\}^{\ell_{\text{stmt}}} \times \{0, 1\}^{\ell_{\text{wit}}} \rightarrow \{0, 1\}$ be a boolean circuit consisting of s NAND gates. For a positive integer $n \in \mathbb{N}$, the batch boolean circuit satisfiability language BSAT_n is defined as

$$\text{BSAT}_n = \{(C, \mathbf{x}_1, \dots, \mathbf{x}_n) \mid \forall i \in [1, n] \exists \mathbf{w}_i \in \{0, 1\}^{\ell_{\text{wit}}}, C(\mathbf{x}_i, \mathbf{w}_i) = 1\}.$$

A BARG is required to satisfy the succinctness, completeness and non-adaptive soundness properties, as defined in [Definition 4](#), except that the statement now consists of n circuit inputs as defined in BSAT_n . A BARG that is additionally zero-knowledge is called a zkBARG. Observe that it suffices for a BARG to amortize the cost of verification across multiple instances to achieve succinctness. Specifically, a BARG is succinct if the proof size and verification times are sublinear in $n\ell_{\text{wit}}$ but linear in $n + \ell_{\text{wit}}$. We will also view BSAT_n as the circuit satisfiability language for SIMD boolean circuits. Specifically, an instance $(C, \mathbf{x}_1, \dots, \mathbf{x}_n) \in \text{BSAT}_n$ can alternatively be viewed as an instance of a circuit satisfiability language for a class of SIMD boolean circuits C' such that C' consists of n parallel copies of C and is satisfiable iff each copy of C with input $\mathbf{x}_i$ is satisfiable. Thus, a BARG is essentially a SNARG for SIMD-circuit satisfiability with non-adaptive soundness.

4 Splittable zkSNARG

In this section, we introduce the notion of Splittable zkSNARGs, which are used to capture the properties of zkSNARGs required by our ve-HSS construction. We begin by defining Splittable zkSNARGs, then establish the key properties of these zkSNARGs when proofs are generated by distributed provers, and finally present two instantiations of Splittable zkSNARGs.

The main property of a Splittable zkSNARG is a structural requirement called the *split prover* property, which allows the prover to be split into a non-linear phase followed by a linear phase, where the non-linear phase is “simple” to compute. In our ve-HSS construction, the non-linear phase will be computed using an HSS scheme and the linear phase will be computed over the additive shares obtained from HSS evaluation, leveraging the linearity of additive secret sharing. Since most HSS schemes only support evaluating a restricted class of circuits, we parameterize Splittable zkSNARGs by the class of circuits $\mathbb{C}_{\text{NL}}$ required for computing the non-linear phase of the zkSNARG, when proving satisfiability of circuits in a set $\mathbb{C}_{\text{ZK}}$. Our ve-HSS scheme can then be realized using an HSS scheme that supports evaluating circuits in $\mathbb{C}_{\text{NL}}$.

In addition to the split prover property, a Splittable zkSNARG also satisfies the *succinct randomness* and *robust verification* properties. Succinct randomness requires that the length of the random tape required to compute the proof is sublinear in the size of the circuit being proved satisfiable. This ensures that the size of the input shares in the ve-HSS construction is sublinear in the circuit size. Robust verification is a security property that requires that any adversarially introduced additive error to a valid proof will cause the proof to fail verification. In [Section 4.1](#), we show that the robust verification property naturally lends itself to security in a distributed prover setting. Looking ahead, this is crucial to the security of our ve-HSS construction, ensuring that an adversary does not learn any additional information about the private witness, even if they cause the proof to be computed incorrectly.

Definition 5 (Splittable zkSNARG). *Let $\mathbb{C}_{\text{NL}}$ and $\mathbb{C}_{\text{ZK}}$ be two classes of circuits. A zkSNARG $\text{SNARG} = (\text{Setup}, \text{Prove}, \text{Verify})$ is said to be a $\mathbb{C}_{\text{NL}}$ -simple Splittable zkSNARG for $\mathbb{C}_{\text{ZK}}$ -circuit satisfiability if it additionally satisfies the following properties.*

- **Split Prover:** *There exist PPT algorithm Prove_{NL} and polynomial time algorithm Prove_{L} such that for any circuit $C \in \mathbb{C}_{\text{ZK}}$, we have the following.*
 - $\text{Prove}_{\text{NL}}(\text{crs}, C, x, y, w) \rightarrow M_{\text{NL}}$ *takes the common reference string crs, circuit C, statement (x, y) and witness w as input and outputs a tuple M_{NL} .*
 - $\text{Prove}_{\text{L}}(\text{crs}, M_{\text{NL}}) =: \pi$ *takes M_{NL} as input and outputs a proof π . The map $\text{Prove}_{\text{L}}(\text{crs}, \cdot)$ is a group homomorphism.*

We require that for all $C \in \mathbb{C}_{\text{ZK}}$

$$\left\{ \pi \left| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ \pi \leftarrow \text{Prove}(\text{crs}, x, y, w) \end{array} \right. \right\}_{\lambda, x, w} \stackrel{c}{\approx} \left\{ \pi \left| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ M_{\text{NL}} \leftarrow \text{Prove}_{\text{NL}}(\text{crs}, C, x, y, w) \\ \pi := \text{Prove}_{\text{L}}(\text{crs}, M_{\text{NL}}) \end{array} \right. \right\}_{\lambda, x, w}$$

where $\lambda \in \mathbb{N}$ and $y = C(x, w)$.

- **Robust Verification:** *For all polynomial size adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}_{\text{ZK}}$, we have*

$$\Pr \left[\begin{array}{l} C(x, w) = y \wedge \\ \epsilon \neq 0 \wedge \\ \text{Verify}(\text{crs}, x, y, \bar{\pi}) = 1 \end{array} \left| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (x, y, w, \epsilon) \leftarrow \mathcal{A}(1^\lambda, \text{crs}) \\ \pi \leftarrow \text{Prove}(\text{crs}, x, y, w) \\ \bar{\pi} := \pi + \epsilon \end{array} \right. \right] \leq \text{negl}(\lambda).$$

- **Efficiency:** A $\mathbb{C}_{\text{NL}}$ -simple Splittable zkSNARG should satisfy the following efficiency properties.
 - **$\mathbb{C}_{\text{NL}}$ -simple:** For every circuit $C \in \mathbb{C}_{\text{ZK}}$ there exists an efficiently constructible circuit $C_{\text{NL}} \in \mathbb{C}_{\text{NL}}$ such that for all inputs x and w , and randomness r we have $C_{\text{NL}}(x, w, r) = \text{Prove}_{\text{NL}}(\text{crs}, C, x, y, w; r)$ where $y = C(x, w)$.
 - **Succinct Randomness:** There exists polynomial p and a sublinear function $g(x) \in o(x)$ such that for all $\lambda \in \mathbb{N}$ and all ℓ_{inp} -input circuits $C \in \mathbb{C}_{\text{ZK}}$, the length of the random tape required by Prove_{NL} is at most $p(\lambda \ell_{\text{inp}})g(|C|)$.

We note that $\mathbb{C}_{\text{NL}}$ is parameterized by the CRS though we omit it from the notation for simplicity. If the Splittable zkSNARG additionally satisfies knowledge soundness, we say it is a Splittable zkSNARK. We will say a zkSNARG is a Splittable zkSNARG, and omit the description of $\mathbb{C}_{\text{ZK}}$ and $\mathbb{C}_{\text{NL}}$, when only referring to the split prover and succinct randomness property of the zkSNARG.

Splittable zkSNARGs from Linear PCPs. It is easy to see that zkSNARGs based on linear PCPs [BCI⁺13] satisfy the structural properties of a Splittable zkSNARG. In more detail, a linear PCP is an oracle where responses to verification queries are computed as inner products between a PCP string and the query. A zkSNARG can be obtained from a linear PCP by embedding the verifier’s queries in the common reference string (CRS) using a linearly homomorphic encoding scheme. The zkSNARG prover holding the witness can first compute the PCP string and then leverage the linear homomorphism of the encoding scheme to compute the inner product of the verification queries embedded in the CRS with the PCP string. These encodings of the inner product serve as the zkSNARG proof. The encoding scheme also ensures privacy of the verification queries embedded in the CRS, which in turn ensures soundness of the zkSNARG. Thus, the prover of a zkSNARG scheme based on linear PCPs can be naturally split into a linear and non-linear phase — the computation of the linear PCP constitutes the non-linear phase and using the CRS to compute the responses to the embedded verification queries constitutes the linear phase.

We also note that zkSNARGs based on linear PCPs can be easily modified to realize the robust verification property.¹⁵ In particular, the setup additionally samples a MAC key K uniformly at random, and the non-linear phase computes the linear PCP M_{NL} as well as a MAC on the linear PCP $K \cdot M_{\text{NL}}$. The linear homomorphism of the encoding scheme allows evaluating the linear phase on M_{NL} and $K \cdot M_{\text{NL}}$ to compute a proof π and a MAC on the proof $K \cdot \pi$. The modified proof π' consists of $\pi' = (\pi, K \cdot \pi)$. The verification algorithm, given the MAC key K and a proof $\pi' = (\pi_1, \pi_2)$, first verifies the MAC on the proof by checking if $K \cdot \pi_1 = \pi_2$ and then runs the zkSNARG verifier on π_1 . It is easy to see that the resulting proof preserves zero-knowledge, soundness and the split-prover property. Moreover, it has robust verification because any additive error to the proof will be detected with probability at least $1/2$ by the MAC-check; which can in turn be amplified to detect error with overwhelming probability by computing λ independent MACs. The modified proof system, however, is *designated-verifier*, since it requires the verifier to store the MAC keys generated during setup. Nevertheless, such a proof system remains compatible with our ve-HSS construction, up to some syntactic modifications—specifically, the sharing algorithm will output both the server shares and the private client state.

In Sections 4.2 and 4.3, we discuss two Splittable zkSNARGs, where the split prover property of the zkSNARGs follow from the fact they are based on linear PCPs. Moreover, we show that these zkSNARGs already satisfy the robust verification property, without any modifications. In particular, these zkSNARGs are publicly verifiable and have robust verification, unlike the generic transformation described previously. The non-linear phases of these zkSNARGs can be computed by a large number of existing HSS schemes, and since it does not make use of any cryptographic primitives, computing it within HSS does not make non-black box use of cryptography.

¹⁵Suggested by an anonymous TCC 2024 reviewer.

4.1 Distributed Prover Setting

As discussed earlier, our ve-HSS construction separates the work of a Splittable zkSNARG prover by running the non-linear phase within an HSS scheme and computing the linear phase over additive shares obtained from HSS evaluation. In this section, we show that Splittable zkSNARGs, when computed in a distributed manner, naturally provide completeness and security properties.

In more detail, we consider a set of distributed provers who are given an additive sharing of the output of the non-linear phase M_{NL} of the Splittable zkSNARG. We aim to show that the provers can non-interactively compute additive shares of a valid proof and that if any subset of the provers behave maliciously, the reconstructed proof will fail to verify. Since the non-linear phase is run within HSS, the security of the ve-HSS construction will follow from the security of the HSS scheme and the security of the Splittable zkSNARG in the distributed prover setting.

DP-Completeness. Distributed prover completeness guarantees that the provers, given additive shares of M_{NL} , can non-interactively compute additive shares of a valid zkSNARG proof. In the definition, the adversary is allowed to choose the shares of M_{NL} for corrupt provers. Intuitively, this means that the reconstructed proof must verify successfully irrespective of how corrupt provers obtain their shares of M_{NL} . In case of our ve-HSS construction, these shares will be computed using an HSS scheme. Without loss of generality, we consider adversaries that corrupt all but one prover.

Lemma 6 (DP-Completeness). *Let SNARG be a Splittable zkSNARG for $\mathbb{C}_{\text{ZK}}$ -circuit satisfiability. Then for all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, m \in \mathbb{N}$, where $m > 1$, and circuits $C \in \mathbb{C}_{\text{ZK}}$ we have*

$$\Pr \left[\begin{array}{c} C(x, w) = y \\ \wedge \\ \text{Verify}(\text{crs}, x, y, \pi) = 0 \end{array} \middle| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (x, y, w, \mathcal{I}, [M_{\text{NL}}]_{\mathcal{I}}) \leftarrow \mathcal{A}(1^\lambda, \text{crs}) \\ M_{\text{NL}} \leftarrow \text{Prove}_{\text{NL}}(\text{crs}, C, x, y, w) \\ [M_{\text{NL}}]_{i^*} := M_{\text{NL}} - \sum_{i \in \mathcal{I}} [M_{\text{NL}}]_i \\ [\pi]_i := \text{Prove}_{\text{L}}(\text{crs}, [M_{\text{NL}}]_i), \forall i \in [1, m] \\ \pi := \sum_{i=1}^m [\pi]_i \end{array} \right] \leq \text{negl}(\lambda)$$

where $\mathcal{I} \subset [1, m]$ is a set of $m - 1$ corrupt provers and $i^* \notin \mathcal{I}$ is the index of the honest prover.

Proof. Observe that $[M_{\text{NL}}] = ([M_{\text{NL}}]_1, \dots, [M_{\text{NL}}]_m)$ is an additive sharing of M_{NL} , the output of the non-linear phase of the zkSNARG. Since Prove_{L} is a group homomorphism, $[\pi] = ([\pi]_1, \dots, [\pi]_m)$ is an additive sharing of the proof $\pi := \text{Prove}_{\text{L}}(\text{crs}, M_{\text{NL}})$. The split prover and completeness properties of SNARG then imply that verification of π fails with at most negligible probability when $C(x, w) = y$. ■

DP-Robust verification. Distributed prover robust verification guarantees that a proof reconstructed using incorrect shares sent by corrupt provers will fail to verify. Specifically, since the linear phase of the Splittable zkSNARG is deterministic, the proof π is completely determined by the CRS and the output M_{NL} of the non-linear phase. DP-Robust verification then guarantees that if corrupt provers behave maliciously and output incorrect shares of the proof, such that the reconstructed proof $\bar{\pi}$ is not equal to the proof π , then $\bar{\pi}$ will fail to verify with overwhelming probability. We will show that robust verification in the distributed prover setting is implied by the robust verification property of Splittable zkSNARGs. As in the case of DP-completeness, we allow the adversary to choose the corrupt provers' shares of M_{NL} and we consider adversaries that corrupt all but one prover without loss of generality.

This property helps provide a strong privacy guarantee when the proof for a statement consisting of private data is computed in a distributed manner. In particular, DP-robust verification ensures that corrupt provers cannot compromise the privacy of the statement by maliciously computing their proof shares

and learning if the reconstructed proof verifies. This is because any error in the reconstructed proof is introduced only by corrupt parties and verification is guaranteed to fail with overwhelming probability in case of non-zero error—this implies that corrupt parties could have themselves determined if the reconstructed proof verifies successfully. Looking ahead, this is crucial to proving the privacy of our ve-HSS construction.

Lemma 7 (DP-Robust Verification). *Let SNARG be a Splittable zkSNARG for $\mathbb{C}_{\text{ZK}}$ -circuit satisfiability and let $m \in \mathbb{N}$. Then for all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda, m \in \mathbb{N}$, where $m > 1$, and circuits $C \in \mathbb{C}_{\text{ZK}}$ we have*

$$\Pr \left[\begin{array}{l} C(x, w) = y \\ \wedge \\ \epsilon \neq 0 \\ \wedge \\ \text{Verify}(\text{crs}, x, y, \bar{\pi}) = 1 \end{array} \middle| \begin{array}{l} (x, y, w, \mathcal{I}, [M_{\text{NL}}]_{\mathcal{I}}, [\bar{\pi}]_{\mathcal{I}}) \leftarrow \mathcal{A}(1^\lambda, \text{crs}) \\ [M_{\text{NL}}]_{i^*} := M_{\text{NL}} - \sum_{i \in \mathcal{I}} [M_{\text{NL}}]_i \\ [\pi]_i := \text{Prove}_{\text{L}}(\text{crs}, [M_{\text{NL}}]_i), \forall i \in [1, m] \\ [\bar{\pi}]_{i^*} := [\pi]_{i^*} \\ \bar{\pi} := \sum_{i=1}^m [\bar{\pi}]_i \\ \epsilon := \sum_{i=1}^m [\bar{\pi}]_i - [\pi]_i \end{array} \right] \leq \text{negl}(\lambda)$$

where $\text{crs} \leftarrow \text{Setup}(1^\lambda, C)$, $M_{\text{NL}} \leftarrow \text{Prove}_{\text{NL}}(\text{crs}, C, x, y, w)$, $\mathcal{I} \subset [1, m]$ is a set of $m - 1$ corrupt provers and $i^* \notin \mathcal{I}$ is the index of the honest prover.

Proof. Robust verification in the distributed prover setting follows from robust verification of the Splittable zkSNARG since the adversary's view in the experiment is independent of M_{NL} and hence the proof π . More formally, let the output of the experiment be defined as 1 when $C(x, w) = y$, $\epsilon \neq 0$ but Verify outputs 1, and be defined as 0 otherwise. We use a hybrid argument to show that for any polynomial size adversary $\mathcal{A}$, the output of the experiment is 1 with at most negligible probability.

- Hyb_0 : This corresponds to the output of the experiment when run with adversary $\mathcal{A}$.
- Hyb_1 : In this hybrid, the experiment computes $\bar{\pi} := \pi + \epsilon$ where $\pi := \text{Prove}_{\text{L}}(\text{crs}, M_{\text{NL}})$ and $\epsilon := \sum_{i=1}^m [\bar{\pi}]_i - [\pi]_i$.

Observe that in Hyb_0 , $[M_{\text{NL}}]$ is an additive sharing of $M_{\text{NL}} \leftarrow \text{Prove}_{\text{NL}}(\text{crs}, C, x, y, w)$. The fact that Prove_{L} is a group homomorphism implies that $[\pi]$ computed in Hyb_0 is an additive sharing of the proof $\pi := \text{Prove}_{\text{L}}(\text{crs}, M_{\text{NL}})$ computed in this hybrid. We have

$$\bar{\pi} = \sum_{i=1}^m [\bar{\pi}]_i = \pi + \sum_{i=1}^m [\bar{\pi}]_i - [\pi]_i = \pi + \epsilon$$

where the first equality follows from the definition of $\bar{\pi}$, the second equality follows from the fact that $[\pi]$ constitutes an additive sharing of π and the third equality follows from the definition of ϵ . It follows that $\text{Hyb}_0 \stackrel{d}{=} \text{Hyb}_1$.

- Hyb_2 : In this hybrid, the experiment computes $\bar{\pi} := \pi' + \epsilon$ where $\pi' \leftarrow \text{Prove}(\text{crs}, x, y, w)$ and ϵ is computed as in the previous hybrid.
- $\text{Hyb}_1 \stackrel{c}{\approx} \text{Hyb}_2$ since the split prover property of SNARG implies that $\bar{\pi}$ computed in Hyb_1 is computationally indistinguishable from $\pi' + \epsilon$.
- Hyb_3 : In this hybrid, the experiment always outputs 0.

Observe that Hyb_2 is 1 only when $\epsilon \neq 0$ but $\bar{\pi}$ verifies successfully. We show that this occurs with at most negligible probability by a reduction to the robust verification property of SNARG. Consider an adversary $\mathcal{A}'$ against the robust verification of SNARG that does the following. It first runs $\mathcal{A}$ to

obtain $(x, y, w, \mathcal{I}, [M_{\text{NL}}]_{\mathcal{I}}, [\bar{\pi}]_{\mathcal{I}})$ and then computes $[\pi]_{\mathcal{I}} := \text{Prove}_{\text{L}}(\text{crs}, [M_{\text{NL}}]_{\mathcal{I}})$. It then computes $\epsilon := \sum_{i \in \mathcal{I}} [\bar{\pi}]_i - [\pi]_i$ and outputs (x, y, w, ϵ) .

Since $[\bar{\pi}]_{i^*} = [\pi]_{i^*}$ for the honest prover i^* , ϵ computed in the experiment in Hyb_2 is identical to the one computed by $\mathcal{A}'$. Thus, if Hyb_2 is 1 then $\mathcal{A}'$ wins the robust verification experiment. It follows from the robust verification property of SNARG that Hyb_2 is 1 with at most negligible probability which in turn implies that $\text{Hyb}_2 \stackrel{\epsilon}{\approx} \text{Hyb}_3$.

It follows from our sequence of hybrids that $\text{Hyb}_0 \stackrel{\epsilon}{\approx} \text{Hyb}_3$ and since Hyb_3 is always 0, any adversary $\mathcal{A}$ wins the DP-robust verification experiment with at most negligible probability. $\blacksquare$

4.2 The Groth16 zkSNARK

In this section we describe the Groth16 [Gro16] zkSNARK and show that it is a Splittable zkSNARK. We start by defining bilinear groups and Quadratic Arithmetic Programs (QAP), both of which will be important for describing the Groth16 zkSNARK.

Prime-order Bilinear Groups. Let $\mathcal{G}$ be a group generator such that $(p, \mathbb{G}_1, \mathbb{G}_T, e, g_1) \leftarrow \mathcal{G}(1^\lambda)$ be bilinear groups where $\mathbb{G}_1$ and $\mathbb{G}_T$ have prime order p and are equipped with efficient algorithms to carry out group operations, the pairing $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_T$ is an efficiently computable non-degenerate bilinear map, g_1 is a generator for $\mathbb{G}_1$ and $g_T = e(g_1, g_1)$ is a generator for $\mathbb{G}_T$. For any $\mathbf{x} = (x_1, \dots, x_\ell) \in \mathbb{Z}_p^\ell$, we use $g_i^{\mathbf{x}} = (g_i^{x_1}, \dots, g_i^{x_\ell})$ to denote the vector of group elements in $\mathbb{G}_i^\ell$ where $i \in \{1, T\}$.

Quadratic Arithmetic Programs. Given an arithmetic circuit consisting of addition and multiplication gates over a finite field $\mathbb{F}$, one can assign a set of input wires $\mathbf{x}$ and output wires $\mathbf{y}$ as specifying a statement $(\mathbf{x}, \mathbf{y})$ and all other wires in the circuit as defining a witness $\mathbf{w}$. This in turn defines a relation R consisting of values on statement and witness wires that satisfy the circuit. A *Quadratic Arithmetic Program (QAP)* [GGPR13] uses the *same* statement $(\mathbf{x}, \mathbf{y})$ and witness $\mathbf{w}$ but encodes the circuit satisfiability constraints as polynomials over $\mathbb{F}$. Specifically, given a circuit C , a QAP defines the relation

$$\mathcal{R}_C^{\text{qap}} = \left\{ (\mathbf{x}, \mathbf{y}; \mathbf{w}) \mid \begin{array}{l} \mu_0 = 1 \\ (\mathbf{x}, \mathbf{y}) = (\mu_1, \dots, \mu_{\ell_{\text{stmt}}}) \in \mathbb{F}^{\ell_{\text{stmt}}} \\ \mathbf{w} = (\mu_{\ell_{\text{stmt}}+1}, \dots, \mu_L) \in \mathbb{F}^{L-\ell_{\text{stmt}}} \\ \sum_{i=0}^L \mu_i \rho_{1,i}(X) \cdot \sum_{i=0}^L \mu_i \rho_{2,i}(X) \equiv \sum_{i=0}^L \mu_i \rho_{3,i}(X) \pmod{\rho_{\text{mod}}(X)} \end{array} \right\}$$

where $\rho_{\text{mod}} \in \mathbb{F}[X]$ is of degree d and $\rho_{1,i}, \rho_{2,i}, \rho_{3,i} \in \mathbb{F}[X]$ are of degree strictly lesser than d for every $i \in [0, L]$. The polynomials $\rho_{1,i}, \rho_{2,i}, \rho_{3,i}, \rho_{\text{mod}}$ encode the satisfiability constraints for C while $(\mathbf{x}, \mathbf{y})$ and $\mathbf{w}$ correspond to values on the statement and witness wires as described previously. We refer to $\mu_0 \|\mathbf{x}\| \|\mathbf{y}\| \|\mathbf{w}\| = (\mu_0, \dots, \mu_L)$ as the extended witness of the QAP instance.

Remark 2 (QAPs For Boolean Circuits). *We note that a QAP over any finite field $\mathbb{F}$, can be used to encode circuit satisfiability constraints for boolean circuits. Specifically, given a boolean circuit C , we can simulate C using an arithmetic circuit $C_{\mathbb{F}}$ over $\mathbb{F}$ by mapping the 0 bit input to the additive identity $0_{\mathbb{F}}$, the 1 bit input to the multiplicative identity $1_{\mathbb{F}}$, and computing each NAND gate with inputs a and b in C as $1 - ab$ in $C_{\mathbb{F}}$. We can then encode satisfiability constraints for $C_{\mathbb{F}}$ using a QAP over $\mathbb{F}$.*

We recall the zkSNARK presented by Groth [Gro16] in Figure 2, which we denote by Groth16. While Groth16 is a zkSNARK for QAP relations, we present it as a zkSNARK for boolean circuit satisfiability for ease of use in our main constructions. Our primary goal is to demonstrate that it is a Splittable zkSNARK and so we omit a number of details in order to provide a simpler description of the scheme that suffices for our purposes. We refer the reader to [Gro16] for a complete description. While [Gro16] uses asymmetric

The Groth16 zkSNARK

Public parameters. A group generator $\mathcal{G}$ for prime-order bilinear groups.

Groth16.Setup($1^\lambda, C$)

- 1: Takes a boolean circuit $C \in \mathbb{C}_{\text{ZK}}$ as input, runs $(p, \mathbb{G}_1, \mathbb{G}_T, e, g_1) \leftarrow \mathcal{G}(1^\lambda)$ and outputs a common reference string crs consisting of the description of the bilinear group and a vector of elements in $\mathbb{G}_1$ of the form

$$\begin{pmatrix} Q_{a,0}, \dots, Q_{a,L+1} \\ Q_{b,0}, \dots, Q_{b,L+1} \\ Q_{c,0}, \dots, Q_{c,L+d-1} \\ Q_{v,1}, Q_{v,2} \end{pmatrix}.$$

Groth16.Prove(crs, $\mathbf{x}, \mathbf{y}, \mathbf{w}$)

- 1: Parse $(\mathbf{x}, \mathbf{y}) = (\mu_1, \dots, \mu_{\ell_{\text{stmt}}})$.
- 2: Compute the extended witness $(\mu_1, \dots, \mu_L)$ for the corresponding QAP instance.
- 3: Compute the polynomial $\rho_{\text{div}}(X)$ of degree at most $d - 2$ such that

$$\rho_{\text{div}}(X) = \frac{\sum_{j,k \in [0,L]} \rho_{1,j}(X) \cdot \mu_j \mu_k \cdot \rho_{2,k}(X) - \sum_{j=0}^L \mu_j \rho_{3,j}(X)}{\rho_{\text{mod}}(X)}. \quad (1)$$

where $\rho_{1,i}(X), \rho_{2,i}(X), \rho_{3,i}(X)$ for every $i \in [0, L]$ and $\rho_{\text{mod}}(X)$ are in $\mathbb{Z}_p[X]$, defined by $\mathcal{R}_C^{\text{qap}}$.

- 4: Sample $r, s \leftarrow \mathbb{Z}_p$ uniformly at random and compute the proof $\boldsymbol{\pi} = (\pi_a, \pi_b, \pi_c)$ as

$$\begin{aligned} \pi_a &= Q_{a,L+1} \cdot Q_{v,2}^r \cdot \prod_{i=0}^L Q_{a,i}^{\mu_i} \\ \pi_b &= Q_{b,L+1} \cdot Q_{v,2}^s \cdot \prod_{i=0}^L Q_{b,i}^{\mu_i} \\ \pi_c &= \frac{\pi_a^s \cdot \pi_b^r \cdot \prod_{i=\ell_{\text{stmt}}+1}^L Q_{c,i}^{\mu_i} \cdot \prod_{i=0}^{d-2} Q_{c,L+1+i}^{\rho_{\text{div}}[i]}}{Q_{v,2}^{rs}} \end{aligned}$$

where $\rho_{\text{div}}[i]$ denotes the i -th co-efficient of $\rho_{\text{div}}(X)$.

Groth16.Verify(crs, $\mathbf{x}, \mathbf{y}, \boldsymbol{\pi}$)

- 1: Parse $(\mathbf{x}, \mathbf{y}) = (\mu_1, \dots, \mu_{\ell_{\text{stmt}}})$ and $\boldsymbol{\pi} = (\pi_a, \pi_b, \pi_c)$.
- 2: Output 1 if

$$e(\pi_a, \pi_b) = e(Q_{a,L+1}, Q_{b,L+1}) \cdot e\left(\prod_{i=0}^{\ell_{\text{stmt}}} Q_{c,i}^{\mu_i}, Q_{v,1}\right) \cdot e(\pi_c, Q_{v,2})$$

and outputs 0 otherwise.

Figure 2: The Groth16 zkSNARK [Gro16].

bilinear groups for the construction, our presentation uses symmetric bilinear groups for ease of exposition. We note that the scheme is secure even in this case [Gro16]. Additionally, our arguments in favor of Groth16 being Splittable when instantiated with symmetric bilinear groups extends directly to the case when it is instantiated with asymmetric bilinear groups too. The following theorem immediately follows from the fact that Groth16 is a zkSNARK and we show in Theorem 9 that it is additionally a Splittable zkSNARK.

Theorem 8 ([Gro16]). *Groth16 (Figure 2) is a zkSNARK for boolean circuit satisfiability in the generic bilinear group model.*

Theorem 9. *Groth16 (Figure 2) is a Splittable zkSNARK for boolean circuit satisfiability (Definition 5) in the generic bilinear group model. Moreover, for every polynomial size boolean circuit C of depth D , there exists an efficiently constructible polynomial size circuit C_{NL} over $\mathbb{Z}_p$ with multiplicative depth $O(D)$ such that for all crs in the support of Setup, all $(C, \mathbf{x}, \mathbf{y}; \mathbf{w}) \in \text{SAT}$ and all $\mathbf{r} \in \mathbb{Z}_p^2$, we have $C_{NL}(\mathbf{x}, \mathbf{w}, \mathbf{r}) = \text{Prove}_{NL}(crs, C, \mathbf{x}, \mathbf{y}, \mathbf{w}; \mathbf{r})$, where p is the order of the bilinear group.*

Proof. We first show that Groth16 is split prover and then argue that it has robust verification.

Claim 9.1. *Groth16 is split prover such that for every polynomial size boolean circuit C there exists an efficiently constructible circuit C_{NL} as described above.*

Proof. Consider the following split of the Groth16 prover into a non-linear and linear phase.

- Groth16.Prove_{NL}: It takes takes the statement $(\mathbf{x}, \mathbf{y})$, witness $\mathbf{w}$ and uniform random values $r, s \leftarrow \mathbb{Z}_p$ as input and outputs the following.
 1. The extended witness $(1, \mu_0, \dots, \mu_L)$ for the corresponding QAP relation.
 2. The random inputs, their product, and the product of each random input with the extended witness $(r, s, rs, r\mu_1, \dots, r\mu_L, s\mu_1, \dots, s\mu_L)$.
 3. The co-efficients of the polynomial $\rho_{\text{div}}(X)$, defined in Equation (1).
- Groth16.Prove_L: It takes the output of the non-linear phase as input and computes $\boldsymbol{\pi} = (\pi_a, \pi_b, \pi_c)$ as

$$\begin{aligned}\pi_a &= Q_{a,L+1} \cdot Q_{v,2}^r \cdot \prod_{i=0}^L Q_{a,i}^{\mu_i} \\ \pi_b &= Q_{b,L+1} \cdot Q_{v,2}^s \cdot \prod_{i=0}^L Q_{b,i}^{\mu_i} \\ \pi_c &= Q_{v,2}^{rs} \cdot \left(Q_{a,L+1}^s \cdot \prod_{i=0}^L Q_{a,i}^{s\mu_i} \right) \cdot \left(Q_{b,L+1}^r \cdot \prod_{i=0}^L Q_{b,i}^{r\mu_i} \right) \cdot \prod_{i=\ell_{\text{stmt}}+1}^L Q_{c,i}^{\mu_i} \cdot \prod_{i=0}^{d-2} Q_{c,L+1+i}^{\rho_{\text{div}}[i]}\end{aligned}$$

where we highlight the output of the non-linear phase in red for clarity.

It is easy to see that Groth16.Prove_L is a homomorphism from tuples in $\mathbb{Z}_p$ to tuples in $\mathbb{G}_1$. The split prover property then follows from the fact that Groth16.Prove_{NL} and Groth16.Prove_L constitute a partition of the steps of Groth16.Prove.

Given any polynomial size boolean circuit C , we next describe the circuit C_{NL} that computes the output of the non-linear phase when proving satisfiability of C . As discussed in Remark 2, the extended witness for C corresponds to the value on each wire of the circuit $C_{\mathbb{Z}_p}$ that emulates C over $\mathbb{Z}_p$. Thus, the extended witness can be computed by a circuit over $\mathbb{Z}_p$ with multiplicative depth $O(D)$. Note that the *boolean*

statement and witness are implicitly mapped to $0, 1 \in \mathbb{Z}_p$ when used as input to C_{NL} , as described in [Section 3](#). Next, observe that given the extended witness, each value in $(rs, r\mu_1, \dots, r\mu_L, s\mu_1, \dots, s\mu_L)$ can be computed in parallel and only increments the multiplicative depth by 1. Finally, to compute the co-efficients of $\rho_{\text{div}}(X)$, it suffices to first evaluate the right-hand side of [Equation \(1\)](#) at $d - 1$ distinct points that do not contain a root of $\rho_{\text{mod}}(X)$, and then reconstruct the co-efficients from these evaluations. Since the polynomials $\{\rho_{1,i}, \rho_{2,i}, \rho_{3,i}\}_{i=1}^L$ are completely defined by C and are independent of the statement and witness, their evaluations at $d - 1$ distinct points can be hardcoded into C_{NL} . Thus, each co-efficient of $\rho_{\text{div}}(X)$ can be computed as a linear combination of the pairwise product of the extended witness $\{\mu_i \mu_j\}_{i,j}$. Since computing $\{\mu_i \mu_j\}_{i,j}$ requires evaluating an additional layer of multiplication gates, it follows that C_{NL} is a polynomial size circuit over $\mathbb{Z}_p$ with a multiplicative depth of $O(D)$. Moreover, it follows from our description of C_{NL} and the fact that the degree of $\rho_{\text{mod}}(x)$ and each $\rho_{1,i}, \rho_{2,i}, \rho_{3,i}$ is polynomial in $|C|$ that C_{NL} is efficiently constructible given C . $\square$

Claim 9.2. *Groth16 has robust verification.*

Proof. Consider an adversary $\mathcal{A}$ against the robust verification property of Groth16 that outputs a statement $(\mathbf{x}, \mathbf{y})$, witness $\mathbf{w}$ and an error $\epsilon = (g_1^{\epsilon_a}, g_1^{\epsilon_b}, g_1^{\epsilon_c})$ where ϵ_a, ϵ_b and ϵ_c are in $\mathbb{Z}_p$. Let the proof π be of the form $\pi = (g_1^{z_a}, g_1^{z_b}, g_1^{z_c})$. Since we are using a multiplicative group, we have $\bar{\pi} = (g_1^{z_a + \epsilon_a}, g_1^{z_b + \epsilon_b}, g_1^{z_c + \epsilon_c})$. We argue that when $C(\mathbf{x}, \mathbf{w}) = \mathbf{y}$ and at least one among ϵ_a, ϵ_b and ϵ_c is non-zero then $\text{Groth16.Verify}(\text{crs}, \mathbf{x}, \mathbf{y}, \mathbf{w}, \bar{\pi}) = 1$ with at most negligible probability. Informally, our argument relies on the fact that $r, s \leftarrow \mathbb{Z}_p$ sampled uniformly at random for zero-knowledge of Groth16 also ensure robust verification.

From the completeness of Groth16 we have

$$e(g_1^{z_a}, g_1^{z_b}) = e(Q_{a,L+1}, Q_{a,L+1}) \cdot e\left(\prod_{i=0}^{\ell_{\text{stmt}}} Q_{c,i}^{\mu_i}, Q_{v,1}\right) \cdot e(g_1^{z_c}, Q_{v,2}) \quad (2)$$

with overwhelming probability where $(\mathbf{x}, \mathbf{y}) = (\mu_1, \dots, \mu_{\ell_{\text{stmt}}})$. Now, observe that

$$\begin{aligned} \text{Groth16.Verify}(\text{crs}, \mathbf{x}, \mathbf{y}, \bar{\pi}) &= 1 \\ \implies e(g_1^{z_a + \epsilon_a}, g_1^{z_b + \epsilon_b}) &= e(Q_{a,L+1}, Q_{b,L+1}) \cdot e\left(\prod_{i=0}^{\ell_{\text{stmt}}} Q_{c,i}^{\mu_i}, Q_{v,1}\right) \cdot e(g_1^{z_c + \epsilon_c}, Q_{v,2}) \\ &\implies g_T^{z_a \epsilon_b} \cdot g_T^{z_b \epsilon_a} \cdot g_T^{\epsilon_a \epsilon_b} = e(g_1^{\epsilon_c}, Q_{v,2}) \\ &\implies g_T^{z_a \epsilon_b + z_b \epsilon_a + \epsilon_a \epsilon_b - \epsilon_c q_{v,2}} = 1 \end{aligned}$$

holds with overwhelming probability where $Q_{v,2} = g_1^{q_{v,2}}$ and the second implication follows from [Equation \(2\)](#) and the definition of $\bar{\pi}$.

Since r and s are uniformly random over $\mathbb{Z}_p$ and independent of $\mathcal{A}$'s view, z_a and z_b are uniformly random over $\mathbb{Z}_p$ and are independent of ϵ . Therefore, when either $\epsilon_a \neq 0$ or $\epsilon_b \neq 0$ then from the Schwartz-Zippel lemma, $z_a \epsilon_b + z_b \epsilon_a - \epsilon_c q_{v,2} + \epsilon_a \epsilon_b = 0$ over $\mathbb{Z}_p$ with probability at most $1/|\mathbb{Z}_p| = 1/p \in \text{negl}(\lambda)$. Moreover, $q_{v,2} \neq 0$ by definition [\[Gro16\]](#) which implies that $\epsilon_c q_{v,2} = 0$ only when $\epsilon_c = 0$. Thus, $g_T^{z_a \epsilon_b + z_b \epsilon_a + \epsilon_a \epsilon_b - \epsilon_c q_{v,2}} = 1$ with at most negligible probability when at least one among ϵ_a, ϵ_b or ϵ_c is non-zero which implies that $\text{Groth16.Verify}(\text{crs}, \mathbf{x}, \mathbf{y}, \bar{\pi}) = 1$ with at most negligible probability in this case. $\square$

Since $\text{Groth16.Prove}_{\text{NL}}$ only requires two uniformly random value from $\mathbb{Z}_p$, each of which is $O(\log \lambda)$ bits long, it follows from [Theorem 8](#) and [Claims 9.1](#) and [9.2](#) that Groth16 is a Splittable zkSNARG for boolean circuit satisfiability in the generic bilinear group model. $\blacksquare$

Theorem 9 implies that, in terms of circuit depth, the non-linear phase for Groth16 is as easy to compute as the circuit C defined by the statement. Since many HSS schemes only support evaluating circuits with bounded depth, this ensures that for any function computed within HSS, the corresponding non-linear phase for proving correctness of the computation can also be evaluated within the same HSS scheme¹⁶. Moreover, observe that the non-linear phase, as described in the proof of **Theorem 9**, does not require performing any group operations. Consequently, describing C_{NL} does not require making non-black-box use of the group.

Remark 3 (Beyond boolean satisfiability). *While we will primarily focus on boolean circuit satisfiability, the proof of **Theorem 9** extends naturally to a stronger result: for any circuit C over $\mathbb{Z}_p$ with multiplicative depth D , the non-linear phase for proving satisfiability of C can be computed by a circuit C_{NL} over $\mathbb{Z}_p$ with multiplicative depth $O(D)$. In fact, if C is a polynomial size RMS program with modulus p , then it can be shown that C_{NL} is also a polynomial size RMS program with modulus p . Since a depth D computation over $\mathbb{Z}_p$ might incur a multiplicative overhead of $O(\log \lambda)$ when expressed as a boolean circuit, this allows evaluating a broader class of functions without increasing the complexity of the non-linear phase.*

4.3 The Waters-Wu zkBARG

In this section we recall the Waters-Wu Batch Argument (BARG) [WW22] and discuss a variant of the construction that achieves perfect zero-knowledge. We then proceed to show that the zero-knowledge variant of the Waters-Wu BARG is Splittable. We start by defining composite-order bilinear groups and then continue to discuss the details of the Waters-Wu BARG.

Composite-order Bilinear Groups. Let $\mathcal{G}$ be a group generator such that $(p, q, \mathbb{G}, \mathbb{G}_T, e, g) \leftarrow \mathcal{G}(1^\lambda)$ consists of two cyclic groups $\mathbb{G}$ and $\mathbb{G}_T$ of order $N = pq$ where p, q are primes of magnitude $\Theta(2^\lambda)$, a generator g for $\mathbb{G}$ and an efficiently computable pairing $e : \mathbb{G} \times \mathbb{G} \rightarrow \mathbb{G}_T$ that is a non-degenerate bilinear map. Let $g_T = e(g, g)$ be the generator of $\mathbb{G}_T$. By the Chinese Remainder Theorem, we can write $\mathbb{G} = \mathbb{G}_p \times \mathbb{G}_q$ where $\mathbb{G}_p$ is a subgroup of order p (generated by $g_p = g^q$) and $\mathbb{G}_q$ is a subgroup of order q (generated by $g_q = g^p$). The subgroups $\mathbb{G}_p$ and $\mathbb{G}_q$ are orthogonal i.e., $e(g_p, g_q) = 1$, where 1 denotes the identity element in $\mathbb{G}_T$.

Definition 6 (Subgroup Decision Assumption [BGN05]). *The subgroup decision assumption holds with respect to a composite-order bilinear group generator $\mathcal{G}$ if for every polynomial size adversary $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for every $\lambda \in \mathbb{N}$,*

$$\left| \Pr[\mathcal{A}((\mathbb{G}, \mathbb{G}_T, N, g_p, e), g^r) = 1] - \Pr[\mathcal{A}((\mathbb{G}, \mathbb{G}_T, N, g_p, e), g_p^r) = 1] \right| \leq \text{negl}(\lambda)$$

where $(p, q, \mathbb{G}, \mathbb{G}_T, e, g) \leftarrow \mathcal{G}(1^\lambda)$, $N = pq$, $g_p = g^q$, and $r \leftarrow \mathbb{Z}_N$.

We present the zero-knowledge variant of the Waters-Wu BARG in **Figure 3**. Ensuring zero-knowledge only requires modifying the prover's algorithm and retains the same setup and verification algorithms as in [WW22]. We first show that the construction in **Figure 3** satisfies perfect zero-knowledge while retaining the other properties of the Waters-Wu BARG. We then continue to show that it is a Splittable zkSNARG.

Theorem 10. *Assuming the subgroup decision assumption holds with respect to composite-order bilinear groups, the Waters-Wu zkBARG (**Figure 3**) is a batch argument for boolean circuit satisfiability with non-adaptive soundness and perfect zero-knowledge.*

¹⁶This requires that the HSS scheme supports computing over $\mathbb{Z}_p$, which is true for the schemes we consider.

Waters-Wu zkBARG

Public parameters A group generator $\mathcal{G}$ for composite-order bilinear groups.

zkBARG.Setup($1^\lambda, n$)

- 1: Run $(p, q, \mathbb{G}, \mathbb{G}_T, e, g) \leftarrow \mathcal{G}(1^\lambda)$ and let $N = pq$, and $g_p = g^q$.
- 2: Sample $\alpha_i \leftarrow \mathbb{Z}_N$ uniformly at random for each $i \in [1, n]$.
- 3: Let $A_i = g_p^{\alpha_i}$ for each $i \in [1, n]$ and let $A = \prod_{i \in [1, n]} A_i = g_p^{\sum_{i \in [1, n]} \alpha_i}$.
- 4: Let $B_{i,j} = g_p^{\alpha_i \alpha_j}$ for every $i, j \in [1, n]$ such that $i \neq j$.
- 5: Output $\text{crs} = \left((\mathbb{G}, \mathbb{G}_T, N, g_p, e), A, \{A_i\}_{i=1}^n, \{B_{i,j}\}_{i \neq j \in [1, n]} \right)$.

zkBARG.Prove($\text{crs}, C, (\mathbf{x}_1, \dots, \mathbf{x}_n), (\mathbf{w}_1, \dots, \mathbf{w}_n)$)

Let L be the number of wires in C and let $\boldsymbol{\mu}_i = (\mu_{i,1}, \dots, \mu_{i,L})$ be the i -th extended witness where $\mu_{i,j}$ is the value on the j -th wire in $C(\mathbf{x}_i, \mathbf{w}_i)$ for every $i \in [1, n]$ and $j \in [1, L]$. For every $j \in [1, \ell_{\text{stmt}}]$ we have $\mu_{i,j} = x_j$.

- 1: **Commitment To Wire Values:** For each $k \in [1, L]$, **sample $r_k \leftarrow \mathbb{Z}_N$ uniformly at random if $k > \ell_{\text{stmt}}$ and set $r_k := 0$ if $k \leq \ell_{\text{stmt}}$ and compute**

$$U_k = g_p^{r_k} \cdot \prod_{i \in [1, n]} A_i^{\mu_{i,k}} = g_p^{r_k + \sum_{i \in [1, n]} \alpha_i \mu_{i,k}}.$$

- 2: **Validity Of Wire Assignments:** For each $k \in [1, L]$ compute

$$V_k = \frac{\prod_{i \neq j} B_{i,j}^{(1-\mu_{i,k})\mu_{j,k}} \cdot \prod_{i \in [1, n]} A_i^{r_k(1-2\mu_{i,k})}}{g_p^{r_k^2}}.$$

- 3: **Validity Of Gate Computations:** For every $k \in [1, s]$, where the k -th NAND gate has input wires (a, b) and output wire c , compute

$$W_k = \frac{\prod_{i \neq j} B_{i,j}^{\mu_{i,a} \mu_{j,b} + \mu_{j,c} - 1} \cdot A^{r_c} \cdot U_b^{r_a} \cdot U_a^{r_b}}{g_p^{r_a r_b}}$$

- 4: Output the proof $\boldsymbol{\pi} = \left(\{U_k\}_{k=1}^L, \{V_k\}_{k=1}^L, \{W_k\}_{k=1}^s \right)$.

Figure 3: Setup and prover algorithms for the zero-knowledge variant of the Waters-Wu BARG [WW22]. The verification algorithm is described in Figure 4. Changes to the prover algorithm from [WW22], required to achieve zero-knowledge, are highlighted in red.

Waters-Wu zkBARG (continued)

zkBARG.Verify(crs, C, $(\mathbf{x}_1, \dots, \mathbf{x}_n)$, π)

1: **Verification Key:** For each $k \in [1, \ell_{\text{stmt}}]$ compute

$$U_k^* = \prod_{i \in [1, n]} A_i^{x_{i,k}} = g_p^{\sum_{i \in [1, n]} \alpha_i x_{i,k}}.$$

2: **Statement Consistency Check:** For each $k \in [1, \ell_{\text{stmt}}]$, check if

$$U_k = U_k^*.$$

3: **Wire Validity Checks:** For each $k \in [\ell_{\text{stmt}} + 1, L]$, check if

$$e(A, U_k) = e(g_p, V_k) \cdot e(U_k, U_k).$$

4: **Gate Consistency Checks:** For each $k \in [1, s]$, where the k -th NAND gate has input wires (a, b) and output wire c , check if

$$e(A, A) \cdot e(g_p, W_k) = e(U_a, U_b) \cdot e(A, U_c).$$

5: Output 1 if all checks pass and output 0 otherwise.

Figure 4: Verification algorithm for the zero-knowledge variant of the Waters-Wu BARG [WW22]. The setup and prover algorithms are described in Figure 3.

Proof. It is easy to see from inspection that modifications to achieve zero-knowledge do not affect completeness. Moreover, observe that the Setup and Verify algorithms in Figure 3 remain identical to those in the Waters-Wu BARG [WW22]. It follows that Figure 3, like the Waters-Wu BARG, is non-adaptively sound.

We are left to argue that the scheme has perfect zero-knowledge. We first describe the simulator and then argue that a simulated proof is identically distributed to a proof computed using the zkBARG. The simulator works as follows.

1. It runs the setup algorithm and outputs crs and $\text{st} = (g_p, N, \{\alpha_i\}_{i=1}^n)$.
2. When invoked with input $(\mathbf{x}_1, \dots, \mathbf{x}_n)$ and st , it does the following.
 - *Commitments To Wire Values:* For every $k \in [1, \ell_{\text{stmt}}]$, it sets $z_k = \sum_{i \in [1, n]} \alpha_i x_{i,k}$. For the remaining wires $k \in [\ell_{\text{stmt}} + 1, L]$, it samples $z_k \leftarrow \mathbb{Z}_N$ uniformly at random. For all $k \in [1, L]$, it computes $U_k := g_p^{z_k}$.
 - *Validity Of Wire Assignments:* For each $k \in [1, L]$, it sets $V_k = g_p^{y_k}$ where

$$y_k = \left(z_k \sum_{i \in [1, n]} \alpha_i \right) - z_k^2$$

- *Gate Consistency Check:* For each $k \in [1, s]$, where the k -th NAND gate has input wires (a, b) and

output wire c , it sets $W_k = g_p^{p_k}$, where

$$p_k = z_a z_b + \left(z_c \sum_{i \in [1, n]} \alpha_i \right) - \sum_{i \in [1, n]} \alpha_i^2 - \sum_{i \neq j} \alpha_i \alpha_j.$$

- It outputs $\pi = \left(\{U_k\}_{k=1}^L, \{V_k\}_{k=1}^L, \{W_k\}_{k=1}^s \right)$.

Next, we argue that the joint distribution of the CRS and the proof output by the simulator is identical to that computed using the zkBARG. Observe that the distribution of the CRS is the same in both cases since the simulator simply runs Setup. Moreover, the proofs are identically distributed conditioned on the CRS. Specifically, for each input wire $k \in [1, \ell_{\text{stmt}}]$, U_k is identical in the real and simulated proofs. For each $k \in [\ell_{\text{stmt}} + 1, L]$, U_k computed in Prove is uniformly random in $\mathbb{G}_p$ since r_k is sampled uniformly at random from $\mathbb{Z}_N$. Similarly, U_k output by the simulator is also uniformly random in $\mathbb{G}_p$ and thus it follows that $\{U_k\}_{k=1}^L$ is identically distributed in the real and simulated proofs. Now, observe that each V_k and W_k are uniquely determined in $\mathbb{G}_p$ by the verification checks $e(A, U_k) = e(g_p, V_k) \cdot e(U_k, U_k)$ and $e(A, A) \cdot e(g_p, W_k) = e(U_a, U_b) \cdot e(A, U_c)$ respectively. Since the proof output by both Prove and the simulator are in $\mathbb{G}_p$, and the simulated proof passes the verification checks, it follows that the simulated proof is identically distributed to the real proof. Thus, the zkBARG has perfect zero-knowledge. ■

Remark 4 (Somewhere Argument Of Knowledge). *As observed in the proof of Theorem 10, ensuring zero-knowledge only requires changing the prover’s algorithm. This means that the zkBARG achieves any notion of soundness achieved by the Waters-Wu BARG. Specifically, this means that the zkBARG is a somewhere argument of knowledge [CJJ22].*

Theorem 11. *Assuming the subgroup decision assumption holds with respect to composite-order bilinear groups, the Waters-Wu zkBARG (Figure 3) is a Splittable zkSNARG (Definition 5), with non-adaptive soundness, for SIMD boolean circuit satisfiability. Moreover, for every polynomial size boolean circuit C of depth D and all positive integers n , there exists an efficiently constructible circuit C_{NL} over $\mathbb{Z}_N$ of size $|C_{\text{NL}}| = O(|C|n)$ and with multiplicative depth $O(D)$ such that for all crs in the support of Setup, $(C, \mathbf{x}_1, \dots, \mathbf{x}_n; \mathbf{w}_1, \dots, \mathbf{w}_n) \in \text{BSAT}_n$ and $\mathbf{r} \in \mathbb{Z}_N^L$, we have $C_{\text{NL}}(\mathbf{x}_1, \dots, \mathbf{x}_n, \mathbf{w}_1, \dots, \mathbf{w}_n, \mathbf{r}) = \text{Prove}_{\text{NL}}(\text{crs}, C, \mathbf{x}_1, \dots, \mathbf{x}_n, \mathbf{w}_1, \dots, \mathbf{w}_n; \mathbf{r})$, where L is the number of wires in C , and N is the order of the bilinear groups specified by crs.*

Proof. We first show that the zkBARG is split prover and then argue that it has robust verification.

Claim 11.1. *Waters-Wu zkBARG is split prover such that for every polynomial size boolean circuit C there exists an efficiently constructible circuit C_{NL} as described above.*

Proof. Consider the following split of the prover into a non-linear and linear phase.

- $\text{zkBARG.Prove}_{\text{NL}}$: It takes the statements $(\mathbf{x}_1, \dots, \mathbf{x}_n)$, witnesses $(\mathbf{w}_1, \dots, \mathbf{w}_n)$ and uniform random values $r_1, \dots, r_L \leftarrow \mathbb{Z}_N$ as input and outputs the following.
 1. The extended witness $\{\mu_{i,j}\}_{i,j}$, where $\mu_{i,j}$ is the value on the j -th wire of $C(\mathbf{x}_i, \mathbf{w}_i)$, for each $i \in [1, n]$ and $j \in [1, L]$.
 2. The pairwise product of the values in the extended witness, $\mu_{i,j,k_1,k_2} := \mu_{i,k_1} \cdot \mu_{j,k_2}$ for every $i, j \in [1, n]$ such that $i \neq j$ and every $k_1, k_2 \in [1, L]$.
 3. The random inputs $\{r_i\}_i$ and the pairwise product of the random inputs, $r_{i,j} := r_i \cdot r_j$ for every $i, j \in [1, L]$.

4. The product of each random input with the extended witness, $\rho_{i,k_1,k_2} := \mu_{i,k_1} \cdot r_{k_2}$ for every $i \in [1, n]$ and $k_1, k_2 \in [1, L]$.
- zkBARG.Prove_L: It takes the output of the non-linear phase (highlighted in red below) as input and proceeds as follows.

1. For each $k \in [1, L]$, it computes the commitment to wire values as

$$U_k = g_p^{r_k} \cdot \prod_{i \in [1, n]} A_i^{\mu_{i,k}}.$$

2. For each $k \in [1, L]$, it computes

$$V_k = \prod_{i \neq j} B_{i,j}^{\mu_{i,j,k}} \cdot \prod_{i \neq j} B_{i,j}^{\mu_{i,j,k,k}} \cdot \prod_{i \in [1, n]} A_i^{r_k} \cdot \prod_{i \in [1, n]} A_i^{2\rho_{i,k,k}} \cdot g_p^{-r_{k,k}}.$$

3. For each $k \in [1, s]$, where the k -th NAND gate has input wires (a, b) and output wire c , it computes

$$W_k = \prod_{i \neq j} B_{i,j}^{\mu_{i,j,a,b}} \cdot \prod_{i \neq j} B_{i,j}^{\mu_{i,j,c}} \cdot B_{i,j}^{-1} \cdot g_p^{-r_{a,b}} \cdot A_i^{r_c} \cdot \prod_{i \in [1, n]} A_i^{\rho_{i,a,b}} \cdot \prod_{i \in [1, n]} A_i^{\rho_{i,b,a}}.$$

It is easy to see that zkBARG.Prove_L is a homomorphism from tuples in $\mathbb{Z}_N$ to tuples in $\mathbb{G}$. The split prover property then follows from the fact that zkBARG.Prove_{NL} and zkBARG.Prove_L constitute a partition of the steps of zkBARG.Prove.

Given any polynomial size circuit C of depth D , we next show that there exists a circuit C_{NL} over $\mathbb{Z}_N$ of multiplicative depth $O(D)$ that computes the output of the non-linear phase when proving batch satisfiability of C . Observe that $(\mu_{i,1}, \dots, \mu_{i,L})$ can be computed in parallel for each $i \in [1, n]$ by a circuit over $\mathbb{Z}_N$ with multiplicative depth $O(D)$, since any two-input boolean gate can be computed using a constant number of $\mathbb{Z}_N$ operations. Given $\{\mu_{i,j}\}_{i,j}$, each μ_{i,j,k_1,k_2} can be computed in parallel by a constant depth circuit over $\mathbb{Z}_N$. Similarly, each r_i and ρ_{i,k_1,k_2} can be computed in parallel by a constant depth circuit over $\mathbb{Z}_N$ given $\{\mu_{i,j}\}_{i,j}$. Putting everything together, it follows that C_{NL} is a circuit over $\mathbb{Z}_N$ with multiplicative depth $O(D)$. It is also easy to see that C_{NL} is efficiently constructible given C . $\square$

Claim 11.2. *The Waters-Wu zkBARG has robust verification assuming the subgroup decision assumption holds with respect to the composite-order bilinear groups.*

Proof. Let $\mathcal{A}$ be an adversary against the robust verification of the Waters-Wu zkBARG that outputs a statement $(\mathbf{x}_1, \dots, \mathbf{x}_n)$, witness $(\mathbf{w}_1, \dots, \mathbf{w}_n)$ and error

$$\epsilon = \left(\{g^{\epsilon_{u_k}}\}_{k=1}^L, \{g^{\epsilon_{v_k}}\}_{k=1}^L, \{g^{\epsilon_{w_k}}\}_{k=1}^s \right)$$

where all ϵ_{u_k} , ϵ_{v_k} and ϵ_{w_k} are in $\mathbb{Z}_N$. Let the proof $\pi = \left(\{U_k\}_{k=1}^L, \{V_k\}_{k=1}^L, \{W_k\}_{k=1}^s \right)$ which means that

$$\bar{\pi} = \left(\{U_k \cdot g^{\epsilon_{u_k}}\}_{k=1}^L, \{V_k \cdot g^{\epsilon_{v_k}}\}_{k=1}^L, \{W_k \cdot g^{\epsilon_{w_k}}\}_{k=1}^s \right)$$

since we are using multiplicative notation for the group.

Recall from Section 4.3 that $\mathbb{G}$ is isomorphic to $\mathbb{G}_p \times \mathbb{G}_q$ and that the subgroups $\mathbb{G}_p$ and $\mathbb{G}_q$ are orthogonal with respect to the pairing e which implies that given the generator g_p of $\mathbb{G}_p$, we can test if any element g' is in $\mathbb{G}_q$ by checking that $e(g_p, g') = 1_{\mathbb{G}_T}$.

Informally, the verification checks for the zkBARG pass with overwhelming probability only when errors to each term of the proof are in the subgroup $\mathbb{G}_q$. However, the robust verification property requires that $\bar{\pi}$ not verify successfully when the error to any term of the proof is not $1_{\mathbb{G}}$. Thus, our proof proceeds in two steps. First, we show that it is computationally infeasible for the adversary to output an error ϵ where one of the terms is in $\mathbb{G}_q$ but not equal to $1_{\mathbb{G}}$. This means that any error term output by $\mathcal{A}$ that is not equal to $1_{\mathbb{G}}$ cannot be in $\mathbb{G}_q$. Next, we show that verification of $\bar{\pi}$ fails with overwhelming probability if there exists an error term that is not in $\mathbb{G}_q$ and not equal to $1_{\mathbb{G}}$. It then follows that the zkBARG has robust verification.

Let the output of the robust verification experiment be defined as 1 when $(\mathbf{w}_1, \dots, \mathbf{w}_n)$ is a valid witness for $(\mathbf{x}_1, \dots, \mathbf{x}_n)$, there exists a value in ϵ that is not equal to $1_{\mathbb{G}}$ and $\text{zkBARG.Verify}(\text{crs}, \mathbf{x}_1, \dots, \mathbf{x}_n, \bar{\pi}) = 1$; and be defined as 0 otherwise. We use a hybrid argument to show that the output of the experiment is 1 with at most negligible probability.

- Hyb_0 : This corresponds to the output of the experiment when run with $\mathcal{A}$.
- Hyb_1 : In this hybrid, the experiment outputs 0 if there exists $g^\eta \in \epsilon$ such that $e(g_p, g^\eta) = 1_{\mathbb{G}_T}$.
Observe that if every element in ϵ is $1_{\mathbb{G}}$ then both Hyb_0 and Hyb_1 are equal to 0. However, Hyb_0 might not be the same as Hyb_1 if there exists $g^\eta \in \mathbb{G}_q$ such that $g^\eta \neq 1_{\mathbb{G}}$. We show that Hyb_1 is indistinguishable from Hyb_0 by reduction to the subgroup decision assumption. Specifically, consider an adversary $\mathcal{A}'$ for the subgroup decision problem that upon receiving $((\mathbb{G}, \mathbb{G}_T, N, g_p, e), g')$ computes the crs for the zkBARG and runs $\mathcal{A}(1^\lambda, \text{crs})$ to obtain ϵ . If there exists g^η in ϵ such that $g^\eta \neq 1_{\mathbb{G}}$ and $e(g_p, g^\eta) = 1_{\mathbb{G}_T}$, $\mathcal{A}'$ outputs 1 if $e(g', g^\eta) = 1_{\mathbb{G}_T}$ and outputs 0 otherwise. Else, if there does not exist any such g^η in ϵ , $\mathcal{A}'$ outputs 1.

When $g^\eta \in \mathbb{G}_q$ and $g^\eta \neq 1_{\mathbb{G}}$, we have $e(g', g^\eta) = 1_{\mathbb{G}_T}$ iff $g' \in \mathbb{G}_p$. This implies that when $e(g_p, g^\eta) = 1_{\mathbb{G}_T}$ and $g^\eta \neq 1_{\mathbb{G}}$ then $\mathcal{A}'$ solves the subgroup decision problem. It then follows from the subgroup decision assumption that such a g^η can be output by $\mathcal{A}$ with at most negligible probability which in turn implies from our discussion above that $\text{Hyb}_0 \approx \text{Hyb}_1$.

- Hyb_2 : In this hybrid, the experiment always outputs 0.

We show that $\text{Hyb}_1 \approx \text{Hyb}_2$ by considering the following cases.

- **Case 1:** There exists $k \in [1, \ell_{\text{stmt}}]$ such that $g^{\epsilon_{u_k}} \neq 1_{\mathbb{G}}$.
The statement consistency check always fails during verification if $g^{\epsilon_{u_k}} \neq 1_{\mathbb{G}}$ and so $\bar{\pi}$ does not verify successfully and $\text{Hyb}_1 \equiv \text{Hyb}_2$ in this case.
- **Case 2:** $g^{\epsilon_{u_k}} = 1_{\mathbb{G}}$ for every $k \in [1, \ell_{\text{stmt}}]$ but there exists $k \in [\ell_{\text{stmt}} + 1, L]$ such that $g^{\epsilon_{u_k}} \neq 1_{\mathbb{G}}$ or there exists $k \in [1, L]$ such that $g^{\epsilon_{v_k}} \neq 1_{\mathbb{G}}$.

We show that $\text{Hyb}_1 \approx \text{Hyb}_2$ primarily because of the randomness used for zero-knowledge of the zkBARG. Note that if there exists $g^{\epsilon_{u_k}} \in \mathbb{G}_q$ or $g^{\epsilon_{v_k}} \in \mathbb{G}_q$ then both Hyb_1 and Hyb_2 output 0 in this case. Thus, we only consider the case when there exists $g^{\epsilon_{u_k}} \notin \mathbb{G}_q$ or $g^{\epsilon_{v_k}} \notin \mathbb{G}_q$.

From the completeness of the zkBARG, for every $k \in [1, L]$ we have

$$e(A, U_k) = e(g_p, V_k) \cdot e(U_k, U_k). \quad (3)$$

Now, consider the k -th wire consistency check for $\bar{\pi}$

$$\begin{aligned} e(A, \bar{U}_k) &= e(g_p, \bar{V}_k) \cdot e(\bar{U}_k, \bar{U}_k) \\ \implies e(A, U_k \cdot g^{\epsilon_{u_k}}) &= e(g_p, V_k \cdot g^{\epsilon_{v_k}}) \cdot e(U_k \cdot g^{\epsilon_{u_k}}, U_k \cdot g^{\epsilon_{u_k}}) \\ \implies e(A, g^{\epsilon_{u_k}}) &= e(g_p, g^{\epsilon_{v_k}}) \cdot e(U_k, g^{\epsilon_{u_k}})^2 \cdot e(g^{\epsilon_{u_k}}, g^{\epsilon_{u_k}}) \end{aligned}$$

where the first implication follows from the definition of $\bar{\pi}$ and the second implication follows from Equation (3). Observe that if $g^{\epsilon_{u_k}} = 1_{\mathbb{G}}$ and $g^{\epsilon_{v_k}} \notin \mathbb{G}_q$ then $e(g_p, g^{\epsilon_{v_k}}) \neq 1_{\mathbb{G}_T}$ and the k -th wire consistency check always fails for $\bar{\pi}$. Moreover since $g^{\epsilon_{u_k}} = 1_{\mathbb{G}}$ for all $k \in [1, \ell_{\text{stmt}}]$, if Hyb_1 is not equal to Hyb_2 then it must be the case that there exists $k \in [\ell_{\text{stmt}} + 1, L]$ such that $g^{\epsilon_{u_k}} \notin \mathbb{G}_q$.

Next, we show that if there exists $g^{\epsilon_{u_k}} \notin \mathbb{G}_q$ for some $k \in [\ell_{\text{stmt}} + 1, L]$ then $\bar{\pi}$ verifies with at most negligible probability. If the k -th wire consistency check passes then $e(A, g^{\epsilon_{u_k}}) = e(g_p, g^{\epsilon_{v_k}}) \cdot e(U_k, g^{\epsilon_{u_k}})^2 \cdot e(g^{\epsilon_{u_k}}, g^{\epsilon_{u_k}})$, which along with the fact that $g_p = g^q$ and $\mathbb{G}_T$ is a cyclic group of order N with generator $g_T = e(g, g)$ implies that

$$q\epsilon_{u_k} \sum_{i=1}^n \alpha_i - q\epsilon_{v_k} - 2qu_k\epsilon_{u_k} \equiv \epsilon_{u_k}^2 \pmod{N}$$

where $U_k = g_p^{u_k}$. Furthermore, from the Chinese Remainder Theorem, $\epsilon_{u_k}^2 \equiv 0 \pmod{q}$, which in turn implies that $\epsilon_{u_k}^2 = qv$ for some $v \in \mathbb{N}$. Consequently,

$$\begin{aligned} & q\epsilon_{u_k} \sum_{i=1}^n \alpha_i - q\epsilon_{v_k} - 2qu_k\epsilon_{u_k} - qv \equiv 0 \pmod{N} \\ \implies & q \cdot \left(\epsilon_{u_k} \sum_{i=1}^n \alpha_i - \epsilon_{v_k} - 2u_k\epsilon_{u_k} - v \right) \equiv 0 \pmod{N} \\ \implies & \epsilon_{u_k} \sum_{i=1}^n \alpha_i - \epsilon_{v_k} - 2u_k\epsilon_{u_k} - v \equiv 0 \pmod{p}. \end{aligned} \quad (4)$$

Since the adversary's view is independent of r_k and r_k is sampled uniformly at random from $\mathbb{Z}_N$ for every $k \in [\ell_{\text{stmt}} + 1, L]$, u_k is uniformly random over $\mathbb{Z}_N$ and independent of the error ϵ . However, since $g^{\epsilon_{u_k}} \notin \mathbb{G}_q$, we have $\epsilon_{u_k} \not\equiv 0 \pmod{p}$ and there exists at most one root for $\epsilon_{u_k} \sum_{i=1}^n \alpha_i - \epsilon_{v_k} - 2u_k\epsilon_{u_k} - v$ when viewed as a polynomial in u_k over $\mathbb{Z}_p$. From the Chinese Remainder Theorem there are q roots of this polynomial in $\mathbb{Z}_N$ which implies that Equation (4) holds with probability at most $q/N = 1/p \in \text{negl}(\lambda)$. Therefore, Equation (4) holds with at most negligible probability when $g^{\epsilon_{u_k}} \notin \mathbb{G}_q$ which in turn implies that $\text{zkBARG.Verify}(\text{crs}, \mathbf{x}_1, \dots, \mathbf{x}_n, \bar{\pi}) = 0$ with overwhelming probability. Thus, it follows that when there exists $g^{\epsilon_{u_k}} \neq 1_{\mathbb{G}}$ or $g^{\epsilon_{v_k}} \neq 1_{\mathbb{G}}$ then $\text{Hyb}_1 \stackrel{s}{\approx} \text{Hyb}_2$.

- **Case 3:** $g^{\epsilon_{u_k}} = 1_{\mathbb{G}}$ for all $k \in [1, L]$, $g^{\epsilon_{v_k}} = 1_{\mathbb{G}}$ for all $k \in [1, L]$ but there exists $k \in [1, s]$ such that $g^{\epsilon_{w_k}} \neq 1_{\mathbb{G}}$.

If there exists $g^{\epsilon_{w_k}} \in \mathbb{G}_q$ then both Hyb_1 and Hyb_2 output 0. Thus, we only consider the case when there exists $g^{\epsilon_{w_k}} \notin \mathbb{G}_q$ which implies that $e(g_p, g^{\epsilon_{w_k}}) \neq 1_{\mathbb{G}_T}$. From the completeness of the zkBARG, the gate consistency checks for valid proofs always pass which implies that for the k -th NAND gate with input wires (a, b) and output wire c we have

$$e(A, A) \cdot e(g_p, W_k) = e(U_a, U_b) \cdot e(A, U_c). \quad (5)$$

Now, consider the k -th gate consistency check for $\bar{\pi}$

$$\begin{aligned} & e(A, A) \cdot e(g_p, \bar{W}_k) = e(\bar{U}_a, \bar{U}_b) \cdot e(A, \bar{U}_c) \\ \implies & e(A, A) \cdot e(g_p, W_k \cdot g^{\epsilon_{w_k}}) = e(U_a \cdot g^{\epsilon_{u_a}}, U_b \cdot g^{\epsilon_{u_b}}) \cdot e(A, U_c \cdot g^{\epsilon_{u_c}}) \\ \implies & e(g_p, g^{\epsilon_{w_k}}) = e(U_a, g^{\epsilon_{u_a}}) \cdot e(g^{\epsilon_{u_a}}, U_b) \cdot e(g^{\epsilon_{u_a}}, g^{\epsilon_{u_b}}) \cdot e(A, g^{\epsilon_{u_c}}) \end{aligned}$$

where the first implication follows from the definition of $\bar{\pi}$ and the second implication follow from Equation (5). However, since $g^{\epsilon_{u_k}} = 1_{\mathbb{G}}$ for every $k \in [1, L]$ in this case, it follows that

the k -th gate consistency check passes iff $e(g_p, g^{\epsilon_{w_k}}) = 1_{\mathbb{G}_T}$. Since there exists a $k \in [1, s]$ such that $e(g_p, g^{\epsilon_{w_k}}) \neq 1_{\mathbb{G}_T}$, it follows that at least one gate consistency check does not pass and that $\text{zkBARG.Verify}(\text{crs}, \mathbf{x}_1, \dots, \mathbf{x}_n, \overline{\pi}) = 0$ and $\text{Hyb}_1 \stackrel{s}{\approx} \text{Hyb}_2$ in this case.

It follows from our argument above that $\text{Hyb}_1 \stackrel{s}{\approx} \text{Hyb}_2$.

Thus, $\text{Hyb}_0 \stackrel{s}{\approx} \text{Hyb}_2$ from our sequence of hybrids and since Hyb_2 is always 0, it follows that the Waters-Wu zkBARG has robust verification. $\square$

Observe that the zkBARG is succinct since the proof size is $O(\lambda(n + \ell_{\text{wit}}))$ and the verifier runtime is $O(\lambda(n + \ell_{\text{wit}} + n\ell_{\text{stmt}}))$ while the length of the witness $(\mathbf{w}_1, \dots, \mathbf{w}_n)$ is $O(n\ell_{\text{wit}})$. Similarly, the zkBARG has succinct randomness since for a SIMD circuit of size ns it requires $O(s)$ uniformly random values from $\mathbb{Z}_N$ where each value is $\log \lambda$ bits long. It then follows from [Theorem 10](#) and [Claims 11.1](#) and [11.2](#) that the Waters-Wu zkBARG is a Splittable zkSNARG for SIMD boolean circuit satisfiability with non-adaptive soundness. $\blacksquare$

5 HSS with Verifiable Evaluation

In this section, we define HSS with verifiable evaluation and then construct it using HSS and Splittable zkSNARGs.

We define ve-HSS in [Definition 7](#). As discussed in [Section 1](#), a ve-HSS scheme allows a client to secret share a private input $\mathbf{x}$ among m servers. Given a public input $\mathbf{x}_{\text{pub}}$, the servers can then locally compute on their shares to obtain shares of the output, which when reconstructed yields the circuit output $C(\mathbf{x}_{\text{pub}}, \mathbf{x})$. However, since servers may be corrupt, they might send incorrect output shares. To address this, the scheme supports verification of the reconstructed candidate output $\mathbf{y}$. Specifically, we require two forms of verification: (1) *Local verification*, where the client, which knows the full input $(\mathbf{x}_{\text{pub}}, \mathbf{x})$ can verify that $\mathbf{y} = C(\mathbf{x}_{\text{pub}}, \mathbf{x})$, and (2) *Public verification*, where any external entity can verify that the output is consistent with the public input i.e., there exists private input $\mathbf{x}$ such that $\mathbf{y} = C(\mathbf{x}_{\text{pub}}, \mathbf{x})$. We use π_{loc} and π_{pub} to denote the auxiliary output of reconstruction which is used for local and public verification, respectively.

Correctness requires that reconstruction indeed yields the circuit output when all servers are honest. Privacy guarantees that as long as at least one server is honest, the input shares of corrupt servers and the output of public verification preserve the privacy of the private input. Local (public) soundness ensures that local (public) verification fails if the candidate output is incorrect (inconsistent with the public input), even if all servers are corrupt. To ensure that the evaluation algorithm is non-trivial, we require that reconstruction is linear and that the runtime of the sharing algorithm is sublinear in the size of the circuit. To ensure that verification is succinct, we require that the output share size is sublinear in the size of the circuit and that the runtime of the local and public verification algorithms are sublinear in the size of the circuit and independent of the number of servers.

Definition 7 (HSS with verifiable evaluation). *Let $m := m(\lambda)$ be an integer-valued polynomial, R be a ring and let $\mathbb{C}$ be a class of circuits over R . An m -server HSS with verifiable evaluation (ve-HSS) scheme for $\mathbb{C}$ is a tuple of algorithms $\text{ve-HSS} = (\text{Setup}, \text{Share}, \text{Eval}, \text{Recon}, \text{LVerify}, \text{PVerify})$ with the following syntax.*

- $\text{Setup}(1^\lambda, C) \rightarrow \text{crs}$ is a PPT algorithm that takes the description of a circuit $C \in \mathbb{C}$ as input and outputs the common reference string crs .
- $\text{Share}(\text{crs}, \mathbf{x}) \rightarrow (\langle \mathbf{x} \rangle_1, \dots, \langle \mathbf{x} \rangle_m)$ is a PPT algorithm that takes as input the common reference string crs and private input $\mathbf{x} \in R^*$ and outputs a sharing of the inputs $\langle \mathbf{x} \rangle = (\langle \mathbf{x} \rangle_1, \dots, \langle \mathbf{x} \rangle_m)$.

ve-HSS Privacy Experiment	
$\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^0(1^\lambda):$	$\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^1(1^\lambda):$
1: Compute $\text{crs} \leftarrow \text{Setup}(1^\lambda, C)$ and send crs to $\mathcal{A}$.	1: Compute $(\text{crs}, \text{st}) \leftarrow \text{Sim}(1^\lambda, C)$ and send crs to $\mathcal{A}$.
2: Receive $(I, \mathbf{x}_{\text{pub}}, \mathbf{x})$ from $\mathcal{A}$. Output 0 if $I \not\subseteq [1, m]$ or if $ I = m$.	2: Receive $(I, \mathbf{x}_{\text{pub}}, \mathbf{x})$ from $\mathcal{A}$. Output 0 if $I \not\subseteq [1, m]$ or if $ I = m$.
3: Compute	3: Compute
$(\langle \mathbf{x} \rangle_1, \dots, \langle \mathbf{x} \rangle_m) \leftarrow \text{Share}(\text{crs}, \mathbf{x})$	$(\langle \mathbf{x} \rangle_I, \text{st}) \leftarrow \text{Sim}(\text{st}, I, \mathbf{x})$
and send $\langle \mathbf{x} \rangle_I$ to $\mathcal{A}$.	and send $\langle \mathbf{x} \rangle_I$ to $\mathcal{A}$.
4: Receive $[\overline{\text{out}}]_I$ from $\mathcal{A}$. For each $i \in [1, m] \setminus I$ compute	4: Receive $[\overline{\text{out}}]_I$ from $\mathcal{A}$. Compute
$[\text{out}]_i := \text{Eval}(\text{crs}, i, \mathbf{x}_{\text{pub}}, \langle \mathbf{x} \rangle_i),$	$(b, \overline{\pi_{\text{pub}}}) \leftarrow$
and then compute	$\text{Sim}\left(\text{st}, \mathbf{x}_{\text{pub}}, C(\mathbf{x}_{\text{pub}}, \mathbf{x}), [\overline{\text{out}}]_I\right).$
$(\overline{\mathbf{y}}, \overline{\pi_{\text{loc}}}, \overline{\pi_{\text{pub}}}) :=$	5: Send $(b, \overline{\pi_{\text{pub}}})$ to $\mathcal{A}$ and output b' returned by $\mathcal{A}$.
$\text{Recon}(\text{crs}, [\overline{\text{out}}]_I, [\text{out}]_{[1, m] \setminus I}),$	
$b \leftarrow \text{LVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{x}, \overline{\mathbf{y}}, \overline{\pi_{\text{loc}}})$	
5: Send $(b, \overline{\pi_{\text{pub}}})$ to $\mathcal{A}$ and output b' returned by $\mathcal{A}$.	

Figure 5: ve-HSS privacy experiment for [Definition 7](#).

- $\text{Eval}(\text{crs}, i, \mathbf{x}_{\text{pub}}, \langle \mathbf{x} \rangle_i) =: [\text{out}]_i$ is a polynomial time algorithm that takes as input the common reference string crs , a server index i , the public input $\mathbf{x}_{\text{pub}} \in R^*$ and the share of the i -th server. It outputs a share $[\text{out}]_i$.
- $\text{Recon}(\text{crs}, [\text{out}]_1, \dots, [\text{out}]_m) =: (\mathbf{y}, \pi_{\text{loc}}, \pi_{\text{pub}})$ is a polynomial time algorithm that takes as input the common reference string crs and the shares of the output, and reconstructs it to compute the circuit output $\mathbf{y} \in R$ and proofs π_{loc} and π_{pub} .
- $\text{LVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{x}, \mathbf{y}, \pi_{\text{loc}}) \rightarrow b$ is a PPT algorithm that takes the common reference string crs , the circuit inputs $\mathbf{x}_{\text{pub}}$ and $\mathbf{x}$, the circuit output $\mathbf{y}$ and the local proof π_{loc} , and outputs a bit b indicating if the proof is accepted or rejected.
- $\text{PVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{y}, \pi_{\text{pub}}) \rightarrow b$ is a PPT algorithm that takes the common reference string crs , the public inputs $\mathbf{x}_{\text{pub}}$, the circuit output $\mathbf{y}$ and the public proof π_{pub} , and outputs a bit b indicating if the proof is accepted or rejected.

A ve-HSS scheme should satisfy the following properties.

- **Efficiency:** The ve-HSS scheme is said to be efficient if it has linear reconstruction, succinct shares and succinct verification as defined below.

- **Linear Reconstruction:** Recon is linear in $[\text{out}]_1, \dots, [\text{out}]_m$.
- **Succinct Shares:** There exists a polynomial p and a sublinear function $g(x) \in o(x)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$ with ℓ_{out} outputs, the runtime of Share when used for sharing an input of length ℓ_{inp} is at most $p(m\lambda\ell_{\text{inp}}) \cdot g(|C|)$ and the output length of Eval is at most $p(\lambda\ell_{\text{out}}) \cdot g(|C|)$.
- **Succinct Verification:** There exists a polynomial p and a sublinear function $g(x) \in o(x)$ such that for every $\lambda, \ell_{\text{inp}} \in \mathbb{N}$ and circuits $C \in \mathbb{C}$ with $(\ell_{\text{inp}} + \ell_{\text{pub}})$ inputs and ℓ_{out} outputs, the runtime of LVerify is at most $p(\lambda(\ell_{\text{inp}} + \ell_{\text{pub}} + \ell_{\text{out}})) \cdot g(|C|)$ and the runtime of PVerify is at most $p(\lambda(\ell_{\text{pub}} + \ell_{\text{out}})) \cdot g(|C|)$. Note that the verification time is independent of the number of servers m .
- **Correctness:** For all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$ we have

$$\Pr \left[\begin{array}{l} \mathbf{y} \neq C(\mathbf{x}_{\text{pub}}, \mathbf{x}) \vee \\ \text{LVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{x}, \mathbf{y}, \pi_{\text{loc}}) = 0 \vee \\ \text{PVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{y}, \pi_{\text{pub}}) = 0 \end{array} \middle| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (\mathbf{x}_{\text{pub}}, \mathbf{x}) \leftarrow \mathcal{A}(\text{crs}) \\ (\langle \mathbf{x} \rangle_1, \dots, \langle \mathbf{x} \rangle_m) \leftarrow \text{Share}(\text{crs}, \mathbf{x}) \\ [\text{out}]_i := \text{Eval}(\text{crs}, i, \mathbf{x}_{\text{pub}}, \langle \mathbf{x} \rangle_i), \forall i \in [1, m] \\ (\mathbf{y}, \pi_{\text{loc}}, \pi_{\text{pub}}) := \text{Recon}(\text{crs}, [\text{out}]_1, \dots, [\text{out}]_m) \end{array} \right] \leq \text{negl}(\lambda).$$

- **Privacy:** There exists a PPT simulator Sim such that for all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$,

$$\left| \Pr \left[\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^0(1^\lambda) = 1 \right] - \Pr \left[\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^1(1^\lambda) = 1 \right] \right| \leq \text{negl}(\lambda)$$

where $\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^b(1^\lambda)$ is defined in [Figure 5](#).

- **Local Soundness:** For all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$ we have

$$\Pr \left[\begin{array}{l} \mathbf{y} \neq C(\mathbf{x}_{\text{pub}}, \mathbf{x}) \wedge \\ \text{LVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{x}, \mathbf{y}, \pi_{\text{loc}}) = 1 \end{array} \middle| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (\mathbf{x}_{\text{pub}}, \mathbf{x}, \text{st}) \leftarrow \mathcal{A}(\text{crs}) \\ (\langle \mathbf{x} \rangle_1, \dots, \langle \mathbf{x} \rangle_m) \leftarrow \text{Share}(\text{crs}, \mathbf{x}) \\ ([\text{out}]_1, \dots, [\text{out}]_m) \leftarrow \mathcal{A}(\text{st}, \langle \mathbf{x} \rangle_1, \dots, \langle \mathbf{x} \rangle_m) \\ (\mathbf{y}, \pi_{\text{loc}}, \pi_{\text{pub}}) := \text{Recon}(\text{crs}, [\text{out}]_1, \dots, [\text{out}]_m) \end{array} \right] \leq \text{negl}(\lambda).$$

- **Public Soundness:** For all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$ we have

$$\Pr \left[\begin{array}{l} (C, \mathbf{x}_{\text{pub}}, \mathbf{y}) \notin \text{SAT} \wedge \\ \text{PVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{y}, \pi_{\text{pub}}) = 1 \end{array} \middle| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (\mathbf{x}_{\text{pub}}, \mathbf{y}, \pi_{\text{pub}}) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \text{negl}(\lambda).$$

We use $\langle \mathbf{x} \rangle = (\langle \mathbf{x} \rangle_1, \dots, \langle \mathbf{x} \rangle_m)$ to denote the shares of all servers and use $\langle \mathbf{x} \rangle_I$ as a shorthand to denote $\{\langle \mathbf{x} \rangle_i\}_{i \in I}$ where $I \subseteq [1, m]$.

Remarks. We next discuss some natural variants of our definition of ve-HSS that may be of interest.

- **Weak Privacy:** [Definition 7](#) ensures a strong notion of privacy, which requires simulating the output of LVerify and the public proof π_{pub} , in addition to the shares of the inputs for corrupt parties. One could consider a weaker yet meaningful notion of privacy which only requires simulating the shares of inputs for corrupt parties, similar to the security of HSS ([Definition 1](#)).

- **Public Knowledge Soundness:** The public soundness property in [Definition 7](#) can be strengthened by requiring knowledge soundness. Specifically, knowledge soundness requires that for all polynomial size adversaries $\mathcal{A}$ there exists a polynomial size extractor $\text{Ext}_{\mathcal{A}}$ and a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$,

$$\Pr \left[\begin{array}{c} y \neq C(\mathbf{x}_{\text{pub}}, \mathbf{x}) \\ \wedge \\ \text{PVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, y, \pi_{\text{pub}}) = 1 \end{array} \middle| \begin{array}{c} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (\mathbf{x}_{\text{pub}}, y, \pi_{\text{pub}}) \leftarrow \mathcal{A}(\text{crs}) \\ \mathbf{x} \leftarrow \text{Ext}_{\mathcal{A}}(\text{crs}, \mathbf{x}_{\text{pub}}, y, \pi_{\text{pub}}) \end{array} \right] \leq \text{negl}(\lambda).$$

- **Non-adaptive Soundness And Privacy:** It is also meaningful to consider weaker, non-adaptive variants of the soundness and privacy definitions where the adversary first chooses the inputs and then receives the CRS. Specifically, a ve-HSS scheme is said to have non-adaptive public soundness if for all polynomial size adversaries there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$ we have

$$\Pr \left[\begin{array}{c} (C, \mathbf{x}_{\text{pub}}, y) \notin \text{SAT} \\ \wedge \\ \text{PVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, y, \pi_{\text{pub}}) = 1 \end{array} \middle| \begin{array}{c} (\mathbf{x}_{\text{pub}}, y, \text{st}) \leftarrow \mathcal{A}(1^\lambda) \\ \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ \pi_{\text{pub}} \leftarrow \mathcal{A}(\text{crs}, \text{st}) \end{array} \right] \leq \text{negl}(\lambda).$$

A similar definition follows for non-adaptive local soundness. The non-adaptive privacy experiment proceeds similarly to [Figure 5](#) except that step 2 is run before step 1.

Construction. We present our construction of ve-HSS from HSS and Splittable in [Figure 6](#). We refer the reader to [Section 2.1](#) for an overview of the construction.

Theorem 12. *Let HSS be an m -server HSS scheme for a class of circuits $\mathbb{C}_{\text{hss}}$ over a finite commutative ring R and let SNARG be a $\mathbb{C}_{\text{hss}}$ -simple Splittable zkSNARG for $\mathbb{C}$ -circuit satisfiability such that $\mathbb{C} \subseteq \mathbb{C}_{\text{hss}}$. Then, [Figure 6](#) is an m -server ve-HSS scheme for $\mathbb{C}$. Moreover, it is black-box in HSS and the linear phase of SNARG.*

Proof. We argue that [Figure 6](#) is a ve-HSS scheme by showing that it satisfies the required correctness, privacy, soundness and efficiency properties.

Correctness. For any $C \in \mathbb{C}$, the correctness of HSS ensures that $y = C(\mathbf{x}_{\text{pub}}, \mathbf{x})$ with overwhelming probability. Additionally, the fact that SNARG is $\mathbb{C}_{\text{hss}}$ -simple implies that $[M_{\text{loc}}]$ is an additive sharing of $M_{\text{loc}} = \text{SNARG.Prove}_{\text{NL}}(C_{\text{loc}}, \mathbf{x}_{\text{pub}}, \mathbf{x}, y; r_{\text{loc}})$ and $[M_{\text{pub}}]$ is an additive sharing of $M_{\text{pub}} = \text{SNARG.Prove}_{\text{L}}(C_{\text{pub}}, \mathbf{x}_{\text{pub}}, \mathbf{x}, y; r_{\text{pub}})$ with overwhelming probability. Finally, DP-completeness ([Lemma 6](#)) and the fact that SNARG is for $\mathbb{C}$ -circuit satisfiability ensure that the reconstructed proofs π_{loc} and π_{pub} verify successfully with overwhelming probability.

Privacy. Intuitively, the security of HSS and zero-knowledge property of the publicly verifiable proof π_{pub} ensure the privacy of the input. Moreover, an adversary does not gain any advantage by deviating from the protocol and learning if local verification succeeded or failed due to DP-robust verification property of Splittable zkSNARGs. We formally show that the construction is private by constructing a simulator and arguing that any adversary $\mathcal{A}$ has only negligible advantage in winning the privacy experiment. Without loss of generality, we consider an adversary that corrupts all but one server.

Simulator:

1. It receives the circuit C as input and computes crs_{loc} as in Setup and simulates crs_{pub} using the SNARG simulator to compute $\text{crs} = (\text{crs}_{\text{loc}}, \text{crs}_{\text{pub}}, C)$. It outputs crs .
2. It then receives as input the corruption set $\mathcal{I}$ and length of the input ℓ_{inp} , and computes $\langle \mathbf{x} \rangle$, $\langle r_{\text{loc}} \rangle$, and $\langle r_{\text{pub}} \rangle$ as HSS shares of the zero vector of appropriate length. It outputs $(\langle \mathbf{x} \rangle_{\mathcal{I}}, \langle r_{\text{loc}} \rangle_{\mathcal{I}}, \langle r_{\text{pub}} \rangle_{\mathcal{I}})$.

HSS with verifiable evaluation

Public parameters. Let HSS be an m -server HSS scheme that supports computation over a finite commutative ring R and let SNARG be a Splittable zkSNARG. Let ℓ_{rand} denote the number of random elements from R required by the zkSNARG prover.

Notation. For a circuit C , we use C_{loc} and C_{pub} to denote the circuit satisfiability instances for the local and public proof i.e., when $y = C(x_{\text{pub}}, x)$ then $(C_{\text{loc}}, x_{\text{pub}} \| x, y) \in \text{SAT}$ and $(C_{\text{pub}}, x_{\text{pub}}, y) \in \text{SAT}$. We use $C_{\text{NL}}^{\text{loc}}$ and $C_{\text{NL}}^{\text{pub}}$ to denote the circuits that compute the non-linear phase Prove_{NL} of the Splittable zkSNARG for C_{loc} and C_{pub} , respectively.

ve-HSS.Setup($1^\lambda, C$)

```

1 :  $\text{crs}_{\text{loc}} \leftarrow \text{SNARG.Setup}(1^\lambda, C_{\text{loc}})$ 
2 :  $\text{crs}_{\text{pub}} \leftarrow \text{SNARG.Setup}(1^\lambda, C_{\text{pub}})$ 
3 :  $\text{crs} := (\text{crs}_{\text{loc}}, \text{crs}_{\text{pub}}, C)$ 
4 : return crs

```

ve-HSS.Share(x)

```

1 :  $r_{\text{loc}}, r_{\text{pub}} \leftarrow R^{\ell_{\text{rand}}}$ 
2 :  $(\langle x \rangle, \langle r_{\text{loc}} \rangle, \langle r_{\text{pub}} \rangle) \leftarrow \text{HSS.Share}(1^\lambda, x, r_{\text{loc}}, r_{\text{pub}})$ 
3 : return  $(\langle x \rangle, \langle r_{\text{loc}} \rangle, \langle r_{\text{pub}} \rangle)$ 

```

ve-HSS.Eval($\text{crs}, i, x_{\text{pub}}, \langle x \rangle_i, \langle r_{\text{loc}} \rangle_i, \langle r_{\text{pub}} \rangle_i$)

```

1 : parse  $\text{crs} = (\text{crs}_{\text{loc}}, \text{crs}_{\text{pub}}, C)$ 
2 :  $[y]_i := \text{HSS.Eval}(i, C, x_{\text{pub}}, \langle x \rangle_i)$ 
3 :  $[M_{\text{loc}}]_i := \text{HSS.Eval}(i, C_{\text{NL}}^{\text{loc}}, x_{\text{pub}}, \langle x \rangle_i, \langle r_{\text{loc}} \rangle_i)$ 
4 :  $[\pi_{\text{loc}}]_i := \text{SNARG.Prove}_L(\text{crs}_{\text{loc}}, [M_{\text{loc}}]_i)$ 
5 :  $[M_{\text{pub}}]_i := \text{HSS.Eval}(i, C_{\text{NL}}^{\text{pub}}, x_{\text{pub}}, \langle x \rangle_i, \langle r_{\text{pub}} \rangle_i)$ 
6 :  $[\pi_{\text{pub}}]_i := \text{SNARG.Prove}_L(\text{crs}_{\text{pub}}, [M_{\text{pub}}]_i)$ 
7 :  $[\text{out}]_i = ([y]_i, [\pi_{\text{loc}}]_i, [\pi_{\text{pub}}]_i)$ 
8 : return  $[\text{out}]_i$ 

```

ve-HSS.Recon($[\text{out}]_1, \dots, [\text{out}]_m$)

```

1 : parse each  $[\text{out}]_i = ([y]_i, [\pi_{\text{loc}}]_i, [\pi_{\text{pub}}]_i)$ 
2 :  $y := \sum_{i=1}^m [y]_i$ 
3 :  $\pi_{\text{loc}} := \sum_{i=1}^m [\pi_{\text{loc}}]_i$ 
4 :  $\pi_{\text{pub}} := \sum_{i=1}^m [\pi_{\text{pub}}]_i$ 
5 :  $\text{out} := (y, \pi_{\text{loc}}, \pi_{\text{pub}})$ 
6 : return out

```

ve-HSS.LVerify($\text{crs}, x_{\text{pub}}, x, y, \pi_{\text{loc}}$)

```

1 : parse  $\text{crs} = (\text{crs}_{\text{loc}}, \text{crs}_{\text{pub}}, C)$ 
2 :  $b := \text{SNARG.Verify}(\text{crs}_{\text{loc}}, x_{\text{pub}}, x, y, \pi_{\text{loc}})$ 
3 : return  $b$ 

```

ve-HSS.PVerify($\text{crs}, x_{\text{pub}}, y, \pi_{\text{pub}}$)

```

1 : parse  $\text{crs} = (\text{crs}_{\text{loc}}, \text{crs}_{\text{pub}}, C)$ 
2 :  $b := \text{SNARG.Verify}(\text{crs}_{\text{pub}}, x_{\text{pub}}, y, \pi_{\text{pub}})$ 
3 : return  $b$ 

```

Figure 6: ve-HSS construction from HSS and Splittable zkSNARG.

3. It then receives the public input $\mathbf{x}_{\text{pub}}$, the circuit output $\mathbf{y} = C(\mathbf{x}_{\text{pub}}, \mathbf{x})$ and the evaluated shares of the corrupt servers $[\text{out}]_{\mathcal{I}} = ([\bar{\mathbf{y}}]_{\mathcal{I}}, [\bar{\pi}_{\text{loc}}]_{\mathcal{I}}, [\bar{\pi}_{\text{pub}}]_{\mathcal{I}})$ as input and does the following.

(a) For every $i \in \mathcal{I}$, it re-computes the output share of the i -th server as

$$([y]_i, [\pi_{\text{loc}}]_i, [\pi_{\text{pub}}]_i) := \text{Eval}(\text{crs}, i, \mathbf{x}_{\text{pub}}, \langle \mathbf{x} \rangle_i, \langle r_{\text{loc}} \rangle_i, \langle r_{\text{pub}} \rangle_i).$$

(b) It computes

$$\epsilon_y = \sum_{i \in \mathcal{I}} [\bar{y}]_i - [y]_i, \quad \epsilon_{\text{loc}} = \sum_{i \in \mathcal{I}} [\bar{\pi}_{\text{loc}}]_i - [\pi_{\text{loc}}]_i, \quad \epsilon_{\text{pub}} = \sum_{i \in \mathcal{I}} [\bar{\pi}_{\text{pub}}]_i - [\pi_{\text{pub}}]_i.$$

(c) It simulates $\pi_{\text{pub}} \leftarrow \text{SNARG.Sim}(\text{st}_s, \mathbf{x}_{\text{pub}}, \mathbf{y})$ and computes $\bar{\pi}_{\text{pub}} := \pi_{\text{pub}} + \epsilon_{\text{pub}}$.

(d) If $\epsilon_y \neq 0$ or if $\epsilon_{\text{loc}} \neq 0$ then it sets $b := 0$; else it sets $b := 1$.

(e) It outputs $(b, \bar{\pi}_{\text{pub}})$.

Consider any polynomial size adversary $\mathcal{A}$ for the Privacy experiment. We show using a hybrid argument that for any circuit $C \in \mathbb{C}$, the view of $\mathcal{A}$ in $\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^0$ is indistinguishable from that in $\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^1$.

– Hyb_0 : This corresponds to the output of $\mathcal{A}$ in $\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^0(1^\lambda)$.

– Hyb_1 : In this hybrid, the experiment runs step 4 in the following manner.

– For each $i \in \mathcal{I}$, it computes the output share $[\text{out}]_i$ of the corrupt server as

$$([y]_i, [\pi_{\text{loc}}]_i, [\pi_{\text{pub}}]_i) := \text{Eval}(\text{crs}, i, \mathbf{x}_{\text{pub}}, \langle \mathbf{x} \rangle_i, \langle r_{\text{loc}} \rangle_i, \langle r_{\text{pub}} \rangle_i).$$

– It computes the reconstructed output $\text{out} = (\mathbf{y}, \pi_{\text{loc}}, \pi_{\text{pub}})$ as

$$\begin{aligned} \mathbf{y} &:= C(\mathbf{x}_{\text{pub}}, \mathbf{x}), \\ \pi_{\text{loc}} &:= \text{Prove}_{\text{L}}(\text{crs}_{\text{loc}}, \text{Prove}_{\text{NL}}(C_{\text{loc}}, \mathbf{x}_{\text{pub}}, \mathbf{x}, \mathbf{y}; r_{\text{loc}})), \\ \pi_{\text{pub}} &:= \text{Prove}_{\text{L}}(\text{crs}_{\text{pub}}, \text{Prove}_{\text{NL}}(C_{\text{pub}}, \mathbf{x}_{\text{pub}}, \mathbf{x}, \mathbf{y}; r_{\text{pub}})). \end{aligned}$$

– It then computes the share of the honest server as

$$[y]_{i^*} = \mathbf{y} - \sum_{i \in \mathcal{I}} [y]_i, \quad [\pi_{\text{loc}}]_{i^*} = \pi_{\text{loc}} - \sum_{i \in \mathcal{I}} [\pi_{\text{loc}}]_i, \quad [\pi_{\text{pub}}]_{i^*} = \pi_{\text{pub}} - \sum_{i \in \mathcal{I}} [\pi_{\text{pub}}]_i.$$

– It then continues to run step 4 of the $\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^0$ experiment using $[\text{out}]_{i^*}$ and $[\bar{\text{out}}]_{\mathcal{I}}$, where the latter is the output shares returned by the adversary.

In this hybrid, we essentially simulate the output share of the honest server as discussed in [Remark 1](#). The main difference between Hyb_0 and Hyb_1 is that in Hyb_0 the output shares of the honest servers were computed using Eval while in this hybrid, they are reverse sampled using the circuit output and proofs and the corrupt servers' shares of the circuit output and proofs. Specifically, since $C \in \mathbb{C}_{\text{hss}}$ and SNARG is $\mathbb{C}_{\text{hss}}$ -simple, it follows from the correctness of HSS and the linearity of additive secret sharing that the output shares of honest servers computed using Eval in Hyb_0 indeed correspond to shares of $\mathbf{y}$, π_{loc} , and π_{pub} as computed in Hyb_1 . Moreover, since $\mathcal{I} \subset [1, m]$, the privacy of additive secret sharing imply that the shares of honest servers reverse sampled in Hyb_1 correspond to shares of $\mathbf{y}$, π_{loc} , and π_{pub} . Thus, it follows that $\text{Hyb}_0 \stackrel{\epsilon}{\approx} \text{Hyb}_1$.

- Hyb_2 : In this hybrid, the experiment sends the HSS shares of the input computed by the simulator to the adversary.

The only difference in this hybrid is that $\langle \mathbf{x} \rangle_I$, $\langle r_{\text{loc}} \rangle_I$, and $\langle r_{\text{pub}} \rangle_I$ sent to the adversary are now computed as shares of 0 instead of the circuit inputs. Observe that this does not impact the other steps of the experiment since the experiment no longer uses the HSS shares of honest servers. It follows that $\text{Hyb}_1 \approx \text{Hyb}_2$ from the security of HSS.

- Hyb_3 : In this hybrid, the experiment outputs $b = 0$ if $\epsilon_y \neq 0$ where ϵ_y is as computed by the simulator. Observe that

$$\bar{y} = [y]_{i^*} + \sum_{i \in I} [\bar{y}]_i = y + \sum_{i \in I} [\bar{y}]_i - [y]_i = y + \epsilon_y$$

where the first equality follows from the definition of $\bar{y}$ and the third equality follows from the definition of ϵ_y . Thus, if $\epsilon_y \neq 0$, then $\bar{y} \neq C(\mathbf{x}_{\text{pub}}, \mathbf{x})$ which then implies that LVerify outputs 0 with overwhelming probability in Hyb_2 from the soundness of the SNARG and the fact that $C \in \mathbb{C}$. It follows that $\text{Hyb}_2 \approx \text{Hyb}_3$.

- Hyb_4 : In this hybrid, the experiment additionally outputs $b = 0$ when $\epsilon_{\text{loc}} \neq 0$.

Observe that $\mathcal{A}$'s view is independent of r_{loc} used to compute π_{loc} since the HSS shares are simulated. It follows from the DP-robust verification property of SNARG (Lemma 7) and the fact that SNARG is a zkSNARG for $\mathbb{C}$ -circuit satisfiability that LVerify outputs 0 in Hyb_3 with overwhelming probability when $\epsilon_{\text{loc}} \neq 0$. Thus, $\text{Hyb}_3 \approx \text{Hyb}_4$.

- Hyb_5 : In this hybrid, the experiment outputs $b = 1$ when $\epsilon_y = 0$ and $\epsilon_{\text{loc}} = 0$.

When $\epsilon_y = 0$ and $\epsilon_{\text{loc}} = 0$, we have $\bar{y} = y$ and $\overline{\pi_{\text{loc}}} = \pi_{\text{loc}}$ respectively. Since $C \in \mathbb{C}$, it follows from the DP-completeness (Lemma 6) of SNARG that that $\text{LVerify}(\text{crs}_{\text{loc}}, \mathbf{x}_{\text{pub}}, \mathbf{x}, \bar{y}, \overline{\pi_{\text{loc}}}) = 1$ in Hyb_4 with overwhelming probability. Thus, $\text{Hyb}_4 \approx \text{Hyb}_5$.

- Hyb_6 : In this hybrid, the experiment computes crs_{pub} and $\overline{\pi_{\text{pub}}}$ using the simulator.

Observe that

$$\overline{\pi_{\text{pub}}} = [\pi_{\text{pub}}]_{i^*} + \sum_{i \in I} [\overline{\pi_{\text{pub}}}]_i = \pi_{\text{pub}} + \sum_{i \in I} [\overline{\pi_{\text{pub}}}]_i - [\pi_{\text{pub}}]_i = \pi_{\text{pub}} + \epsilon_{\text{pub}}$$

where the first equality follows from the definition of $\overline{\pi_{\text{pub}}}$ and the third equality follows from the definition of ϵ_{pub} . Moreover, the adversary's view is independent of the randomness r_{pub} used to compute π_{pub} in Hyb_5 since the HSS shares $\langle r_{\text{pub}} \rangle_I$ are simulated, which from the zero-knowledge property of SNARG implies that $\text{Hyb}_5 \approx \text{Hyb}_6$.

Hyb_6 corresponds to the output of $\mathcal{A}$ in $\text{Privacy}_{\mathcal{A}, \text{Sim}, \mathbb{C}}^1(1^\lambda)$ which in turn implies from our sequence of hybrids that

$$\left| \Pr \left[\text{Privacy}_{\mathcal{A}, \text{Sim}, \mathbb{C}}^0(1^\lambda) = 1 \right] - \Pr \left[\text{Privacy}_{\mathcal{A}, \text{Sim}, \mathbb{C}}^1(1^\lambda) = 1 \right] \right| \leq \text{negl}(\lambda).$$

Soundness and efficiency. Local and public soundness of Figure 6 follow immediately from the soundness of SNARG. Moreover, the construction has succinct input shares since SNARG has succinct randomness and it has succinct output shares since HSS is compact and SNARG has succinct proof size. Succinct verification of the ve-HSS construction follows from succinct verification of SNARG. It then follows from our argument above that Figure 6 is an m -server ve-HSS scheme for $\mathbb{C}$. ■

Public knowledge soundness. Observe that public soundness of Figure 6 follows directly from the soundness of SNARG. Thus, if SNARG is a Splittable zkSNARK then the resulting ve-HSS scheme additionally has public knowledge soundness.

Remark 5 (On Requiring HSS with Negligible Correctness Error). *Note that privacy of the ve-HSS construction (specifically, the indistinguishability of Hyb_0 and Hyb_1) requires that the underlying HSS scheme have negligible correctness error. As a result, HSS schemes such as that of [BGI16a], which have inverse polynomial correctness error—where the probability of error depends on the input—not only induce an inverse polynomial correctness error in the resulting ve-HSS from Figure 6, but also weaken the privacy guarantee. In particular, learning if the output reconstruction was successful could leak information about the private inputs. However, the resulting ve-HSS construction still satisfies weak privacy as defined above, which guarantees that the adversary’s input shares do not provide any additional information about the inputs.*

Remark 6 (Extension to threshold HSS). *In this work, we primarily consider m -out-of- m HSS schemes, where reconstruction requires the output shares of all m servers and privacy is guaranteed as long as at least one server is honest. However, we note that our techniques naturally extend to HSS schemes with arbitrary access structures. Specifically, our construction only requires that the output shares of HSS correspond to that of a linear secret sharing scheme and that the output shares of honest servers are efficiently sampleable given the output shares of corrupt servers.*

5.1 Instantiations

We now discuss instantiating our ve-HSS template described in Figure 6 using HSS schemes from prior works and the Splittable zkSNARKs discussed in Section 4, namely Groth16 and the Waters-Wu zkBARG.

In the remainder of this section we will use $\text{VP}[p]$ to denote the set of polynomial size circuits over $\mathbb{Z}_p$ and use $\text{VNC}^1[p]$ denote the set of polynomial size circuits over $\mathbb{Z}_p$ that have logarithmic depth in the number of inputs.

Recall that our ve-HSS construction requires an HSS scheme with negligible correctness error that supports evaluating the non-linear phase of the Splittable zkSNARG. As discussed in Section 3.1.1, there are two broad groups of HSS schemes with negligible correctness error in the literature. The first group consists of 2-server HSS schemes from group-based assumptions [BKS19, OSY21, RS21, ADOS22] that support evaluating RMS programs with an exponentially large magnitude bound. From Lemma 5, these HSS schemes can also be used to evaluate circuits in $\text{VNC}^1[p]$ for $p \in 2^{O(\lambda)}$.¹⁷ The second group consists of m -server HSS schemes for evaluating $\text{VP}[p]$ [BGI15, COS⁺22], where m is polynomial in λ and $p \in 2^{O(\lambda)}$. Thus, our discussion below will focus on two classes of ve-HSS instantiations, depending on the underlying HSS scheme.

Groth16 based ve-HSS. From Theorem 9, we know that the non-linear phase of Groth16, when proving satisfiability of a boolean circuit of depth D , requires evaluating a circuit of depth $O(D)$ over $\mathbb{Z}_p$, where $p \in O(2^\lambda)$ is the order of the bilinear group used for Groth16. It then follows that Groth16 is a $\text{VNC}^1[p]$ -simple Splittable zkSNARG for NC^1 -circuit satisfiability and that it is a $\text{VP}[p]$ -simple Splittable zkSNARG for P/poly-circuit satisfiability. The following corollary of Theorem 12 then follows from the fact that HSS schemes from prior works support computing over $\text{VNC}^1[p]$ and $\text{VP}[p]$, as discussed above.

Corollary 13. *There exist the following ve-HSS schemes with local and public knowledge soundness in the generic bilinear group model:*

¹⁷In general, the RMS programs resulting from Lemma 5 will have a magnitude bound of $2^{O(\lambda \cdot n)}$, leading to HSS input shares of size $\text{poly}(\lambda \cdot n^2)$ bits in existing group-based schemes. However, as noted in Remark 7, our ve-HSS instantiations only require bounds of $2^{O(\lambda)}$, reducing the share size to $O(\lambda \cdot n)$ bits.

- A 2-server ve-HSS scheme for NC^1 assuming the hardness of the LWE assumption,
- A 2-server ve-HSS scheme for NC^1 assuming the hardness of the DCR assumption,
- A 2-server ve-HSS scheme for NC^1 assuming the hardness of the DDH assumption and the small exponent assumption over class groups,
- An $m(\lambda)$ -server ve-HSS scheme for P/poly , for any integer valued polynomial $m(\cdot)$, assuming the existence of sub-exponentially secure indistinguishability obfuscation and sub-exponentially secure one-way functions,
- An $m(\lambda)$ -server ve-HSS scheme for P/poly , for any integer valued polynomial $m(\cdot)$, assuming the existence of circular secure AFS-spooky encryption.

We note that verifiable evaluation comes at little to no cost to the input and output share sizes in this case. Specifically, the Groth16 prover requires two uniformly random elements from $\mathbb{Z}_p$ and the Groth16 proof consists of 3 group elements each of which is $O(\log \lambda)$ bits long. Thus, Figure 6 when instantiated with an HSS scheme HSS and Groth16 incurs only an additive overhead of $O(\log \lambda)$ to the size of the input and output shares compared to that of HSS.

Waters-Wu zkBARG based ve-HSS. Similar to Groth16, it follows from Theorem 11 that the Waters-Wu zkBARG is a $\text{VNC}^1[N]$ -simple zkSNARG for SIMD NC^1 -circuit satisfiability and is a $\text{VP}[N]$ -simple zkSNARG for SIMD P/poly -circuit satisfiability, where N is the order of the bilinear groups used for the zkBARG. Since the zkBARG is only non-adaptively sound, the resulting ve-HSS only achieves non-adaptive local and public soundness and non-adaptive privacy.¹⁸ Thus, we have the following corollary of Theorem 12.

Corollary 14. *There exist the following ve-HSS schemes with non-adaptive local and public soundness, and non-adaptive privacy, all of which assume the hardness of the subgroup decision assumption over composite-order bilinear groups:*

- A 2-server ve-HSS scheme for SIMD NC^1 additionally assuming the hardness of the LWE assumption,
- A 2-server ve-HSS scheme for SIMD NC^1 additionally assuming the hardness of the DCR assumption,
- A 2-server ve-HSS scheme for SIMD NC^1 additionally assuming the hardness of the DDH assumption and the small exponent assumption over class groups,
- An $m(\lambda)$ -server ve-HSS scheme for SIMD P/poly , for any integer valued polynomial $m(\cdot)$, additionally assuming the existence of sub-exponentially secure indistinguishability obfuscation and sub-exponentially secure one-way functions,
- An $m(\lambda)$ -server ve-HSS scheme for SIMD P/poly , for any integer valued polynomial $m(\cdot)$, additionally assuming the existence of circular secure AFS-spooky encryption.

Remark 7. As discussed earlier, existing group-based HSS schemes support evaluating RMS programs over bounded integers, which from Lemma 5, suffice for the non-linear phases of Groth16 and the Waters-Wu zkBARG when proving satisfiability of NC^1 and SIMD NC^1 circuits, respectively. When these non-linear phases are evaluated using RMS programs, the values are bounded by $O(p^2)$ for Groth16 and $O(N^2)$ for the Waters-Wu zkBARG. This follows from the fact that a magnitude bound of 1 suffices to compute the extended witness

¹⁸Non-adaptive privacy stems from using the soundness of the zkBARG to ensure correctness of the output in the security proof. Standard privacy can be recovered using a MAC check, as done in the multi-client construction in Section 6.

produced by the non-linear phase—since these are just intermediate values in a Boolean circuit [BGI16a]—and the only additional operations involve pairwise multiplications of random values in $\mathbb{Z}_p$ (or $\mathbb{Z}_N$ for Waters-Wu) and their products with the extended witness. Hence, all intermediate values remain bounded by $O(p^2)$ and $O(N^2)$ in the respective constructions.

Finally, we note that an HSS scheme supporting RMS programs with magnitude bounds polynomial in the security parameter also suffices to instantiate the ve-HSS scheme. Specifically, the extended witness from the non-linear phase can be computed using such a scheme, as discussed above. Since the computation over the larger ring has constant depth, it can be expressed by a log-depth Boolean circuit [vzGS91] and thus by an RMS program with magnitude bound 1 [BGI16a]. However, this requires sharing each bit of the non-binary ring elements and involves non-black-box use of arithmetic over the large ring.

6 Multi-Client HSS with Verifiable Evaluation

In this section, we generalize our ve-HSS definition to accommodate multiple clients. We first define multi-client ve-HSS for semi-honest clients and malicious servers in Definition 8, then strengthen the definition to model maliciously corrupt clients and servers in Definition 9.

Similar to public-key HSS (Definition 2), a multi-client ve-HSS scheme generates a common public key pk and an evaluation key ek_i for each server. Each client P_i^c computes an encryption $\tilde{x}_i$ of its input x_i using the public key, which it sends to all servers. Given a public input x_{pub} and the client ciphertexts $\{\tilde{x}_i\}_i$, each server uses its evaluation key to locally compute a share of the output $y = C(x_{\text{pub}}, x_1, \dots, x_n)$. Public verification supports verifying that the reconstructed output y is consistent with the public input x_{pub} . Moreover, each server obtains an output share for each client P_i^c , which when reconstructed, allows P_i^c to locally verify if the output y is consistent with its input x_i i.e., there exists $\{x_j\}_{j \neq i}$ such that $y = C(x_{\text{pub}}, x_1, \dots, x_n)$. Privacy requires that an adversary who semi-honestly corrupts a subset of the clients and maliciously corrupts a subset of the servers learns nothing beyond the inputs of the corrupt clients and the output, even if it learns whether verification by honest clients succeeds or fails. Similarly, correctness, efficiency, and soundness are straightforward extensions of the properties defined for ve-HSS.

Definition 8 (Multi-Client HSS with verifiable evaluation). *Let $m := m(\lambda)$ and $n := n(\lambda)$ be integer-valued polynomials, R be a ring, and $\mathbb{C}$ be a class of circuits over R . An n -client, m -server Multi-Client HSS with verifiable evaluation (MveHSS) scheme for $\mathbb{C}$ is a tuple of algorithms $\text{MveHSS} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Eval}, \text{Recon}, \text{LVerify}, \text{PVerify})$ with the following syntax.*

- $\text{Setup}(1^\lambda, C) \rightarrow \text{crs}$ is a PPT algorithm that takes the description of a circuit $C \in \mathbb{C}$ as input and outputs the common reference string crs .
- $\text{KeyGen}(1^\lambda) \rightarrow (pk, ek_1, \dots, ek_m)$ is a PPT algorithm that outputs a public key pk and an evaluation key ek_i for the i -th server, for each $i \in [1, m]$.
- $\text{Enc}(\text{crs}, i, pk, x) \rightarrow (st, \tilde{x})$ is a PPT algorithm that takes as input the common reference string crs , the client index i , the public key pk , the client input x and outputs a ciphertext $\tilde{x}$ and local client state st .
- $\text{Eval}(\text{crs}, ek_i, x_{\text{pub}}, \tilde{x}_1, \dots, \tilde{x}_n) =: ([\text{out}_{\text{pub}}]_i, [\text{out}_1]_i, \dots, [\text{out}_n]_i)$ is a polynomial time algorithm that takes as input the common reference string crs , the evaluation key of the i -th server ek_i , the public input x_{pub} , and the ciphertexts corresponding to the inputs of n clients. It outputs a share of the j -th client's output $[\text{out}_j]_i$, for each $j \in [1, n]$, as well as a share of the public output $[\text{out}_{\text{pub}}]_i$.
- $\text{Recon}(\text{crs}, [\text{out}_i]) =: (y, \pi_{\text{pub}})/\pi_i$ is a polynomial time algorithm that takes the shares of the public output $[\text{out}_{\text{pub}}]$ and reconstructs it to compute the circuit output y and public proof π_{pub} . When run on the shares of the i -th client's output $[\text{out}_i]$, it outputs the proof π_i for the i -th client.

MveHSS Privacy Experiment

Privacy $^0_{\mathcal{A}, \text{Sim}, \mathbf{C}}(1^\lambda)$

- 1: Compute $\text{crs} \leftarrow \text{Setup}(1^\lambda, \mathbf{C})$ and send crs to $\mathcal{A}$.
- 2: Receive $(I, I_c, \mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n)$ from $\mathcal{A}$. Output 0 if $I_c \notin [1, n]$, $I \notin [1, m]$ or if $|I| = m$.
- 3: Compute

$$(\text{pk}, \text{ek}_1, \dots, \text{ek}_m) \leftarrow \text{KeyGen}(1^\lambda)$$

$$\forall i \in [1, n], (\text{st}_i, \tilde{\mathbf{x}}_i) \leftarrow \text{Enc}(\text{crs}, i, \text{pk}, \mathbf{x}_i)$$

and send pk , $\{\text{ek}_i\}_{i \in I}$, $\{\text{st}_i\}_{i \in I_c}$, and $(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_n)$ to $\mathcal{A}$.

- 4: Receive $[\overline{\text{out}_{\text{pub}}}]_I, [\overline{\text{out}_1}]_I, \dots, [\overline{\text{out}_n}]_I$ from $\mathcal{A}$.
- 5: For each $i \in [1, m] \setminus I$, compute

$$([\overline{\text{out}_{\text{pub}}}]_i, [\overline{\text{out}_1}]_i, \dots, [\overline{\text{out}_n}]_i) :=$$

$$\text{Eval}(\text{crs}, \text{ek}_i, \mathbf{x}_{\text{pub}}, \tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_n).$$

Then, compute

$$(\bar{\mathbf{y}}, \overline{\pi_{\text{pub}}}) :=$$

$$\text{Recon}(\text{crs}, [\overline{\text{out}_{\text{pub}}}]_I, [\overline{\text{out}_{\text{pub}}}]_{[1, m] \setminus I}),$$

and for each $i \in [1, n]$ compute

$$\bar{\pi}_i := \text{Recon}(\text{crs}, [\overline{\text{out}_i}]_I, [\overline{\text{out}_i}]_{[1, m] \setminus I})$$

$$b_i \leftarrow \text{LVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{x}_i, \bar{\mathbf{y}}, \text{st}_i, \bar{\pi}_i).$$

- 6: Send $(\{b_i\}_{i \notin I_c}, \overline{\pi_{\text{pub}}}, \{\bar{\pi}_i\}_{i \in I_c})$ to $\mathcal{A}$ and output the bit returned by $\mathcal{A}$.

Privacy $^1_{\mathcal{A}, \text{Sim}, \mathbf{C}}(1^\lambda)$

- 1: Compute $(\text{crs}, \text{st}) \leftarrow \text{Sim}(1^\lambda, \mathbf{C})$ and send crs to $\mathcal{A}$.
- 2: Receive $(I, I_c, \mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n)$ from $\mathcal{A}$. Output 0 if $I_c \notin [1, n]$, $I \notin [1, m]$ or if $|I| = m$.
- 3: Compute

$$(\text{pk}, \{\text{ek}_i\}_{i \in I}, \{\text{st}_i\}_{i \in I_c}, \{\tilde{\mathbf{x}}_i\}_{i=1}^n, \text{st}) \leftarrow$$

$$\text{Sim}(\text{st}, \text{pk}, \{|\mathbf{x}_i|\}_{i \notin I_c}, \{\mathbf{x}_i\}_{i \in I_c})$$

and send pk , $\{\text{ek}_i\}_{i \in I}$ and $(\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_n)$ to $\mathcal{A}$.

- 4: Receive $[\overline{\text{out}_{\text{pub}}}]_I, [\overline{\text{out}_1}]_I, \dots, [\overline{\text{out}_n}]_I$ from $\mathcal{A}$.
- 5: Compute

$$(\{b_i\}_{i \notin I_c}, \overline{\pi_{\text{pub}}}, \{\bar{\pi}_i\}_{i \in I_c}) \leftarrow$$

$$\text{Sim}(\text{st}, \mathbf{x}_{\text{pub}}, \mathbf{y}, [\overline{\text{out}_{\text{pub}}}]_I, \{[\overline{\text{out}_i}]_I\}_{i=1}^n)$$

where $\mathbf{y} = \mathbf{C}(\mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n)$.

- 6: Send $(\{b_i\}_{i \notin I_c}, \overline{\pi_{\text{pub}}}, \{\bar{\pi}_i\}_{i \in I_c})$ to $\mathcal{A}$ and output the bit returned by $\mathcal{A}$.

Figure 7: MveHSS privacy experiment for [Definition 8](#).

- $\text{LVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{x}_i, \mathbf{y}, \text{st}_i, \pi_i) \rightarrow b$ is a PPT algorithm that takes the common reference string crs , the public input $\mathbf{x}_{\text{pub}}$, the i -th client's input $\mathbf{x}_i$ and local state st_i , the circuit output $\mathbf{y}$, and a proof π_i as input. It outputs a bit b indicating if π_i is accepted or rejected.
- $\text{PVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{y}, \pi_{\text{pub}}) \rightarrow b$ is a PPT algorithm that takes the common reference string crs , the public input $\mathbf{x}_{\text{pub}}$, the circuit output $\mathbf{y}$ and a proof π_{pub} as input, and outputs a bit b indicating if π_{pub} is accepted or rejected.

An MveHSS scheme should satisfy the following properties.

- **Efficiency:** An MveHSS scheme is said to be efficient if it has linear reconstruction, succinct shares, succinct verification, and circuit independent key generation as defined below.
 - **Linear Reconstruction:** $\text{Recon}(\text{crs}, \cdot)$ is linear in its inputs.
 - **Succinct Shares:** There exists a polynomial p and a sublinear function $g(x) \in o(x)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$ with ℓ_{inp} -inputs and ℓ_{out} -outputs, the output length of Enc is at most $p(m\lambda\ell_{\text{inp}})g(|C|)$ and the output length of Eval is at most $p(\lambda\ell_{\text{out}})g(|C|)$.
 - **Succinct Verification:** There exists a polynomial p and a sublinear function $g(x) \in o(x)$ such that for every $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$ with ℓ_{inp} -inputs and ℓ_{out} -outputs, the runtime of LVerify and PVerify are at most $p(\lambda(\ell_{\text{inp}} + \ell_{\text{out}}))g(|C|)$. Note that the verification time is independent of the number of servers m .
 - **Circuit Independent Key Generation:** KeyGen is circuit independent i.e., the keys output by KeyGen can be used to evaluate any circuit in $\mathbb{C}$.
- **Correctness:** There exists a negligible function $\text{negl}(\cdot)$ such that for all integer-valued polynomials $\ell := \ell(\lambda)$, circuits $C \in \mathbb{C}$, inputs $\mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n \in R^\ell$, and $\lambda \in \mathbb{N}$, we have

$$\Pr \left[\begin{array}{l} \mathbf{y} \neq C(\mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n) \quad \vee \\ \text{PVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{y}, \pi_{\text{pub}}) \neq 1 \quad \vee \\ \exists i \in [1, n], \text{LVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{x}_i, \mathbf{y}, \text{st}_i, \pi_i) \neq 1 \end{array} \right] \leq \text{negl}(\lambda),$$

where $\text{crs} \leftarrow \text{Setup}(1^\lambda, C)$, $(\text{pk}, \text{ek}_1, \dots, \text{ek}_m)$ is generated using KeyGen , each $(\text{st}_i, \tilde{\mathbf{x}}_i) \leftarrow \text{Enc}(\text{crs}, i, \text{pk}, \mathbf{x}_i)$, $[\text{out}_{\text{pub}}]$ and each $[\text{out}_i]$ are obtained by running Eval with each server's evaluations key, $(\mathbf{y}, \pi_{\text{pub}})$ is obtained by reconstructing $[\text{out}_{\text{pub}}]$ and each π_i is obtained by reconstructing $[\text{out}_i]$.

- **Privacy:** There exists a PPT simulator Sim such that for all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$, such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$,

$$\left| \Pr \left[\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^0(1^\lambda) = 1 \right] - \Pr \left[\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^1(1^\lambda) = 1 \right] \right| \leq \text{negl}(\lambda)$$

where $\text{Privacy}_{\mathcal{A}, \text{Sim}, C}^b(1^\lambda)$ is defined in Figure 7.

- **Local Soundness:** For every polynomial size adversary $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$ we have

$$\Pr \left[\begin{array}{l} (C, \mathbf{x}_{\text{pub}}, \mathbf{x}_i, \mathbf{y}) \notin \text{SAT} \\ \wedge \\ \text{LVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{x}_i, \mathbf{y}, \text{st}_i, \pi_i) = 1 \end{array} \right] \left| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (\text{pk}, \text{ek}_1, \dots, \text{ek}_m) \leftarrow \text{KeyGen}(1^\lambda) \\ (\mathbf{x}_{\text{pub}}, i, \mathbf{x}_i, \text{st}_a) \leftarrow \mathcal{A}(\text{crs}, \text{pk}, \text{ek}_1, \dots, \text{ek}_m) \\ (\text{st}_i, \tilde{\mathbf{x}}_i) \leftarrow \text{Enc}(\text{crs}, i, \text{pk}, \mathbf{x}_i) \\ ([\text{out}_{\text{pub}}], [\text{out}_i]) \leftarrow \mathcal{A}(\text{st}_a, \tilde{\mathbf{x}}_i) \\ (\mathbf{y}, \pi_{\text{pub}}) \leftarrow \text{Recon}(\text{crs}, [\text{out}_{\text{pub}}]) \\ \pi_i \leftarrow \text{Recon}(\text{crs}, [\text{out}_i]) \end{array} \right| \leq \text{negl}(\lambda).$$

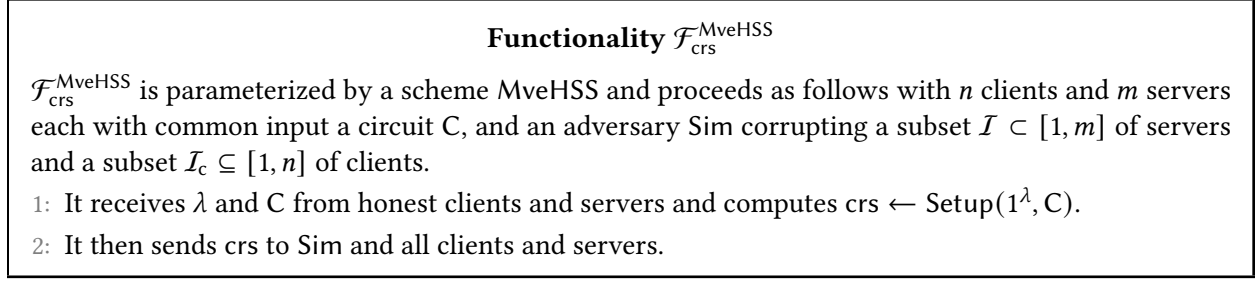


Figure 8: CRS functionality.

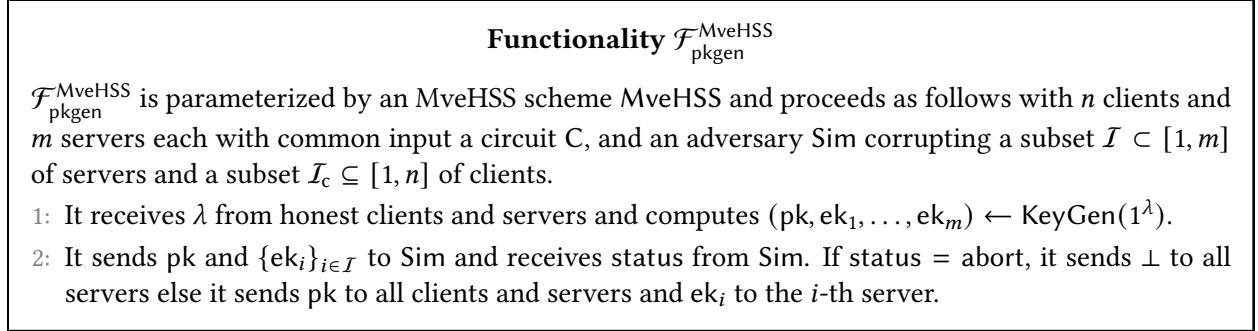


Figure 9: Key generation functionality.

- **Public Soundness:** For all polynomial size adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}$ we have

$$\Pr \left[\begin{array}{c} (C, \mathbf{x}_{\text{pub}}, \mathbf{y}) \notin \text{SAT} \\ \wedge \\ \text{PVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{y}, \pi_{\text{pub}}) = 1 \end{array} \middle| \begin{array}{c} \text{crs} \leftarrow \text{Setup}(1^\lambda, C) \\ (\mathbf{x}_{\text{pub}}, \mathbf{y}, \pi_{\text{pub}}) \leftarrow \mathcal{A}(\text{crs}) \end{array} \right] \leq \text{negl}(\lambda).$$

Public and local knowledge soundness, as well as non-adaptive soundness and privacy, can also be defined for MveHSS, similar to how they're defined for ve-HSS in [Section 5](#). We omit the explicit definitions here, since they can be easily obtained by adapting [Definition 8](#).

Note that privacy in [Definition 8](#) is defined with respect to semi-honest clients since the encryption of each input is computed by the game and not the adversary in [Figure 7](#). We next define a stronger notion of MveHSS that considers an adversary that maliciously corrupts both clients and servers.

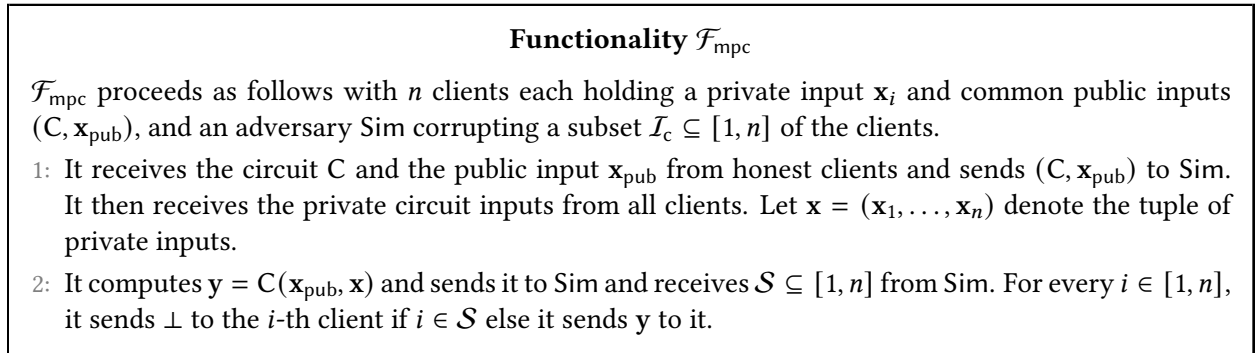


Figure 10: Functionality for ideal world definition of strong privacy in [Definition 9](#).

Protocol $\Pi_{\text{mpc}}^{\text{MveHSS}}$

$\Pi_{\text{mpc}}^{\text{MveHSS}}$ is parameterized by a scheme MveHSS and is described in the $\{\mathcal{F}_{\text{crs}}^{\text{MveHSS}}, \mathcal{F}_{\text{pkgen}}^{\text{MveHSS}}\}$ -hybrid model (Figures 8 and 9). It runs with clients $\mathcal{P}_c = \{P_1^c, \dots, P_n^c\}$ and servers $\mathcal{P} = \{P_1, \dots, P_m\}$ where P_i^c has private input $\mathbf{x}_i$ and all clients and servers have the public input $\mathbf{x}_{\text{pub}}$ and the circuit C . The protocol runs in the presence of a static adversary $\mathcal{A}$ that corrupts a subset $\mathcal{I} \subset [1, m]$ of servers and a subset $\mathcal{I}_c \subseteq [1, n]$ of clients.

- 1: Each client and server invokes $\mathcal{F}_{\text{crs}}^{\text{MveHSS}}$ with circuit C to receive crs .
- 2: Each client and server invokes $\mathcal{F}_{\text{pkgen}}^{\text{MveHSS}}$ to receive pk . For each $i \in [1, m]$, the i -th server P_i additionally receives its evaluation key ek_i .
- 3: Each client P_i^c encrypts its input as $(\text{st}_i, \tilde{\mathbf{x}}_i) \leftarrow \text{Enc}(\text{crs}, \text{pk}, \mathbf{x}_i)$ and broadcasts $\tilde{\mathbf{x}}_i$.
- 4: Each server P_i locally computes $([\text{out}_{\text{pub}}]_i, [\text{out}_1]_i, \dots, [\text{out}_n]_i) := \text{Eval}(\text{crs}, \text{ek}_i, \mathbf{x}_{\text{pub}}, \tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_n)$.
- 5: Each server P_i sends $[\text{out}_{\text{pub}}]_i$ to all clients and $[\text{out}_j]_i$ to P_j^c , for each $j \in [1, n]$.
- 6: Each client P_i^c locally computes $(\mathbf{y}, \pi_{\text{pub}}) := \text{Recon}(\text{crs}, [\text{out}_{\text{pub}}])$ and $\pi_i := \text{Recon}(\text{crs}, [\text{out}_i])$.
- 7: Each client P_i^c locally computes $\text{LVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \text{st}_i, \mathbf{y}, \pi_i) \rightarrow b_i$. It outputs $\mathbf{y}$ if $b_i = 1$ and outputs $\perp$ otherwise.

Figure 11: Protocol for real world definition of strong privacy in Definition 9.

Definition 9 (MveHSS With Strong Privacy). *An m -server MveHSS scheme MveHSS for a class of circuits $\mathbb{C}$ over a ring R is said to have strong privacy if for every polynomial size adversary $\mathcal{A}$ there exists a simulator Sim such that*

$$\left\{ \text{Real}_{\mathcal{A}(z)}^{\mathcal{I}, \mathcal{I}_c}(1^\lambda, z, \mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n) \right\} \stackrel{\mathcal{L}}{\approx} \left\{ \text{Ideal}_{\text{Sim}(z)}^{\mathcal{I}, \mathcal{I}_c}(1^\lambda, z, \mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n) \right\}$$

where the ensembles are parameterized by $\lambda \in \mathbb{N}$, circuit $C \in \mathbb{C}$, an integer-valued polynomial n , inputs $\mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n \in R^*$, sets $\mathcal{I} \subset [1, m]$ and $\mathcal{I}_c \subseteq [1, n]$, and auxiliary information $z \in \{0, 1\}^*$. $\text{Real}_{\mathcal{A}(z)}^{\mathcal{I}, \mathcal{I}_c}$ corresponds to the view of $\mathcal{A}$ and output of honest clients in $\Pi_{\text{mpc}}^{\text{MveHSS}}$ (Figure 11). $\text{Ideal}_{\text{Sim}(z)}^{\mathcal{I}, \mathcal{I}_c}$ corresponds to the output of the simulator and honest clients upon interacting with the $\mathcal{F}_{\text{mpc}}$ functionality (Figure 10).

Since the real and the ideal world ensembles include the output of honest parties, strong privacy captures correctness as defined in Section 6.2.

Remark 8 (Publicly Auditable MPC). *MveHSS with strong privacy implies a two-round client-server MPC with selective abort, assuming trusted setup. This follows immediately from the definition of strong privacy. Moreover, publishing $\mathbf{y}$ and the proof π_{pub} onto a bulletin board allows any external party to verify the correctness of $\mathbf{y}$ without providing any additional information about the private inputs of clients. Thus, strong privacy implies the stronger notion of MPC with public auditability [BDO14], with succinct public proofs.*

6.1 Construction With Strong Privacy

In this section, we present our construction for Multi-Client HSS with verifiable evaluation with strong privacy. In Section 6.2, we discuss how the construction can be simplified when security is required only against semi-honest clients.

Our MveHSS construction follows a similar approach as our ve-HSS construction where clients first use an HSS scheme to share their inputs followed by each server evaluating a circuit C and the non-linear

phase of a Splittable zkSNARG for C using the HSS scheme to obtain its share of the outputs. Servers then evaluate the linear phase of the zkSNARG over their shares to obtain shares of the proofs and then reconstruct the circuit output and proofs towards the clients. To support multiple clients, we use a public-key HSS scheme which allows any client holding a public key pk to encrypt its input for HSS evaluation. However, as discussed in [Section 2](#), there are a number of changes we need to make to our approach to ensure security of the construction.

Semi-malicious correctness. Firstly, we require a stronger correctness property from public-key HSS schemes, called *semi-malicious correctness*, defined below. This requires that correctness of the scheme holds even when the randomness used for encrypting inputs is adversarially chosen. This is because, in our construction, the adversary computes encryptions of corrupt clients' inputs possibly based on the ciphertexts sent by honest clients. This stronger notion of adaptive correctness can be realized in the random oracle model for most HSS schemes, as discussed in [\[RS21, ADOS22\]](#).

Definition 10 (HSS Scheme with Semi-malicious Correctness). *An m -server public-key HSS scheme $HSS = (\text{KeyGen}, \text{Enc}, \text{Eval})$ for a class of circuits $\mathbb{C}_{\text{hss}}$ over a ring R is said to have semi-malicious correctness if for all polynomial size adversaries, there exists a negligible function $\text{negl}(\cdot)$, such that for all $\lambda \in \mathbb{N}$ and circuits $C \in \mathbb{C}_{\text{hss}}$, we have*

$$\Pr \left[\begin{array}{c} [y]_1 + \dots + [y]_m \\ \neq \\ C(x_{\text{pub}}, x_1, \dots, x_n) \end{array} \mid \begin{array}{l} (pk, ek_1, \dots, ek_m) \leftarrow \text{KeyGen}(1^\lambda) \\ (x_{\text{pub}}, (x_1, r_1), \dots, (x_n, r_n)) \leftarrow \mathcal{A}(1^\lambda, pk, ek_1, \dots, ek_m) \\ \tilde{x}_i \leftarrow \text{Enc}(pk, x_i; r_i), \forall i \in [1, n] \\ [y]_i := \text{Eval}(ek_i, C, x_{\text{pub}}, \tilde{x}_1, \dots, \tilde{x}_n) \end{array} \right] \leq \text{negl}(\lambda).$$

Randomness for zkSNARG. The randomness used to compute each zkSNARG proof must be uniformly random to ensure zero-knowledge, and consequently, the privacy of the inputs. This randomness is thus computed as the sum of the randomness shared by each client. This ensures uniform randomness as long as at least one client is honest; if all clients are corrupt, the adversary already knows all inputs, making the zero-knowledge requirement vacuous.

Well-formedness of ciphertexts. To ensure ciphertexts shared by clients are well-formed, we employ a NIZK functionality $\mathcal{F}_{\text{NIZK}}$ for the relation $\mathcal{R}_{\text{hss}}$ ([Figure 12](#)), to verify that each ciphertext indeed corresponds to an encryption under pk . The NIZK statement consists of the HSS public key pk and a ciphertext $\tilde{x}$ while the witness consists of the plaintext x and the randomness r used for encryption.

Correctness of output. Finally, to ensure correctness of the reconstructed output, we employ a BeDOZa-style MAC check [\[BDOZ11\]](#) as in [\[ADOS22\]](#). Each party shares a MAC key that is used to authenticate their respective output. The client can then use the MAC to verify the output correctness. Let the HSS scheme support evaluating circuits over a ring R . To ensure soundness of the MAC check, we assume that the smallest prime divisor p of the ring order $|R|$ is such that $1/p$ is negligible in the security parameter.¹⁹ If this is not the case, simply repeating the check with k independently sampled MAC keys, such that $1/p^k$ is negligible, suffices to ensure soundness with overwhelming probability.

Our MveHSS scheme is described in [Figures 13 and 14](#) and its security is established in the following theorem. The theorem requires that for every circuit supported by the MveHSS scheme, the underlying HSS scheme supports two operations: (1) evaluating a MAC on the output under each client's key, and (2) evaluating the non-linear phase of the Splittable zkSNARG with randomness computed as the sum of inputs shared by clients.

¹⁹Formally, this requires that the HSS scheme be defined over a family of rings indexed by the security parameter, but we abuse notation for ease of exposition.

Functionality $\mathcal{F}_{\text{NIZK}}$

$\mathcal{F}_{\text{NIZK}}$ is parameterized by a public-key HSS scheme HSS. Let $\mathcal{R}_{\text{hss}}$ denote the relation

$$\mathcal{R}_{\text{hss}} = \{(\text{pk}, \mathcal{M}, \tilde{x}; x, r) \mid \tilde{x} = \text{HSS.Enc}(\text{pk}, x; r) \wedge x \in \mathcal{M}\}$$

Proof: On input (prove, x, w) from a party P , check if $(x; w) \in \mathcal{R}_{\text{hss}}$ and if so send (prove, P, x) to Sim. Upon receiving (proof, π) from Sim, store (x, w, π) and send (proof, π) to P .

Verify: On input (verify, x, π) from any party V , do the following.

- Check if (x, w, π) is stored for some w and if not send (verify, V, x, π) to Sim. Upon receiving (witness, w) from Sim store (x, w, π) .
- Send (verification, x, π, b) to V where (x, w, π) is stored and $b = 1$ iff $(x; w) \in \mathcal{R}_{\text{hss}}$.

Figure 12: NIZK Functionality

Theorem 15. Let HSS be an m -server public-key HSS scheme with semi-malicious correctness for a class of circuits $\mathbb{C}_{\text{hss}}$ over a finite commutative ring R and let SNARG be a $\mathbb{C}_{\text{hss}}$ -simple Splittable zkSNARG for $\mathbb{C}$ -circuit satisfiability such that the following holds:

- $\mathbb{C} \subseteq \mathbb{C}_{\text{hss}}$,
- For every ℓ -input circuit $C \in \mathbb{C}_{\text{hss}}$, for every integer-valued polynomial $n(\cdot)$, and every linear function $f : R^{n(\ell)} \rightarrow R^\ell$, the circuit $\hat{C}$ computing $\hat{C}(\mathbf{x}) = C(f(\mathbf{x}))$ for $\mathbf{x} \in R^{n(\ell)}$ is in $\mathbb{C}_{\text{hss}}$, and
- For every ℓ -input circuit $C \in \mathbb{C}$ and for every integer-valued polynomial $n(\cdot)$, the circuit $\hat{C}$ computing $\hat{C}(\mathbf{x}, K_1, \dots, K_{n(\ell)}) = (K_1 \cdot C(\mathbf{x}), \dots, K_{n(\ell)} \cdot C(\mathbf{x}))$ for $\mathbf{x} \in R^\ell$ and $K_1, \dots, K_{n(\ell)} \in R$ is in $\mathbb{C}_{\text{hss}}$.

Then, for any integer-valued polynomial $n := n(\lambda)$, there exists an n -client, m -server MveHSS scheme for $\mathbb{C}$ with strong privacy (Figures 13 and 14) in the $\mathcal{F}_{\text{NIZK}}$ -hybrid model that is black-box in HSS and the linear phase of SNARG.

Proof. We first show that the construction has strong privacy and then show that it satisfies the required soundness and efficiency properties. To ensure privacy, we assume that computation is performed over a ring R such that the smallest prime divisor p of its order $|R|$ is such that $1/p$ is negligible in the security parameter. If this is not the case, modifying the construction to repeat the check with k independently sampled MAC keys, such that $1/p^k$ is negligible, suffices to ensure soundness. Since $k \leq \lambda$ suffices for any ring R , this only results in a $\text{poly}(\lambda)$ increase in each input and output share size.

Privacy. Intuitively, privacy of the honest clients' inputs follow from the security of HSS, the zero knowledge property of the proofs computed using $\mathcal{F}_{\text{NIZK}}$, and the zero-knowledge property of the Splittable zkSNARG. Correctness of the scheme follows from the soundness of proofs computed using $\mathcal{F}_{\text{NIZK}}$, correctness of HSS and the zkSNARG, and the soundness of the MAC check. An adversary does not gain any advantage by deviating from the protocol and learning if local verification was successful for honest clients due to the DP-robust verification property of the zkSNARG.

We next formally prove that the construction is secure against any polynomial size adversary $\mathcal{A}$ by describing a simulator Sim for $\mathcal{A}$ and arguing indistinguishability of $\mathcal{A}$'s view and the honest clients' outputs in the real and ideal worlds. Without loss of generality, we consider an adversary that corrupts all but one server and denote the index of the honest server by i^* .

Simulator:

Multi-Client HSS with Verifiable Evaluation

Public parameters. Let HSS be an m -server public-key HSS scheme that supports evaluating circuits over a finite commutative ring R and let SNARG be a Splittable zkSNARG. Let ℓ_{rand} denote the number of random elements from R required by the zkSNARG prover.

Notation. For a circuit C , let C_{pub} and C_i denote the circuit satisfiability instances for the public proof and the i -th client's proof, respectively i.e., when $y = C(x_{\text{pub}}, x_1, \dots, x_n)$ then $(C_{\text{pub}}, x_{\text{pub}}, y) \in \text{SAT}$ and $(C_i, x_{\text{pub}} \| x_i, y) \in \text{SAT}$. Let $C_{\text{NL}}^{\text{pub}}$ and C_{NL}^i denote the circuits that compute the non-linear phase Prove_{NL} of the Splittable zkSNARG for C_{pub} and C_i , respectively. Let $\hat{C}_{\text{NL}}$ denote the circuit that evaluates C_{NL} using randomness derived from the sum of the last n inputs. For a circuit C such that $y = C(x_{\text{pub}}, x_1, \dots, x_n)$, let $\hat{C}$ denote the augmented circuit such that $(y, \Gamma_y^1, \dots, \Gamma_y^n) = \hat{C}(x_{\text{pub}}, x_1, \dots, x_n, K_1, \dots, K_n)$, where $\Gamma_y^i = y \cdot K_i$ for each $i \in [1, n]$.

MveHSS.Setup($1^\lambda, C$)

```

1 :  $\text{crs}_{\text{pub}} \leftarrow \text{SNARG.Setup}(1^\lambda, C_{\text{pub}})$ 
2 : for  $i \in [1, n]$ 
3 :    $\text{crs}_i \leftarrow \text{SNARG.Setup}(1^\lambda, C_i)$ 
4 :  $\text{crs} := (\text{crs}_{\text{pub}}, \text{crs}_1, \dots, \text{crs}_n, C)$ 
5 : return  $\text{crs}$ 

```

MveHSS.KeyGen($1^\lambda, C$)

```

1 :  $(\text{pk}, \text{ek}_1, \dots, \text{ek}_m) \leftarrow \text{HSS.KeyGen}(1^\lambda)$ 
2 : return  $(\text{pk}, \text{ek}_1, \dots, \text{ek}_m)$ 

```

MveHSS.Enc($\text{crs}, i, \text{pk}, x$)

```

1 :  $K \leftarrow R$ 
2 :  $r_{\text{pub}}, r_1, \dots, r_n \leftarrow R^{\ell_{\text{rand}}}$ 
3 : foreach  $y \in \{x, K, r_{\text{pub}}, r_1, \dots, r_n\}$ 
4 :    $\rho \leftarrow \{0, 1\}^\lambda$ 
5 :    $\text{ct}_y \leftarrow \text{HSS.Enc}(\text{pk}, y; \rho)$ 
6 :   Send (prove,  $(\text{pk}, R, \text{ct}_y), (y, \rho)$ ) to
        $\mathcal{F}_{\text{NIZK}}$  and receive (proof,  $\pi_y^{\text{inp}}$ )
7 :  $\tilde{x} := ((\text{ct}_x, \pi_x^{\text{inp}}), (\text{ct}_K, \pi_K^{\text{inp}}), (\text{ct}_{r_{\text{pub}}}, \pi_{r_{\text{pub}}}^{\text{inp}}),$ 
        $(\text{ct}_{r_1}, \pi_{r_1}^{\text{inp}}), \dots, (\text{ct}_{r_n}, \pi_{r_n}^{\text{inp}}))$ 
8 :  $\text{st} := K$ 
9 : return  $(\tilde{x}, \text{st})$ 

```

MveHSS.Eval($\text{crs}, \text{ek}_i, x_{\text{pub}}, \tilde{x}_1, \dots, \tilde{x}_n$)

```

1 : parse  $\text{crs} = (\text{crs}_{\text{pub}}, \text{crs}_1, \dots, \text{crs}_n, C)$ 
2 : parse each  $\tilde{x}_i = ((\text{ct}_{x_i}, \pi_{x_i}^{\text{inp}}), (\text{ct}_{K_i}, \pi_{K_i}^{\text{inp}}),$ 
        $(\text{ct}_{r_{\text{pub},i}}, \pi_{r_{\text{pub},i}}^{\text{inp}}), (\text{ct}_{r_{1,i}}, \pi_{r_{1,i}}^{\text{inp}}), \dots, (\text{ct}_{r_{n,i}}, \pi_{r_{n,i}}^{\text{inp}}))$ 
3 : Verify each  $(\text{ct}_y, \pi_y^{\text{inp}})$  using  $\mathcal{F}_{\text{NIZK}}$  and abort
       if verification fails
4 :  $([y]_i, [\Gamma_y^1]_i, \dots, [\Gamma_y^n]_i) :=$ 
        $\text{HSS.Eval}(\text{ek}_i, \hat{C}, x_{\text{pub}}, \text{ct}_{x_1}, \dots, \text{ct}_{x_n}, \text{ct}_{K_1}, \dots, \text{ct}_{K_n})$ 
5 :  $[M_{\text{pub}}]_i := \text{HSS.Eval}(\text{ek}_i, \hat{C}_{\text{NL}}^{\text{pub}}, x_{\text{pub}}, \text{ct}_{x_1}, \dots, \text{ct}_{x_n},$ 
        $\text{ct}_{r_{\text{pub},1}}, \dots, \text{ct}_{r_{\text{pub},n}})$ 
6 :  $[\pi_{\text{pub}}]_i := \text{SNARG.Prove}_L(\text{crs}_{\text{pub}}, [M_{\text{pub}}]_i)$ 
7 :  $[\text{out}_{\text{pub}}]_i := ([y]_i, [\pi_{\text{pub}}]_i)$ 
8 : for  $j \in [1, n]$ 
9 :    $[M_j]_i := \text{HSS.Eval}(\text{ek}_i, \hat{C}_{\text{NL}}^j, x_{\text{pub}}, \text{ct}_{x_1}, \dots, \text{ct}_{x_n},$ 
        $\text{ct}_{r_{j,1}}, \dots, \text{ct}_{r_{j,n}})$ 
10 :  $[\pi_j^{\text{loc}}]_i := \text{SNARG.Prove}_L(\text{crs}_j, [M_j]_i)$ 
11 :  $[\text{out}_j]_i := ([\Gamma_y^j]_i, [\pi_j^{\text{loc}}]_i)$ 
12 : return  $([\text{out}_{\text{pub}}]_i, [\text{out}_1]_i, \dots, [\text{out}_n]_i)$ 

```

Figure 13: Multi-client HSS with verifiable evaluation and strong privacy from HSS and Splittable zk-SNARG. The reconstruction and verification algorithms are described in Figure 14.

Multi-Client HSS with Verifiable Evaluation (continued)

MveHSS.Recon(crs, [out]₁, ..., [out]_m)

```

1 : parse each [out]i = ([z]i, [π]i)
2 :  $\mathbf{z} := \sum_{i=1}^m [\mathbf{z}]_i$ 
3 :  $\pi := \sum_{i=1}^m [\pi]_i$ 
4 : return (z, π)

```

MveHSS.LVerify(crs, x_{pub}, x_i, y, st_i, π_i)

```

1 : parse crs = (crspub, crs1, ..., crsn)
2 : parse sti = Ki
3 : parse πi = (Γyi, πiloc)
4 : if Ki · y ≠ Γyi ∨
    SNARG.Verify(crsi, xpub || xi, y, πiloc) ≠ 1
5 : return 0
6 : else return 1

```

MveHSS.PVerify(crs, x_{pub}, y, π_{pub})

```

1 : parse crs = (crspub, crs1, ..., crsn)
2 : return SNARG.Verify(crspub, xpub, y, πpub)

```

Figure 14: Reconstruction and verification algorithms for multi-client HSS with verifiable evaluation and strong privacy. The remaining algorithms are described in Figure 13.

1. It receives the circuit C and the public input $\mathbf{x}_{\text{pub}}$ from $\mathcal{F}_{\text{mpc}}$. If all clients are corrupt, it computes $\text{crs} \leftarrow \text{MveHSS.Setup}(1^\lambda, C)$. Else, it simulates the CRS for the public proof crs_{pub} and the CRS for the local proofs of corrupt clients $\{\text{crs}_i\}_{i \in \mathcal{I}_c}$ using the zkSNARG simulator, and computes the CRS for the local proofs of honest clients $\{\text{crs}_i\}_{i \notin \mathcal{I}_c}$ honestly, using SNARG.Setup . It sends $\text{crs} = (\text{crs}_{\text{pub}}, \text{crs}_1, \dots, \text{crs}_n)$ to $\mathcal{A}$ on behalf of $\mathcal{F}_{\text{crs}}$.
2. It computes $(\text{pk}, \text{ek}_1, \dots, \text{ek}_m) \leftarrow \text{HSS.KeyGen}(1^\lambda)$ and sends the public-key pk and the evaluation keys of corrupt servers $\{\text{ek}_i\}_{i \in \mathcal{I}}$ to $\mathcal{A}$ on behalf of $\mathcal{F}_{\text{pkgen}}$. It receives status from $\mathcal{A}$. If status = abort then it sends $\mathcal{S} = [1, n] \setminus \mathcal{I}_c$ to $\mathcal{F}_{\text{mpc}}$ and halts.
3. It simulates the inputs of honest clients for each $i \in [1, n] \setminus \mathcal{I}_c$ by computing each HSS ciphertext in $\tilde{\mathbf{x}}_i$ as an encryption of 0. It sends the corresponding statements under $\mathcal{R}_{\text{hss}}$ to $\mathcal{A}$ on behalf of $\mathcal{F}_{\text{NIZK}}$ and receives the NIZK proof of well-formedness for each ciphertext. It returns 1 on behalf of $\mathcal{F}_{\text{NIZK}}$ on subsequent verification queries on these proofs. It then sends $\{\tilde{\mathbf{x}}_i\}_{i \notin \mathcal{I}_c}$, consisting of the encryptions of 0 and the NIZK proofs, to $\mathcal{A}$.
4. For each $i \in \mathcal{I}_c$, it obtains $\{\tilde{\mathbf{x}}_i\}_{i \in \mathcal{I}_c}$ broadcast by $\mathcal{A}$. On behalf of each honest server, it emulates a verification query to $\mathcal{F}_{\text{NIZK}}$ (where the simulator runs the steps of $\mathcal{F}_{\text{NIZK}}$ honestly) for every NIZK proof sent by corrupt clients. If there exists a proof that does not verify, it sends $\mathcal{S} = [1, n] \setminus \mathcal{I}_c$ to $\mathcal{F}_{\text{mpc}}$ and halts. At this point, the simulator obtains the witness $(\mathbf{x}_i, K_i, \mathbf{r}_{\text{pub}, i}, \mathbf{r}_{1, i}, \dots, \mathbf{r}_{n, i})$ for each corrupt client $i \in \mathcal{I}_c$, either by receiving it from corrupt clients during the proof phase or from $\mathcal{A}$ during the verification phase.
5. It computes the output share of the i -th corrupt server, for each $i \in \mathcal{I}$, as

$$\left([\mathbf{y}]_i, [\pi_{\text{pub}}]_i, \{[\pi_j]_i\}_{j=1}^n, \left\{ \left[\Gamma_y^j \right]_i \right\}_{j=1}^n \right) \leftarrow \text{Eval}(\text{crs}, \text{ek}_i, \mathbf{x}_{\text{pub}}, \tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_n).$$

6. It sends $\{\mathbf{x}_i\}_{i \in \mathcal{I}_c}$ to $\mathcal{F}_{\text{mpc}}$ and obtains the output $\mathbf{y}$. For every corrupt client $i \in \mathcal{I}_c$, it computes $\Gamma_y^i = \mathbf{y} \cdot K_i$. If all clients are corrupt, it computes

$$M_{\text{pub}} := \text{SNARG.Prove}_{\text{NL}}(\text{crs}_{\text{pub}}, \mathbf{C}, \mathbf{x}_{\text{pub}}, \mathbf{y}, (\mathbf{x}_1, \dots, \mathbf{x}_n); \mathbf{r}_{\text{pub},1} + \dots + \mathbf{r}_{\text{pub},n})$$

$$\pi_{\text{pub}} := \text{SNARG.Prove}_{\text{L}}(\text{crs}, M_{\text{pub}}).$$

Similarly, it computes $\{\pi_i^{\text{loc}}\}_{i=1}^n$. Else, if at least one client is honest, it simulates π_{pub} and $\{\pi_i^{\text{loc}}\}_{i \in \mathcal{I}}$ using the zkSNARG simulator.

7. It reverse samples the output share of the honest server $[\mathbf{y}]_{i^*}$, $[\pi_{\text{pub}}]_{i^*}$, $[\Gamma_y^i]_{i^*}$, and $[\pi_i^{\text{loc}}]_{i^*}$. It then sends the shares of honest servers in $[\text{out}_{\text{pub}}]$ and $\{[\text{out}_i]\}_{i \in \mathcal{I}_c}$ to $\mathcal{A}$ and receives $[\overline{\text{out}_i}]_{\mathcal{I}} = \left([\overline{\Gamma_y^i}]_{\mathcal{I}_c}, [\overline{\pi_i^{\text{loc}}}]_{\mathcal{I}} \right)$ and $[\overline{\text{out}_{\text{pub}}^i}]_{\mathcal{I}} = \left([\overline{y_i}]_{\mathcal{I}}, [\overline{\pi_{\text{pub}}^i}]_{\mathcal{I}} \right)$ for each $i \notin \mathcal{I}_c$ from $\mathcal{A}$ (out_{pub} is indexed by i because the output shares sent to each client might be different).

8. For every honest client $i \in [1, n] \setminus \mathcal{I}_c$ it does the following.

- It computes $[\epsilon_i]_{\mathcal{I}} = [\overline{y_i}]_{\mathcal{I}} - [\mathbf{y}]_{\mathcal{I}}$ and sets $[\epsilon_i]_j = 0$ for every $j \notin \mathcal{I}$. It then computes $\epsilon_i := \sum_{j=1}^m [\epsilon_i]_j$.
- It computes $[\delta_i]_{\mathcal{I}} = [\overline{\Gamma_y^i}]_{\mathcal{I}} - [\Gamma_y^i]_{\mathcal{I}}$ and sets $[\delta_i]_j = 0$ for every $j \notin \mathcal{I}$. It then computes $\delta_i := \sum_{j=1}^m [\delta_i]_j$.
- It computes $[\eta_i]_{\mathcal{I}} = [\overline{\pi_i^{\text{loc}}}]_{\mathcal{I}} - [\pi_i^{\text{loc}}]_{\mathcal{I}}$ and sets $[\eta_i]_j = 0$ for every $j \notin \mathcal{I}$. It then computes $\eta_i := \sum_{j=1}^m [\eta_i]_j$.

It then computes $\mathcal{S}$ as the set of all indices $i \in [1, n] \setminus \mathcal{I}_c$ such that at least one among ϵ_i , δ_i or η_i is non-zero. It sends $\mathcal{S}$ to $\mathcal{F}_{\text{mpc}}$ and outputs the value returned by $\mathcal{A}$.

We show using a hybrid argument that for any $\mathbf{C} \in \mathbb{C}$, $\text{Real}_{\mathcal{A}(z)}^{\mathcal{I}, \mathcal{I}_c}$ is indistinguishable from $\text{Ideal}_{\text{Sim}(z)}^{\mathcal{I}, \mathcal{I}_c}$.

- Hyb_0 : This corresponds to $\text{Real}_{\mathcal{A}}(1^\lambda, \mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n)$ i.e., the simulator follows the steps of $\mathcal{F}_{\text{mpc}}$ using the honest parties' inputs.
- Hyb_1 : In this hybrid, the simulator computes output shares of honest servers as follows.
 - On behalf of $\mathcal{F}_{\text{NIZK}}$, it learns the input shared by corrupt clients, as described in step 4 above. It then computes $\mathbf{y}$, $\{\Gamma_y^i\}_{i=1}^n$, π_{pub} and $\{\pi_i^{\text{loc}}\}_{i=1}^n$ in the clear by evaluating the corresponding circuits on client inputs.
 - It recomputes the output shares of corrupt servers as described in step 5 above.
 - Finally, it reverse samples the shares of the honest server in $[\mathbf{y}]$, $\{\Gamma_y^i\}_{i=1}^n$, $[\pi_{\text{pub}}]$, and $\{\pi_i^{\text{loc}}\}_{i=1}^n$ using the shares of the corrupt servers and the corresponding secrets.

In Hyb_0 , the honest servers' shares of the output were computed by evaluating their HSS shares of the inputs while in this hybrid they are reverse sampled from the actual outputs and the corrupt servers' shares of the output. Observe that if the ciphertexts sent by corrupt clients are not well-formed then the previous hybrid is identical to this hybrid since the honest servers abort in both cases. Conversely, the honest servers do not abort only when all NIZK proofs successfully verify, in which case it follows from the definition of $\mathcal{F}_{\text{NIZK}}$ that all input ciphertexts are well-formed. From the properties of $\mathbb{C}_{\text{hss}}$ and

$\mathbb{C}$ stated in the theorem statement, it follows that for each $C \in \mathbb{C}_{\text{ZK}}, \widehat{C}, \hat{C}_{\text{NL}}^{\text{pub}}, \hat{C}_{\text{NL}}^1, \dots, \hat{C}_{\text{NL}}^n \in \mathbb{C}_{\text{hss}}$. This combined with the semi-malicious correctness of HSS and the linearity of additive secret sharing implies that $\text{Hyb}_0 \approx \text{Hyb}_1$.

- Hyb_2 : In this hybrid, the shares of honest servers in $[y]$ are computed using the output y returned by $\mathcal{F}_{\text{mpc}}$ and the output shares of corrupt servers computed by the simulator.

Observe that honest clients and the simulator send the same inputs to $\mathcal{F}_{\text{mpc}}$ as the ones used to compute y in Hyb_1 . It thus follows that $\text{Hyb}_1 \stackrel{\text{D}}{=} \text{Hyb}_2$.

- Hyb_3 : In this hybrid, the simulator simulates HSS inputs of honest clients as described in step 3 above.

Observe that the verification of the honest clients' inputs using $\mathcal{F}_{\text{NIZK}}$ is always successful in both hybrids. Thus, the main difference between Hyb_2 and Hyb_3 is that in Hyb_2 the simulator used the honest clients' inputs to compute the ciphertexts sent to the adversary while in this hybrid the ciphertexts correspond to encryptions of 0. The security of HSS then implies that $\text{Hyb}_2 \approx \text{Hyb}_3$ from a standard hybrid argument.

- Hyb_4 : In this hybrid, the simulator computes $\text{crs}_{\text{pub}}, \pi_{\text{pub}}, \{\text{crs}_i, \pi_i^{\text{loc}}\}_{i \in \mathcal{I}_c}$ as described in steps 1 and 6 above. It then uses these proofs to compute the output shares of honest servers as described in step 7.

If all clients are corrupt then Hyb_3 is identical to Hyb_4 . The only difference compared to the previous hybrid is that when at least one client is honest, the public proof π_{pub} and the local proofs of corrupt clients $\{\pi_i^{\text{loc}}\}_{i \in \mathcal{I}_c}$ are simulated using the zkSNARG simulator. Observe that the randomness used to compute these proofs corresponds to the sum of random inputs shared by each client. Thus, when at least one of the clients is honest, the randomness used to compute the proofs is uniformly distributed. Since $\mathcal{A}$'s view in Hyb_3 is independent of the random inputs shared by honest clients, it can be shown using a standard hybrid argument that $\text{Hyb}_3 \approx \text{Hyb}_4$ from the zero-knowledge property of the zkSNARG.

- Hyb_5 : In this hybrid, the output of all honest clients P_i^c such that $\epsilon_i \neq 0$ or $\delta_i \neq 0$ is set to $\perp$ where ϵ_i and δ_i are as computed by the simulator in step 8.

Observe that

$$\overline{y}_i = y + \sum_{j=1}^n [\overline{y}_i]_j - [y]_j = y + \epsilon_i.$$

Similarly, we have $\overline{\Gamma}_y^i = \Gamma_y^i + \delta_i$. The difference between Hyb_4 and Hyb_5 is that in Hyb_4 the output of honest clients was $\perp$ if $\overline{y}_i \cdot K_i \neq \overline{\Gamma}_y^i$ while in Hyb_5 it is set based on ϵ_i and δ_i . We show that the MAC check fails with overwhelming probability in Hyb_4 when $\epsilon_i \neq 0$ or $\delta_i \neq 0$.

Observe that if $\overline{y}_i \cdot K_i = \overline{\Gamma}_y^i$ then

$$\begin{aligned} \overline{y}_i \cdot K_i &= \overline{\Gamma}_y^i \\ \implies (y + \epsilon_i) \cdot K_i &= \Gamma_y^i + \delta_i \\ \implies \epsilon_i \cdot K_i &= \delta_i \end{aligned}$$

where the second implication follows from the fact that $\Gamma_y^i = y \cdot K_i$ by definition of $\widehat{C}$. Note that K_i is uniformly random over R for honest clients and $\mathcal{A}$'s view is independent of K_i . Thus, when $\epsilon_i \neq 0$ or $\delta_i \neq 0$, it follows that $\epsilon_i \cdot K_i = \delta_i$ with probability at most $1/p$, where p is the smallest prime divisor of $|R|$. This implies that $\overline{y}_i \cdot K_i = \overline{\Gamma}_y^i$ with probability at most $1/p$, which is negligible. Thus, $\text{Hyb}_4 \approx \text{Hyb}_5$.

- Hyb_6 : In this hybrid, the output of all honest clients P_i^c such that $\eta_i \neq 0$ is set to $\perp$ where η_i is as computed by the simulator in step 8.

As in the previous case, we have $\overline{\pi_i^{\text{loc}}} = \pi_i^{\text{loc}} + \eta_i$ for each $i \notin \mathcal{I}_c$. If the output of any honest client P_i^c is $\perp$ in this hybrid but not $\perp$ in the previous hybrid then $\overline{y_i}$ reconstructed by P_i^c is equal to y but the reconstructed local proof $\overline{\pi_i^{\text{loc}}}$ is not equal to π_i^{loc} . Observe that the randomness used to compute the local proof of each honest client is uniformly distributed, since this randomness corresponds to the sum of random inputs shared by all clients. Since the adversary's view is independent of the random inputs shared by honest clients, the DP-robust verification property of SNARG (Lemma 7) implies that the output of P_i^c in Hyb_5 is $\perp$ with overwhelming probability when $\eta_i \neq 0$. It can then be shown using a standard hybrid argument that $\text{Hyb}_5 \approx \text{Hyb}_6$.

- Hyb_7 : In this hybrid, the outputs of honest clients is as received from $\mathcal{F}_{\text{mpc}}$.

It is easy to see that if $\mathcal{F}_{\text{pkgen}}$ sends $\perp$ or verification of corrupt clients' inputs using $\mathcal{F}_{\text{NIZK}}$ fails then honest clients abort in Hyb_6 and Hyb_7 . Moreover, if one among ϵ_i , δ_i or η_i is non-zero for a client P_i^c then $\mathcal{F}_{\text{mpc}}$ sends $\perp$ to P_i^c since the simulator adds i to $\mathcal{S}$. Note that in this case, the output of honest clients is identical to that in the previous hybrid. Conversely, when $\epsilon_i = 0$, $\delta_i = 0$ and $\eta_i = 0$, the correctness of the MAC check and the DP-completeness property (Lemma 6) imply that $\text{Hyb}_6 \approx \text{Hyb}_7$.

Hyb_7 corresponds to $\text{Ideal}_{\text{Sim}(z)}^{\mathcal{I}, \mathcal{I}_c}$ which in turn implies that

$$\left\{ \text{Real}_{\mathcal{A}(z)}^{\mathcal{I}, \mathcal{I}_c}(1^\lambda, z, \mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n) \right\} \approx \left\{ \text{Ideal}_{\text{Sim}(z)}^{\mathcal{I}, \mathcal{I}_c}(1^\lambda, z, \mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n) \right\}$$

where the ensembles are parameterized by $\lambda \in \mathbb{N}$, circuit $C \in \mathbb{C}$, an integer-valued polynomial n , inputs $\mathbf{x}_{\text{pub}}, \mathbf{x}_1, \dots, \mathbf{x}_n \in R^*$, sets $\mathcal{I} \subset [1, m]$ and $\mathcal{I}_c \subseteq [1, n]$, and auxiliary information $z \in \{0, 1\}^*$.

Local soundness and public soundness of the construction follow immediately from the soundness of the zkSNARG. The construction has succinct share size and succinct verification since HSS is compact and SNARG has succinct proof size and verification. It then follows that the construction described in Figures 13 and 14 is an n -client, m -server MveHSS scheme for $\mathbb{C}$ with strong privacy. ■

6.2 Simpler Construction Without Strong Privacy

Our MveHSS construction can be simplified when strong privacy is not required. Specifically, we no longer need the $\mathcal{F}_{\text{NIZK}}$ functionality and semi-malicious correctness of the public-key HSS scheme. In more detail, $\mathcal{F}_{\text{NIZK}}$ serves two purposes in the previous construction: firstly, to ensure well-formedness of the ciphertexts sent by corrupt clients and secondly, it allows the simulator to extract the inputs of corrupt clients. However, if clients are only semi-honestly corrupt, then the well-formedness of ciphertexts is guaranteed since all clients follow the protocol and the simulator no longer needs to extract inputs. Thus, in this case, the construction no longer requires $\mathcal{F}_{\text{NIZK}}$, and the Enc algorithm only requires clients to encrypt their inputs, MAC keys and their contribution to the randomness used for computing the Splittable zk-SNARG. Moreover, when clients are semi-honest the randomness used for this encryption is uniformly distributed which in turn implies that the standard correctness of the public-key HSS scheme (as defined in Definition 2) suffices, as opposed to the stronger semi-malicious correctness.

Theorem 16. *Let HSS be an m -server public-key HSS scheme for a class of circuits $\mathbb{C}_{\text{hss}}$ over a finite commutative ring R and let SNARG be a $\mathbb{C}_{\text{hss}}$ -simple Splittable zkSNARG for $\mathbb{C}$ -circuit satisfiability such that $\mathbb{C}_{\text{hss}}$ and $\mathbb{C}$ satisfy the same requirements as in Theorem 15. Then, for any integer-valued polynomial $n := n(\lambda)$, there exists an n -client, m -server MveHSS scheme for $\mathbb{C}$ that is black-box in HSS and the linear phase of SNARG.*

6.3 Instantiations

We now discuss instantiating the MveHSS template described in Figures 13 and 14 using the HSS schemes from prior works and the Splittable zkSNARGs discussed in Section 4.

Let $\text{VP}[p]$ denote the set of polynomial size circuits over $\mathbb{Z}_p$ and let $\text{VNC}^1[p]$ denote the set of polynomial size circuits over $\mathbb{Z}_p$ that have logarithmic depth in the number of inputs. As discussed in Section 5.1, we consider two classes of public-key HSS schemes. The first group consists of 2-server public-key HSS schemes from group based assumptions [BKS19, OSY21, RS21, ADOS22] that can be used to evaluate circuits in $\text{VNC}^1[p]$, where $p \in \mathcal{O}(2^\lambda)$. The second group consists of m -server public-key HSS schemes [BGI15, COS⁺22], where m is polynomial in the security parameter, that can be used to evaluate circuits in $\text{VP}[p]$, for $p \in \mathcal{O}(2^\lambda)$. Note that both groups satisfy the preconditions for Theorem 15 since $\text{VNC}^1[p]$ and $\text{VP}[p]$ are closed under linear transformations on the input and computing MACs on the output.

Groth16 based MveHSS. As discussed in Section 5.1, Theorem 9 implies that Groth16 is a $\text{VNC}^1[p]$ -simple SplittablezkSNARG for NC^1 -circuit satisfiability and that it is a $\text{VP}[p]$ -simple Splittable zkSNARG for P/poly circuit satisfiability. This gives the following corollary of Theorem 15.

Corollary 17. *There exist the following MveHSS schemes with local and public knowledge soundness in the generic bilinear group model:*

- A 2-server MveHSS scheme for NC^1 assuming the hardness of the LWE assumption,
- A 2-server MveHSS scheme for NC^1 assuming the hardness of the DCR assumption,
- A 2-server MveHSS scheme for NC^1 assuming the hardness of the DDH assumption and the small exponent assumption over class groups,
- An m -server MveHSS scheme for P/poly, for any integer-valued polynomial $m(\cdot)$, assuming the existence of circular secure AFS-spooky encryption.

Waters-Wu zkBARG based MveHSS. The following corollary of Theorem 15 follows from Theorem 11, which implies that the Waters-Wu zkBARG has non-adaptive soundness and is a $\text{VNC}^1[N]$ -simple zkSNARG for SIMD NC^1 -circuit satisfiability and is a $\text{VP}[N]$ -simple zkSNARG for SIMD P/poly-circuit satisfiability, where N is the order of the bilinear groups used for the zkBARG. Interestingly, unlike in the case of single client ve-HSS where non-adaptive soundness implied non-adaptive privacy, the privacy of the multi-client scheme is not affected by the soundness of the zkBARG. This is because the single client ve-HSS construction relied on the soundness of the zkBARG for ensuring correctness of the output while the multi-client construction uses MACs.

Corollary 18. *There exist the following MveHSS schemes with non-adaptive local and public soundness all of which assume the hardness of the subgroup decision assumption over composite-order bilinear groups:*

- A 2-server MveHSS scheme for SIMD NC^1 additionally assuming the hardness of the LWE assumption,
- A 2-server MveHSS scheme for SIMD NC^1 additionally assuming the hardness of the DCR assumption,
- A 2-server MveHSS scheme for SIMD NC^1 additionally assuming the hardness of the DDH assumption and the small exponent assumption over class groups,
- An m -server MveHSS scheme for SIMD P/poly, for any integer-valued polynomial $m(\cdot)$, additionally assuming the existence of circular secure AFS-spooky encryption.

MveHSS with strong privacy. Instantiating an MveHSS scheme with strong privacy additionally requires a public-key HSS scheme with semi-malicious correctness and a primitive that realizes $\mathcal{F}_{\text{NIZK}}$. As discussed in [RS21, ADOS22], semi-malicious correctness can be achieved for most 2-server public key HSS schemes in the random oracle model. On the other hand, to realize $\mathcal{F}_{\text{NIZK}}$ while making only black-box use of cryptography, we use the NIZK scheme proposed by Abram et al. [ADOS22]. Specifically, they describe a 2-server public-key HSS scheme with semi-malicious correctness and a NIZK scheme that realizes $\mathcal{F}_{\text{NIZK}}$ in the random oracle model assuming the hardness of the Decisional Diffie-Hellman (DDH) assumption, Decisional Cross-group Diffie-Hellman (DXDH) assumption, weak hidden order assumption and small exponent assumption over class groups. The HSS and NIZK schemes are black-box in the group. Furthermore, [ADOS22] also describe a one-round protocol realizing $\mathcal{F}_{\text{pkey}}$ that is black-box in the class group. The following theorem then follows from the HSS and NIZK schemes of [ADOS22] and Theorems 9, 11 and 15.

Corollary 19. *Assuming the hardness of DDH, DXDH, the weak hidden order assumption and small exponent assumption over class groups, there exist*

- 2-server MveHSS scheme with strong privacy for NC^1 in the generic bilinear group and random oracle model with local and public knowledge soundness that is black-box in the class group,
- 2-server MveHSS scheme with strong privacy for SIMD NC^1 with non-adaptive local and public soundness in the random oracle model, additionally assuming the hardness of the subgroup decision assumption over composite order bilinear groups, that is black-box in the class group and the composite order bilinear group.

Additionally, both MveHSS schemes above have a one-round protocol that realizes $\mathcal{F}_{\text{pkey}}$ assuming the servers have a common reference string.

7 Applications

In this section we use HSS with verifiable evaluation for private delegation of computation and for collaborative computation of zkSNARKs.

7.1 Private Delegation of Computation

Private delegation of computation [GGP10] allows a computationally weak client to outsource the computation of a program on private inputs to one or more servers such that the servers do not learn any information about the client’s inputs. The servers return the program output and prove the correctness of the returned output to the client. Verification of the proof should require significantly lower computational effort on behalf of the client compared to evaluating the program from scratch. Moreover, the client should obtain a publicly verifiable proof attesting that the program, when evaluated on some private inputs, produces the claimed output.

A ve-HSS scheme provides a natural approach for private delegation of computation as shown in Figure 15. Specifically, given a common reference string computed using a setup process, the client can simply share its private inputs among the servers, who then compute their share of the output and reconstruct it back towards the client. The client then learns the function output as well as a proof of correctness for the output that can be non-interactively verified. The ve-HSS scheme also provides the client with a publicly verifiable non-interactive proof that allows the client to prove that the output is consistent with a public part of the program input.

From Corollaries 13 and 14, we have private delegation of computation schemes for NC^1 , SIMD NC^1 , P/poly , and SIMD P/poly from a wide range of assumptions that ensure privacy as long as at least one

Protocol $\Pi_{\text{pdc}}^{\text{ve-HSS}}$

The protocol runs with a client P^c and m servers $\{P_1, \dots, P_m\}$ where the client wishes to evaluate a circuit C on public input $\mathbf{x}_{\text{pub}}$ and private input $\mathbf{x}$. Let ve-HSS be an m -server ve-HSS scheme for a class of circuits $\mathbb{C}$ such that $C \in \mathbb{C}$. The protocol runs in the presence of an adversary $\mathcal{A}$ that corrupts a subset of servers $\mathcal{I} \subset [1, m]$.

Setup: Output $\text{crs} \leftarrow \text{ve-HSS.Setup}(1^\lambda, C)$.

Input Phase: The client P^c computes $\langle \mathbf{x} \rangle \leftarrow \text{ve-HSS.Share}(\text{crs}, \mathbf{x})$ and sends $(\mathbf{x}_{\text{pub}}, \langle \mathbf{x} \rangle_i)$ to the i -th server P_i for each $i \in [1, m]$.

Evaluation Phase: Each server P_i computes $[\text{out}]_i := \text{ve-HSS.Eval}(\text{crs}, \mathbf{x}_{\text{pub}}, \langle \mathbf{x} \rangle_i)$ where $i \in [1, m]$.

Output Phase: Each server P_i sends $[\text{out}]_i$ to the client P^c . P^c then computes

$$(\mathbf{y}, \pi_{\text{loc}}, \pi_{\text{pub}}) := \text{ve-HSS.Recon}(\text{crs}, [\text{out}]_1, \dots, [\text{out}]_m).$$

It outputs $\perp$ if either $\text{ve-HSS.LVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{x}, \mathbf{y}, \pi_{\text{loc}}) = 0$ or $\text{ve-HSS.PVerify}(\text{crs}, \mathbf{x}_{\text{pub}}, \mathbf{y}, \pi_{\text{pub}}) = 0$, else it outputs $(\mathbf{x}_{\text{pub}}, \mathbf{y}, \pi_{\text{pub}})$.

Figure 15: Protocol for private delegation of computation using a ve-HSS scheme.

server is honest. An attractive feature of these protocols is that the client is guaranteed the correctness of the output even when all servers are corrupt. Next, we briefly discuss the security and efficiency properties of the protocol.

Security. Correctness of the ve-HSS scheme implies that the local and public proofs verify and that the client obtains the correct circuit output when servers follow the description of the protocol. Privacy of the scheme ensures that an adversary corrupting any subset of the servers, or one that learns the publicly verifiable proof and the circuit output does not gain any additional information about the private inputs of the client. Local soundness of the ve-HSS scheme guarantees the correctness of the circuit output when the local proof verifies successfully, even if an adversary corrupts all servers. Similarly, public soundness ensures the consistency of the circuit output with the public inputs when the public proof verifies successfully, irrespective of the behavior of the servers or the client.

Efficiency. Efficiency of the ve-HSS scheme ensures that the computational effort of the client required for the input phase and verification of output is sublinear in the size of the circuit being computed. Moreover, the resulting protocol is a 2 round protocol for verifiable computation.

7.2 Collaborative zkSNARKs

A zkSNARK for a relation $\mathcal{R}$ allows a prover holding a private witness $\mathbf{w}$ for a statement $\mathbf{x}$ to compute a publicly verifiable proof π that convinces a verifier that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$, while ensuring that π does not leak any information about $\mathbf{w}$. Collaborative zkSNARKs [OB22] generalize zkSNARKs from considering a single prover holding a witness $\mathbf{w}$ to the setting where the witness $\mathbf{w} = (w_1, \dots, w_n)$ is distributed among n provers, where the i -th prover holds w_i . The provers compute a publicly verifiable proof π to prove that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$ whilst ensuring that their witness remains private even when π is made public.

We define the ideal functionality for collaborative zkSNARKs in Figure 16. The ideal functionality returns the zkSNARK proof to provers only if they input a valid witness for the statement. If the witness is

Functionality $\mathcal{F}_{\text{czk}}^{\text{SNARK}}$

The functionality $\mathcal{F}_{\text{czk}}^{\text{SNARK}}$ is parameterized by a zkSNARK SNARK and proceeds as follows with n provers in the presence of an adversary Sim that corrupts a subset $\mathcal{I}_c \subset [1, n]$ of provers. Each prover has a private witness w_i and all provers have $(\mathbf{x}, \mathcal{R}, \text{crs})$ as common input, where $\mathbf{x}$ is the public statement, $\mathcal{R}$ is a relation, and crs is a CRS for SNARK.

- 1: It receives the CRS crs and a relation $\mathcal{R}$ from the honest provers and sends crs to the adversary Sim.
- 2: It receives the statement $\mathbf{x}$ from the honest provers and sends it to Sim.
- 3: It receives w_i for every $i \in [1, n] \setminus \mathcal{I}$ from the i -th prover and receives $\{w_i\}_{i \in \mathcal{I}}$ from Sim. Let $\mathbf{w} = (w_1, \dots, w_n)$.
- 4: If $(\mathbf{x}, \mathbf{w}) \notin \mathcal{R}$, it sends $\perp$ to the Sim and all provers. Else, it computes $\pi \leftarrow \text{SNARK.Prove}(\text{crs}, \mathbf{x}, \mathbf{w})$ and sends π to Sim.
- 5: It receives $\mathcal{S}$ from Sim. For every $i \in [1, n]$, it sends $\perp$ to the i -th client if $i \in \mathcal{S}$ else it sends $(\mathbf{x}, \pi)$ to it.

Figure 16: Ideal functionality for collaborative zkSNARKs.

not valid, the adversary does not learn any additional information about the witness of honest provers. On the other hand, when provers input a valid witness, the ideal functionality does not provide any additional information to the prover than what is leaked by the zkSNARK proof. However, since the proof is zero-knowledge, it preserves the privacy of its witness.

The protocol described in Figure 17 allows a set of provers, each holding a private witness for a public statement, to collaboratively compute a zkSNARK by delegating computation to a set of servers. The adversary can maliciously corrupt provers and a subset of servers. As in our MveHSS construction, the protocol uses an HSS scheme to compute shares of the output of the non-linear phase of the zkSNARK and then leverages linearity of the secret sharing scheme to compute shares of the proof. While the protocol uses the same approach as our MveHSS construction, the construction is simpler in this case since we no longer need to compute the local proofs. On the other hand, the protocol needs to achieve a slightly stronger security requirement where provers obtain the (public) proof only if they hold a valid witness. We achieve this conditional reconstruction of the proof by exploiting the split prover property of the Splittable zkSNARK. Specifically, servers compute an additive sharing of $y \cdot M_{\text{NL}}$ using HSS where $y = C_{\mathcal{R}}(\mathbf{x}, \mathbf{w})$ denotes the output of the relation circuit on the public statement and the witness input by provers. The servers then compute their share of the proof as $[y \cdot \pi] = \text{Prove}_L([y \cdot M_{\text{NL}}])$. However, before reconstructing the proof, the servers employ a MAC-check to ensure that the output y of the relation-checking circuit $C_{\mathcal{R}}$ was indeed computed using the witness shared by provers. If the MAC-check passes successfully, the servers publicly reconstruct the shares of the proof. As in the case of our MveHSS construction, we ensure that the randomness used for computing the proof is uniformly distributed, when at least one prover is honest, by deriving it using the randomness shared by each prover.

Observe that once the provers send their inputs to the servers, the protocol allows servers to publicly reconstruct the proof, without requiring any further involvement from the provers. This can serve as a useful property in practical applications where provers can share their witness to delegate computation of the zkSNARK proof and then go offline. The servers can put up their shares of the proof on a public bulletin board to allow any external verifier to reconstruct and verify the validity of the statement.

Theorem 20. *Let HSS be an m -server public-key HSS scheme for $\mathbb{C}_{\text{hss}}$ with semi-malicious correctness and let SNARK be a $\mathbb{C}_{\text{hss}}$ -simple Splittable zkSNARK for $\mathbb{C}$ -circuit satisfiability. Then Π_{czk} (Figure 17) realizes $\mathcal{F}_{\text{czk}}^{\text{SNARK}}$ (Figure 16) in the $\{\mathcal{F}_{\text{NIZK}}, \mathcal{F}_{\text{pkgen}}\}$ -hybrid model for any relation $\mathcal{R}$, where the relation circuit $C_{\mathcal{R}} \in \mathbb{C}$ satisfies*

Protocol Π_{czk}

Π_{czk} is described in the $\{\mathcal{F}_{\text{NIZK}}, \mathcal{F}_{\text{pkgen}}\}$ -hybrid (Figures 9 and 12) and runs with provers $\mathcal{P}_c = \{P_1^c, \dots, P_n^c\}$ and servers $\mathcal{P} = \{P_1, \dots, P_m\}$ in the presence of a static adversary $\mathcal{A}$ that corrupts a subset of servers $\mathcal{I} \subset [1, m]$ and provers $\mathcal{I}_c \subset [1, n]$. Each prover P_i^c has private witness w_i and all provers have the public statement $\mathbf{x}$ and a CRS crs .

Public parameters. Let HSS be an m -server public-key HSS scheme that supports evaluating circuits over a finite commutative ring R and let SNARK be a Splittable zkSNARK. Let ℓ_{rand} denote the number of random elements from R required by the zkSNARK prover. Let $\mathcal{R}$ be an NP relation.

Notation. Let $C_{\mathcal{R}}$ be the relation-checking circuit for $\mathcal{R}$. Let C_{NL} denote the circuit that computes the non-linear phase Prove_{NL} of the Splittable zkSNARK for the circuit satisfiability statement $(C_{\mathcal{R}}, \mathbf{x}, y) \in \text{SAT}$. Let $\widehat{C}_{\text{NL}}$ denote the circuit that evaluates C_{NL} using randomness derived from the sum of the last n inputs and outputs $C_{\mathcal{R}}(\mathbf{x}, \mathbf{w}) \cdot C_{\text{NL}}(\mathbf{x}, \mathbf{w})$, for $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}$. Let $\widehat{C}$ be a circuit such that $(\Gamma_y^1, \dots, \Gamma_y^m) = \widehat{C}(\mathbf{x}, \mathbf{w}, K_1, \dots, K_m)$, where $y = C_{\mathcal{R}}(\mathbf{x}, \mathbf{w})$ and $\Gamma_y^i = y \cdot K_i$ for each $i \in [1, m]$.

- 1: Each prover and server invokes $\mathcal{F}_{\text{pkgen}}^{\text{HSS}}$ to receive pk and the i -th server P_i additionally receives its evaluation key ek_i .
- 2: The servers and provers do the following.
 1. Each server P_i samples $K_i \leftarrow R$ uniformly at random, computes $\widetilde{K}_i \leftarrow \text{HSS.Enc}(\text{pk}, K_i)$ and a proof of correctness $\pi_{K_i}^{\text{inp}}$ for the statement $(\text{pk}, R, \widetilde{K}_i)$ using $\mathcal{F}_{\text{NIZK}}$.
 2. Each prover P_i^c samples $\mathbf{r}_i \leftarrow R^{\ell_{\text{rand}}}$ uniformly at random. It then computes $\widetilde{\mathbf{r}}_i \leftarrow \text{HSS.Enc}(\text{pk}, \mathbf{r}_i)$ and $\widetilde{w}_i \leftarrow \text{HSS.Enc}(\text{pk}, w_i)$, and proofs of correct encryption $\pi_{\mathbf{r}_i}^{\text{inp}}$ and $\pi_{w_i}^{\text{inp}}$ using $\mathcal{F}_{\text{NIZK}}$.
 3. Each server P_i broadcasts $(\widetilde{K}_i, \pi_{K_i}^{\text{inp}})$ to all servers and each prover P_i^c broadcasts $(\text{crs}, \mathbf{x}, \widetilde{\mathbf{r}}_i, \widetilde{w}_i, \pi_{\mathbf{r}_i}^{\text{inp}})$ to all servers.
- 3: Each server P_i uses $\mathcal{F}_{\text{NIZK}}$ to verify the proofs sent by all provers and servers, and aborts if any proof does not verify. Moreover, if the CRS or statement sent by all provers is not identical it aborts. Else, it computes the following.

$$\begin{aligned} \left(\left[\Gamma_y^1 \right]_i, \dots, \left[\Gamma_y^m \right]_i \right) &:= \text{HSS.Eval}(\text{ek}_i, \widehat{C}, \mathbf{x}, \widetilde{w}_1, \dots, \widetilde{w}_n, \widetilde{K}_1, \dots, \widetilde{K}_n), \\ [y \cdot M_{\text{NL}}]_i &:= \text{HSS.Eval}(\text{ek}_i, \widehat{C}_{\text{NL}}, \mathbf{x}, \widetilde{w}_1, \dots, \widetilde{w}_n, \widetilde{\mathbf{r}}_1, \dots, \widetilde{\mathbf{r}}_n), \\ [y \cdot \pi]_i &:= \text{SNARK.Prove}_{\text{L}}(\text{crs}, [y \cdot M_{\text{NL}}]_i). \end{aligned}$$

For each $j \in [1, m]$, it sends $\left[\Gamma_y^j \right]_i$ to the j -th server P_j .

- 4: Each server P_i receives $\left[\Gamma_y^j \right]_i$ and reconstructs it compute Γ_y^i . It then checks if $\Gamma_y^i = K_i$; if so, it broadcasts $[y \cdot \pi]_i$, else it aborts.
- 5: Provers reconstruct $[y \cdot \pi]$ to either obtain 0 or the proof π . If they obtain 0 or if $\text{SNARK.Verify}(\text{crs}, \mathbf{x}, \pi) = 0$ then they output $\perp$, else they output π .

Figure 17: Collaborative zkSNARK protocol for Splittable zkSNARKs using a public-key HSS scheme.

$\widehat{C}, \widehat{C}_{NL} \in \mathbb{C}_{\text{hss}}$ for $\widehat{C}$ and $\widehat{C}_{NL}$ defined in [Figure 17](#). Additionally, Π_{czk} is black-box in HSS and the linear phase of SNARK.

Proof Sketch. The proof proceeds similar to that of [Theorem 15](#) and we only discuss the key changes to the simulator here. The simulator works by first simulating the inputs of honest provers, relying on the security of the HSS scheme, and uses $\mathcal{F}_{\text{NIZK}}$ to extract the inputs of corrupt provers. It then sends the extracted inputs to the ideal functionality and obtains the output. It also computes the expected output shares of corrupt servers, which subsequently allows it to verify if the MAC check passes for honest servers. If the ideal functionality returns $\perp$, it reverse samples the shares of the honest servers so that the output reconstructs to 0. Else, it reverse samples the shares of the honest servers so that the output reconstructs to the proof π returned by the ideal functionality. Finally, upon receiving the share sent by the corrupt servers, it can reconstruct the output, using the reverse sampled shares of the honest servers, and check if the reconstructed proof verifies since it knows the statement. ■

Similar to [Corollary 19](#), it follows that the Groth16 zkSNARK can be computed collaboratively for NC^1 relations in the random oracle model. Moreover, if provers are semi-honest, we no longer require $\mathcal{F}_{\text{NIZK}}$ and the semi-malicious correctness of the HSS scheme (see [Section 6.2](#)), and it follows from [Corollary 17](#) that the Groth16 zkSNARK can be computed collaboratively for NC^1 and P/poly from a much wider range of assumptions.

Acknowledgements

A majority of this work was done while the first and second authors were at NTT Research. The third and fourth authors were supported in part by NSF CNS-1814919, NSF CAREER 1942789 and Johns Hopkins University Catalyst award. The fourth author was additionally supported in part by JP Morgan Faculty Award, and research gifts from Ethereum, Stellar and Cisco.

References

- [ACGS23] Diego F. Aranha, Anamaria Costache, Antonio Guimarães, and Eduardo Soria-Vazquez. HE-LIOPOLIS: Verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical. *Cryptology ePrint Archive*, Report 2023/1949, 2023. [5](#)
- [ADOS22] Damiano Abram, Ivan Damgård, Claudio Orlandi, and Peter Scholl. An algebraic framework for silent preprocessing with trustless setup and active security. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 421–452, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland. [1](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#), [17](#), [18](#), [19](#), [22](#), [23](#), [48](#), [55](#), [62](#), [63](#)
- [BBC⁺19] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Zero-knowledge proofs on secret-shared data via fully linear PCPs. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 67–97, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland. [7](#)
- [BBC⁺21] Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Lightweight techniques for private heavy hitters. In *2021 IEEE Symposium on Security and Privacy*, pages 762–776, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press. [6](#)
- [BCC⁺17] Nir Bitansky, Ran Canetti, Alessandro Chiesa, Shafi Goldwasser, Huijia Lin, Aviad Rubinfeld, and Eran Tromer. The hunting of the SNARK. *Journal of Cryptology*, 30(4):989–1066, October 2017. [8](#)
- [BCCT13] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. Recursive composition and bootstrapping for SNARKS and proof-carrying data. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th Annual ACM Symposium on Theory of Computing*, pages 111–120, Palo Alto, CA, USA, June 1–4, 2013. ACM Press. [8](#)
- [BCG⁺17] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, and Michele Orrù. Homomorphic secret sharing: Optimizations and applications. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 2105–2122, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press. [1](#), [8](#)
- [BCG⁺19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 489–518, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland. [1](#), [8](#)
- [BCGI18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 896–912, Toronto, ON, Canada, October 15–19, 2018. ACM Press. [1](#)
- [BCH⁺22] Nir Bitansky, Arka Rai Choudhuri, Justin Holmgren, Chethan Kamath, Alex Lombardi, Omer Paneth, and Ron D. Rothblum. PPAD is as hard as LWE and iterated squaring. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022: 20th Theory of Cryptography Conference, Part II*,

volume 13748 of *Lecture Notes in Computer Science*, pages 593–622, Chicago, IL, USA, November 7–10, 2022. Springer, Cham, Switzerland. [12](#)

- [BCI⁺13] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. Succinct non-interactive arguments via linear interactive proofs. In Amit Sahai, editor, *TCC 2013: 10th Theory of Cryptography Conference*, volume 7785 of *Lecture Notes in Computer Science*, pages 315–333, Tokyo, Japan, March 3–6, 2013. Springer, Berlin, Heidelberg, Germany. [8](#), [12](#), [27](#)
- [BDO14] Carsten Baum, Ivan Damgård, and Claudio Orlandi. Publicly auditable secure multi-party computation. In Michel Abdalla and Roberto De Prisco, editors, *SCN 14: 9th International Conference on Security in Communication Networks*, volume 8642 of *Lecture Notes in Computer Science*, pages 175–196, Amalfi, Italy, September 3–5, 2014. Springer, Cham, Switzerland. [54](#)
- [BDOZ11] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In Kenneth G. Paterson, editor, *Advances in Cryptology – EUROCRYPT 2011*, volume 6632 of *Lecture Notes in Computer Science*, pages 169–188, Tallinn, Estonia, May 15–19, 2011. Springer, Berlin, Heidelberg, Germany. [55](#)
- [BG10] Zvika Brakerski and Shafi Goldwasser. Circular and leakage resilient public-key encryption under subgroup indistinguishability - (or: Quadratic residuosity strikes back). In Tal Rabin, editor, *Advances in Cryptology – CRYPTO 2010*, volume 6223 of *Lecture Notes in Computer Science*, pages 1–20, Santa Barbara, CA, USA, August 15–19, 2010. Springer, Berlin, Heidelberg, Germany. [23](#)
- [BGI15] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *Lecture Notes in Computer Science*, pages 337–367, Sofia, Bulgaria, April 26–30, 2015. Springer, Berlin, Heidelberg, Germany. [3](#), [4](#), [24](#), [48](#), [62](#)
- [BGI16a] Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under DDH. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part I*, volume 9814 of *Lecture Notes in Computer Science*, pages 509–539, Santa Barbara, CA, USA, August 14–18, 2016. Springer, Berlin, Heidelberg, Germany. [1](#), [8](#), [20](#), [21](#), [22](#), [23](#), [48](#), [50](#)
- [BGI16b] Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1292–1303, Vienna, Austria, October 24–28, 2016. ACM Press. [6](#)
- [BGI17] Elette Boyle, Niv Gilboa, and Yuval Ishai. Group-based secure computation: Optimizing rounds, communication, and computation. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part II*, volume 10211 of *Lecture Notes in Computer Science*, pages 163–193, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland. [1](#), [8](#)
- [BGI⁺18] Elette Boyle, Niv Gilboa, Yuval Ishai, Huijia Lin, and Stefano Tessaro. Foundations of homomorphic secret sharing. In Anna R. Karlin, editor, *ITCS 2018: 9th Innovations in Theoretical Computer Science Conference*, volume 94, pages 21:1–21:21, Cambridge, MA, USA, January 11–14, 2018. LIPIcs. [3](#), [4](#), [24](#)

- [BGIN19] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Practical fully secure three-party computation via sublinear distributed zero-knowledge proofs. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 869–886, London, UK, November 11–15, 2019. ACM Press. [7](#)
- [BGIN20] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Efficient fully secure computation via distributed zero-knowledge proofs. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020, Part III*, volume 12493 of *Lecture Notes in Computer Science*, pages 244–276, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland. [7](#)
- [BGIN21] Elette Boyle, Niv Gilboa, Yuval Ishai, and Ariel Nof. Sublinear GMW-style compiler for MPC with preprocessing. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part II*, volume 12826 of *Lecture Notes in Computer Science*, pages 457–485, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. [7](#)
- [BGMM20] James Bartusek, Sanjam Garg, Daniel Masny, and Pratyay Mukherjee. Reusable two-round MPC from DDH. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part II*, volume 12551 of *Lecture Notes in Computer Science*, pages 320–348, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland. [1](#)
- [BGN05] Dan Boneh, Eu-Jin Goh, and Kobbi Nissim. Evaluating 2-DNF formulas on ciphertexts. In Joe Kilian, editor, *TCC 2005: 2nd Theory of Cryptography Conference*, volume 3378 of *Lecture Notes in Computer Science*, pages 325–341, Cambridge, MA, USA, February 10–12, 2005. Springer, Berlin, Heidelberg, Germany. [14](#), [34](#)
- [BHK17] Zvika Brakerski, Justin Holmgren, and Yael Tauman Kalai. Non-interactive delegation and batch NP verification from standard computational assumptions. In Hamed Hatami, Pierre McKenzie, and Valerie King, editors, *49th Annual ACM Symposium on Theory of Computing*, pages 474–482, Montreal, QC, Canada, June 19–23, 2017. ACM Press. [4](#)
- [BKP⁺23] Nir Bitansky, Chethan Kamath, Omer Paneth, Ron Rothblum, and Prashant Nalini Vasudevan. Batch proofs are statistically hiding. *Cryptology ePrint Archive*, Report 2023/754, 2023. [4](#)
- [BKS19] Elette Boyle, Lisa Kohl, and Peter Scholl. Homomorphic secret sharing from lattices without FHE. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 3–33, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland. [1](#), [3](#), [4](#), [7](#), [8](#), [22](#), [23](#), [48](#), [62](#)
- [BWW23] Eli Bradley, Brent Waters, and David J. Wu. Batch arguments to NIZKs from one-way functions. *Cryptology ePrint Archive*, Report 2023/1938, 2023. [4](#)
- [CGH⁺18] Koji Chida, Daniel Genkin, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Yehuda Lindell, and Ariel Nof. Fast large-scale honest-majority MPC for malicious adversaries. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018, Part III*, volume 10993 of *Lecture Notes in Computer Science*, pages 34–64, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland. [7](#)
- [CGJ⁺23] Arka Rai Choudhuri, Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Jiaheng Zhang. Correlation intractability and SNARGs from sub-exponential DDH. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084

- of *Lecture Notes in Computer Science*, pages 635–668, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland. [4](#), [14](#)
- [Che23] Xin Chen. Verifiable homomorphic secret sharing for machine learning classifiers. *IEEE Access*, 11:43639–43647, 2023. [6](#), [7](#)
- [CHK⁺19a] Arka Rai Choudhuri, Pavel Hubáček, Chethan Kamath, Krzysztof Pietrzak, Alon Rosen, and Guy N. Rothblum. Finding a nash equilibrium is no easier than breaking Fiat-Shamir. In Moses Charikar and Edith Cohen, editors, *51st Annual ACM Symposium on Theory of Computing*, pages 1103–1114, Phoenix, AZ, USA, June 23–26, 2019. ACM Press. [12](#)
- [CHK⁺19b] Arka Rai Choudhuri, Pavel Hubacek, Chethan Kamath, Krzysztof Pietrzak, Alon Rosen, and Guy N. Rothblum. PPAD-hardness via iterated squaring modulo a composite. *Cryptology ePrint Archive*, Report 2019/667, 2019. [12](#)
- [CJJ21] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. Non-interactive batch arguments for NP from standard assumptions. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part IV*, volume 12828 of *Lecture Notes in Computer Science*, pages 394–423, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. [4](#), [14](#)
- [CJJ22] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. SNARGs for $\mathcal{P}$ from LWE. In *62nd Annual Symposium on Foundations of Computer Science*, pages 68–79, Denver, CO, USA, February 7–10, 2022. IEEE Computer Society Press. [4](#), [14](#), [37](#)
- [CL15] Guilhem Castagnos and Fabien Laguillaumie. Linearly homomorphic encryption from DDH. *Cryptology ePrint Archive*, Report 2015/047, 2015. [23](#)
- [Cle90] Richard Cleve. Towards optimal simulations of formulas by bounded-width programs. In *22nd Annual ACM Symposium on Theory of Computing*, pages 271–277, Baltimore, MD, USA, May 14–16, 1990. ACM Press. [22](#)
- [CLMZ23] Alessandro Chiesa, Ryan Lehmkuhl, Pratyush Mishra, and Yinuo Zhang. Eos: Efficient private delegation of zkSNARK provers. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023: 32nd USENIX Security Symposium*, pages 6453–6469, Anaheim, CA, USA, August 9–11, 2023. USENIX Association. [5](#), [6](#), [18](#), [19](#)
- [CM21] Geoffroy Couteau and Pierre Meyer. Breaking the circuit size barrier for secure computation under quasi-polynomial LPN. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 842–870, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland. [1](#), [7](#), [8](#)
- [CMPR23] Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Constrained pseudorandom functions from homomorphic secret sharing. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 194–224, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland. [1](#)
- [COS⁺22] Ilaria Chillotti, Emmanuela Orsini, Peter Scholl, Nigel P. Smart, and Barry Van Leeuwen. Scooby: Improved multi-party homomorphic secret sharing based on FHE. In Clemente Galdi and Stanislaw Jarecki, editors, *SCN 22: 13th International Conference on Security in Communication Networks*, volume 13409 of *Lecture Notes in Computer Science*, pages 540–563, Amalfi, Italy, September 12–14, 2022. Springer, Cham, Switzerland. [8](#), [24](#), [48](#), [62](#)

- [Cou19] Geoffroy Couteau. A note on the communication complexity of multiparty computation in the correlated randomness model. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 473–503, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland. [1](#)
- [CW23] Jeffrey Champion and David J. Wu. Non-interactive zero-knowledge from non-interactive batch arguments. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 38–71, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland. [4](#)
- [CZ20] Xin Chen and Liang Feng Zhang. Two-server verifiable homomorphic secret sharing for high-degree polynomials. In Willy Susilo, Robert H. Deng, Fuchun Guo, Yannan Li, and Rolly Intan, editors, *ISC 2020: 23rd International Conference on Information Security*, volume 12472 of *Lecture Notes in Computer Science*, pages 75–91, Bali, Indonesia, December 16–18, 2020. Springer, Cham, Switzerland. [6](#), [7](#)
- [CZ21] Xin Chen and Liang Feng Zhang. Two-server delegation of computation on label-encrypted data. *IEEE Trans. Cloud Comput.*, 9(4):1645–1656, 2021. [6](#), [7](#)
- [dCP22] Leo de Castro and Antigoni Polychroniadou. Lightweight, maliciously secure verifiable function secret sharing. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 150–179, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland. [6](#)
- [DFH12] Ivan Damgård, Sebastian Faust, and Carmit Hazay. Secure two-party computation with low communication. In Ronald Cramer, editor, *TCC 2012: 9th Theory of Cryptography Conference*, volume 7194 of *Lecture Notes in Computer Science*, pages 54–74, Taormina, Sicily, Italy, March 19–21, 2012. Springer, Berlin, Heidelberg, Germany. [8](#)
- [DHRW16] Yevgeniy Dodis, Shai Halevi, Ron D. Rothblum, and Daniel Wichs. Spooky encryption and its applications. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part III*, volume 9816 of *Lecture Notes in Computer Science*, pages 93–122, Santa Barbara, CA, USA, August 14–18, 2016. Springer, Berlin, Heidelberg, Germany. [1](#), [3](#), [4](#), [24](#)
- [DIJL23] Quang Dao, Yuval Ishai, Aayush Jain, and Huijia Lin. Multi-party homomorphic secret sharing and sublinear MPC from sparse LPN. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part II*, volume 14082 of *Lecture Notes in Computer Science*, pages 315–348, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland. [1](#), [7](#), [8](#), [24](#)
- [DJ01] Ivan Damgård and Mats Jurik. A generalisation, a simplification and some applications of Paillier’s probabilistic public-key system. In Kwangjo Kim, editor, *PKC 2001: 4th International Workshop on Theory and Practice in Public Key Cryptography*, volume 1992 of *Lecture Notes in Computer Science*, pages 119–136, Cheju Island, South Korea, February 13–15, 2001. Springer, Berlin, Heidelberg, Germany. [23](#)
- [DPSZ12] Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 643–662, Santa Barbara, CA, USA, August 19–23, 2012. Springer, Berlin, Heidelberg, Germany. [7](#)

- [EFKP20] Naomi Ephraim, Cody Freitag, Ilan Komargodski, and Rafael Pass. Continuous verifiable delay functions. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part III*, volume 12107 of *Lecture Notes in Computer Science*, pages 125–154, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland. [12](#)
- [FGJS17] Nelly Fazio, Rosario Gennaro, Tahereh Jafarikhah, and William E. Skeith III. Homomorphic secret sharing from paillier encryption. In Tatsuaki Okamoto, Yong Yu, Man Ho Au, and Yannan Li, editors, *ProvSec 2017: 11th International Conference on Provable Security*, volume 10592 of *Lecture Notes in Computer Science*, pages 381–399, Xi’an, China, October 23–25, 2017. Springer, Cham, Switzerland. [8](#)
- [FIKW22] Ingerid Fosli, Yuval Ishai, Victor I. Kolobov, and Mary Wootters. On the download rate of homomorphic secret sharing. In Mark Braverman, editor, *ITCS 2022: 13th Innovations in Theoretical Computer Science Conference*, volume 215, pages 71:1–71:22, Berkeley, CA, USA, January 31 – February 3, 2022. LIPIcs. [8](#)
- [GGJ⁺23] Sanjam Garg, Aarushi Goel, Abhishek Jain, Guru-Vamsi Policharla, and Sruthi Sekar. zkSaaS: Zero-knowledge SNARKs as a service. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023: 32nd USENIX Security Symposium*, pages 4427–4444, Anaheim, CA, USA, August 9–11, 2023. USENIX Association. [5](#), [6](#), [18](#), [19](#)
- [GGP10] Rosario Gennaro, Craig Gentry, and Bryan Parno. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In Tal Rabin, editor, *Advances in Cryptology – CRYPTO 2010*, volume 6223 of *Lecture Notes in Computer Science*, pages 465–482, Santa Barbara, CA, USA, August 15–19, 2010. Springer, Berlin, Heidelberg, Germany. [63](#)
- [GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct NIZKs without PCPs. In Thomas Johansson and Phong Q. Nguyen, editors, *Advances in Cryptology – EUROCRYPT 2013*, volume 7881 of *Lecture Notes in Computer Science*, pages 626–645, Athens, Greece, May 26–30, 2013. Springer, Berlin, Heidelberg, Germany. [8](#), [12](#), [30](#)
- [GGW24] Sanjam Garg, Aarushi Goel, and Mingyuan Wang. How to prove statements obliviously? In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part X*, volume 14929 of *Lecture Notes in Computer Science*, pages 449–487, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland. [5](#), [6](#), [18](#), [19](#)
- [GI14] Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 640–658, Copenhagen, Denmark, May 11–15, 2014. Springer, Berlin, Heidelberg, Germany. [1](#)
- [GIP⁺14] Daniel Genkin, Yuval Ishai, Manoj Prabhakaran, Amit Sahai, and Eran Tromer. Circuits resilient to additive attacks with applications to secure computation. In David B. Shmoys, editor, *46th Annual ACM Symposium on Theory of Computing*, pages 495–504, New York, NY, USA, May 31 – June 3, 2014. ACM Press. [7](#)
- [GMR89] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM J. Comput.*, 18(1):186–208, 1989. [7](#)

- [GMW87] Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th Annual ACM Symposium on Theory of Computing*, pages 218–229, New York City, NY, USA, May 25–27, 1987. ACM Press. [7](#), [10](#)
- [Gro10] Jens Groth. Short pairing-based non-interactive zero-knowledge arguments. In Masayuki Abe, editor, *Advances in Cryptology – ASIACRYPT 2010*, volume 6477 of *Lecture Notes in Computer Science*, pages 321–340, Singapore, December 5–9, 2010. Springer, Berlin, Heidelberg, Germany. [8](#)
- [Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *Advances in Cryptology – EUROCRYPT 2016, Part II*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326, Vienna, Austria, May 8–12, 2016. Springer, Berlin, Heidelberg, Germany. [2](#), [3](#), [4](#), [8](#), [12](#), [30](#), [31](#), [32](#), [33](#)
- [GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019. [8](#)
- [HJKS22] James Hulett, Ruta Jawale, Dakshita Khurana, and Akshayaram Srinivasan. SNARGs for P from sub-exponential DDH and QR. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part II*, volume 13276 of *Lecture Notes in Computer Science*, pages 520–549, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland. [4](#), [14](#)
- [HZ20] Yan He and Liang Feng Zhang. Cheater-identifiable homomorphic secret sharing for outsourcing computations. *J. Ambient Intell. Humaniz. Comput.*, 11(11):5103–5113, 2020. [6](#)
- [ILM21] Yuval Ishai, Russell W. F. Lai, and Giulio Malavolta. A geometric approach to homomorphic secret sharing. In Juan Garay, editor, *PKC 2021: 24th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 12711 of *Lecture Notes in Computer Science*, pages 92–119, Virtual Event, May 10–13, 2021. Springer, Cham, Switzerland. [8](#)
- [JKKZ21] Ruta Jawale, Yael Tauman Kalai, Dakshita Khurana, and Rachel Yun Zhang. SNARGs for bounded depth computations and PPAD hardness from sub-exponential LWE. In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd Annual ACM Symposium on Theory of Computing*, pages 708–721, Virtual Event, Italy, June 21–25, 2021. ACM Press. [12](#)
- [Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments (extended abstract). In *24th Annual ACM Symposium on Theory of Computing*, pages 723–732, Victoria, BC, Canada, May 4–6, 1992. ACM Press. [7](#)
- [KPY19] Yael Tauman Kalai, Omer Paneth, and Lisa Yang. How to delegate computations publicly. In Moses Charikar and Edith Cohen, editors, *51st Annual ACM Symposium on Theory of Computing*, pages 1115–1124, Phoenix, AZ, USA, June 23–26, 2019. ACM Press. [4](#)
- [KPY20] Yael Tauman Kalai, Omer Paneth, and Lisa Yang. Delegation with updatable unambiguous proofs and PPAD-hardness. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part III*, volume 12172 of *Lecture Notes in Computer Science*, pages 652–673, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland. [12](#)

- [Lip12] Helger Lipmaa. Progression-free sets and sublinear pairing-based non-interactive zero-knowledge arguments. In Ronald Cramer, editor, *TCC 2012: 9th Theory of Cryptography Conference*, volume 7194 of *Lecture Notes in Computer Science*, pages 169–189, Taormina, Sicily, Italy, March 19–21, 2012. Springer, Berlin, Heidelberg, Germany. [8](#)
- [LMS18] Russell W. F. Lai, Giulio Malavolta, and Dominique Schröder. Homomorphic secret sharing for low degree polynomials. In Thomas Peyrin and Steven Galbraith, editors, *Advances in Cryptology – ASIACRYPT 2018, Part III*, volume 11274 of *Lecture Notes in Computer Science*, pages 279–309, Brisbane, Queensland, Australia, December 2–6, 2018. Springer, Cham, Switzerland. [8](#)
- [LP11] Yehuda Lindell and Benny Pinkas. Secure two-party computation via cut-and-choose oblivious transfer. In Yuval Ishai, editor, *TCC 2011: 8th Theory of Cryptography Conference*, volume 6597 of *Lecture Notes in Computer Science*, pages 329–346, Providence, RI, USA, March 28–30, 2011. Springer, Berlin, Heidelberg, Germany. [10](#)
- [LV20] Alex Lombardi and Vinod Vaikuntanathan. Fiat-shamir for repeated squaring with applications to PPAD-hardness and VDFs. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part III*, volume 12172 of *Lecture Notes in Computer Science*, pages 632–651, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland. [12](#)
- [LZW⁺24] Xuanming Liu, Zhelei Zhou, Yinghao Wang, Bingsheng Zhang, and Xiaohu Yang. Scalable collaborative zk-SNARK: Fully distributed proof generation and malicious security. *Cryptology ePrint Archive*, Report 2024/143, 2024. [5](#), [6](#), [18](#), [19](#)
- [Mic94] Silvio Micali. CS proofs (extended abstracts). In *35th Annual Symposium on Foundations of Computer Science*, pages 436–453, Santa Fe, NM, USA, November 20–22, 1994. IEEE Computer Society Press. [2](#), [7](#), [8](#)
- [MTG22] Arup Mondal, Pratyush Ranjan Tiwari, and Debayan Gupta. Poster: Fully homomorphic secret sharing with output verifiability. *NDSS Poster*, 2022. [6](#)
- [OB22] Alex Ozdemir and Dan Boneh. Experimenting with collaborative zk-SNARKs: Zero-knowledge proofs for distributed secrets. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022: 31st USENIX Security Symposium*, pages 4291–4308, Boston, MA, USA, August 10–12, 2022. USENIX Association. [5](#), [6](#), [13](#), [19](#), [64](#)
- [OSY21] Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 678–708, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland. [1](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#), [18](#), [22](#), [23](#), [48](#), [62](#)
- [RRR21] Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. Constant-round interactive proofs for delegating computation. *SIAM J. Comput.*, 50(3), 2021. [9](#), [11](#)
- [RS21] Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 687–717, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. [1](#), [3](#), [4](#), [5](#), [6](#), [7](#), [8](#), [18](#), [22](#), [23](#), [48](#), [55](#), [62](#), [63](#)

- [TBM20] Georgia Tsaloli, Gustavo Banegas, and Aikaterini Mitrokotsa. Practical and provably secure distributed aggregation: Verifiable additive homomorphic secret sharing. *Cryptogr.*, 4(3):25, 2020. [6](#)
- [TLM18] Georgia Tsaloli, Bei Liang, and Aikaterini Mitrokotsa. Verifiable homomorphic secret sharing. In Joonsang Baek, Willy Susilo, and Jongkil Kim, editors, *ProvSec 2018: 12th International Conference on Provable Security*, volume 11192 of *Lecture Notes in Computer Science*, pages 40–55, Jeju, South Korea, October 25–28, 2018. Springer, Cham, Switzerland. [6](#)
- [TM19] Georgia Tsaloli and Aikaterini Mitrokotsa. Sum it up: Verifiable additive homomorphic secret sharing. In Jae Hong Seo, editor, *ICISC 19: 22nd International Conference on Information Security and Cryptology*, volume 11975 of *Lecture Notes in Computer Science*, pages 115–132, Seoul, Korea, December 4–6, 2019. Springer, Cham, Switzerland. [6](#)
- [vzGS91] Joachim von zur Gathen and Gadiel Seroussi. Boolean circuits versus arithmetic circuits. *Inf. Comput.*, 91(1):142–154, mar 1991. [50](#)
- [WW22] Brent Waters and David J. Wu. Batch arguments for NP and more from standard bilinear group assumptions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part II*, volume 13508 of *Lecture Notes in Computer Science*, pages 433–463, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland. [3](#), [4](#), [12](#), [14](#), [34](#), [35](#), [36](#)
- [YO19] Maki Yoshida and Satoshi Obana. Verifiably multiplicative secret sharing. *IEEE Trans. Inf. Theory*, 65(5):3233–3245, 2019. [6](#), [7](#)
- [ZW22] Liang Feng Zhang and Huaxiong Wang. Multi-server verifiable computation of low-degree polynomials. In *2022 IEEE Symposium on Security and Privacy*, pages 596–613, San Francisco, CA, USA, May 22–26, 2022. IEEE Computer Society Press. [6](#), [7](#)